

LGT-ARCH-001 - Living Guide Technology Reference Architecture

Publication Version: 1.0.0

Framework Version: 1.0.0

Classification: Normative

Status: QA Ready - Incremental Release

Release Date: 2026-08-10

Reference Implementation: ConjuresUp

1. Purpose and Normative Scope

1.1 Purpose

LGT-ARCH-001 defines the technology-independent reference architecture through which Living Guide Technology constitutional principles and normative component specifications operate as one coherent system.

1.2 Architectural role

The Reference Architecture SHALL define relationships, responsibilities, information movement, correction paths, and conformance boundaries between LGT components without replacing the normative authority of those component specifications.

1.3 Technology neutrality

LGT-ARCH-001 SHALL NOT prescribe a programming language, database, model provider, hosting platform, user interface framework, deployment topology, or vendor-specific implementation.

1.4 Reference implementation distinction

ConjuresUp is the first reference implementation of Living Guide Technology. ConjuresUp SHALL NOT be treated as the definition of LGT. An implementation using materially different technologies MAY conform when it satisfies the Constitution, this Reference Architecture, and the applicable normative component specifications.

1.5 Constitutional authority

The Living Guide Technology Constitution remains the highest governing architectural authority. Where an implementation interpretation conflicts with the Constitution, the constitutional requirement governs.

1.6 Component authority

Each normative component specification remains authoritative within its defined responsibility. LGT-ARCH-001 coordinates those responsibilities but SHALL NOT silently redefine them.

1.7 Integrated publication scope

Version 0.14.0 consolidates the complete pre-release-candidate Reference Architecture: architectural identity and authority; accountable forward and reverse flows; information envelopes and lifecycle coordination; failure containment; constitutional and component conformance mapping; interoperability; privacy and member control; observability and explanation; security and trust boundaries; performance and resource governance; deployment and production change; testing and conformance verification; and profiles, extension points, implementation boundaries, and capability declaration. The architecture remains technology-neutral and does not make ConjuresUp or any particular implementation stack the conformance template.

2. Architectural Authority and Layering

2.1 Governing hierarchy

A conforming LGT architecture SHALL distinguish at least four conceptual layers: constitutional governance, reference architecture, normative component specifications, and implementation.

2.2 Constitutional governance layer

The Constitution defines non-negotiable principles governing person-centricity, evidence, memory, confidence, continuity, reflection, companionship, correction, member control, and ethical alignment.

2.3 Reference architecture layer

LGT-ARCH-001 defines how normative responsibilities connect without collapsing them into one subsystem.

2.4 Component specification layer

Component specifications define the normative behavior of distinct architectural responsibilities, including observation, evidence, memory, continuity intelligence, reflection, and companion behavior.

2.5 Implementation layer

Implementations select technologies and operational designs that satisfy the higher layers. Implementation convenience SHALL NOT override architectural boundaries.

2.6 Profiles and extensions

Conformance profiles and implementation-specific extensions MAY add constraints or capabilities but SHALL NOT weaken constitutional or normative component requirements.

2.7 No authority by proximity

A component SHALL NOT acquire another component's authority merely because both operate within the same process, model, database, service, or codebase.

3. Core Normative Components

3.1 Observation - LGT-OBS-001

The Observation responsibility governs accountable capture and representation of what occurred before interpretation. Architectural consumers SHALL NOT convert derived interpretation back into raw observation merely for convenience.

3.2 Evidence - LGT-EVD-001

The Evidence responsibility governs how eligible observations and other accountable inputs contribute support, contradiction, confidence, independence, uncertainty, and sufficiency.

3.3 Memory - LGT-MEM-001

The Memory responsibility governs eligibility, retention, authorization, correction, retrieval, provenance, and forgetting. Storage alone SHALL NOT constitute authorized memory use.

3.4 Continuity Intelligence - LGT-CIN-001

The Continuity Intelligence responsibility relates eligible evidence and memory across time while preserving temporal change, contradiction, provenance, correction, integrity, maturity, and downstream readiness.

3.5 Mirror - LGT-MIR-001

The Mirror responsibility governs member-facing reflection. It determines how eligible understanding may be expressed without converting interpretation into authority, identity, diagnosis, certainty, or destiny.

3.6 Companion - LGT-CMP-001

The Companion responsibility governs relational behavior and interaction continuity. Familiarity, trust, or engagement SHALL NOT substitute for evidence or expand architectural authority.

3.7 Component separability

These responsibilities MAY share infrastructure, but their normative distinctions SHALL remain reviewable and enforceable.

3.8 No monolithic reasoning exception

An implementation using a single model or service SHALL still preserve the logical boundaries required by the component specifications. A monolithic runtime is not an exemption from modular accountability.

4. Primary Accountable Flow

4.1 Observation before interpretation

The primary flow SHALL begin with accountable observations or other explicitly eligible inputs rather than unsupported interpretation.

4.2 Evidence formation

Observation becomes evidentiary support only through the applicable evidence rules. Repetition or system restatement SHALL NOT manufacture independent support.

4.3 Memory eligibility

Information entering durable memory SHALL remain subject to memory eligibility, authorization, provenance, retention, and correction requirements.

4.4 Continuity formation

CIN MAY relate eligible evidence and memory across time only within supported domain, temporal, authorization, and integrity boundaries.

4.5 Downstream readiness

Continuity SHALL NOT flow downstream merely because it exists. CIN SHALL determine whether it is sufficiently mature and appropriate for the specified downstream purpose.

4.6 Reflection flow

Member-facing reflection SHALL pass through MIR responsibilities before presentation as a reflection of accumulated understanding.

4.7 Companion flow

Relational presentation or interaction behavior SHALL remain governed by CMP and SHALL NOT amplify the certainty or authority of upstream understanding.

4.8 Traceable transformations

Material transformations between architectural responsibilities SHOULD remain attributable so review can determine what entered, what changed, and which component was responsible.

5. Reverse Correction and Reconciliation Flow

5.1 Architecture is not one-way

LGT SHALL support reverse correction and reconciliation paths. Later understanding SHALL NOT become permanently detached from corrections to earlier evidence or memory.

5.2 Observation correction

Where an underlying observation is corrected, affected evidence and downstream dependencies SHOULD be eligible for reassessment.

5.3 Evidence correction

Material evidence correction, contradiction, invalidation, or confidence change SHOULD propagate to dependent continuity where relevant.

5.4 Memory correction and revocation

Corrected, revoked, deleted, corrupted, or unauthorized memory SHALL NOT remain influential merely through a downstream derivative.

5.5 Continuity reconciliation

CIN SHALL reassess affected patterns, synthesis, salience, integrity, maturity, and downstream readiness when material dependencies change.

5.6 Reflection correction

A prior reflection MAY remain part of historical interaction while becoming superseded, corrected, or presently inapplicable. Historical preservation SHALL NOT imply continuing validity.

5.7 Companion adaptation

CMP SHALL permit relational behavior to adapt when the member corrects the Guide or when upstream understanding changes. Relationship continuity SHALL NOT become resistance to correction.

5.8 Member change

The architecture SHALL distinguish correction of a prior error from genuine member change where the evidence supports that distinction.

6. Downstream Experience Systems

6.1 Downstream status

Experience systems such as a Practice Engine MAY consume eligible LGT outputs but are not automatically core epistemic authorities within LGT.

6.2 Practice Engine relationship

A Practice Engine MAY use sufficiently ready continuity to select, sequence, or adapt experiences. It SHALL NOT redefine observation, evidence, memory, or continuity requirements.

6.3 Experience returns as evidence opportunity

Member interaction with a downstream experience MAY create new observations. Those observations SHALL re-enter the accountable architecture through the appropriate upstream responsibilities rather than automatically confirming the continuity that selected the experience.

6.4 Intervention provenance

Where the system selected or shaped an experience, that intervention SHOULD remain attributable when later outcomes are evaluated.

6.5 Anti-self-reinforcement

A downstream action SHALL NOT independently mature or validate the upstream interpretation that caused the action merely because the member engaged with it.

6.6 Reversibility

Where feasible, downstream personalization SHOULD remain adaptable when subsequent evidence contradicts or narrows its basis.

6.7 Extension systems

Additional experience, interface, hardware, or service integrations MAY connect to LGT provided they respect authorization, provenance, component boundaries, and the constitutional hierarchy.

7. Accountable Information Envelope and Inter-Component Handoff Contract

7.1 Purpose of the information envelope

When material information crosses an LGT component boundary, the receiving component SHOULD receive sufficient accountable context to use that information without silently discarding provenance, authorization, temporal meaning, uncertainty, dependencies, integrity limitations, correction relationships, or purpose restrictions.

7.2 Envelope is a logical contract

The accountable information envelope is a logical architectural contract, not a required file format, object model, database row, API payload, message bus event, or serialization syntax.

7.3 Implementations MAY distribute envelope properties

An implementation MAY store or resolve envelope properties across multiple services or records rather than transmitting every property inline, provided the receiving responsibility can reliably recover the required context when needed.

7.4 Handoff SHALL preserve meaning

A handoff SHALL NOT materially change the meaning, authority, confidence, authorization, or provenance of information merely because it crossed a component boundary.

7.5 Handoff SHALL NOT create evidence

Transport, copying, caching, indexing, summarization, retrieval, or transformation between components SHALL NOT independently create new evidentiary support.

7.6 Handoff SHALL NOT create authorization

Information becoming available to another component SHALL NOT by itself authorize that component to use the information for a new purpose.

7.7 Handoff SHALL NOT erase limitations

Known uncertainty, contradiction, integrity limitations, recovery limitations, scope restrictions, or authorization constraints SHALL remain available to downstream responsibilities where material to their use.

7.8 Identity of the information

A material handoff SHOULD provide or permit recovery of a stable identity for the handed-off state sufficient to distinguish it from unrelated, superseded, corrected, duplicated, or derived states.

7.9 Type and architectural role

A handoff SHOULD identify the architectural type or responsibility of the information sufficiently to prevent an observation from being silently treated as evidence, memory, continuity, reflection, or another state with different authority.

7.10 Provenance

A material handoff SHOULD preserve sufficient provenance to identify the accountable origin and relevant transformations of the information.

7.11 Provenance depth SHALL be proportionate

The receiving component need not receive every historical transformation inline, but the architecture SHOULD preserve enough traceability to review material dependencies when required.

7.12 Source attribution

Where information originated from the member, an external source, a system observation, a model-generated derivative, or a downstream intervention, that distinction SHOULD remain recoverable when material.

7.13 Derived-state attribution

A derived state SHOULD remain distinguishable from the source material from which it was produced.

7.14 System-generated content SHALL remain attributable

A summary, reflection, recommendation, practice selection, synthesis, or other system-generated output SHALL NOT later appear as though it were an independent member-originated observation.

7.15 Authorization context

A material handoff SHOULD preserve or permit reliable resolution of the authorization governing the receiving component's use of the information.

7.16 Purpose limitation

Authorization SHOULD be evaluated for the intended downstream purpose rather than treated as universal permission to use information wherever technically accessible.

7.17 Authorization inheritance SHALL be conservative

Derived information SHALL NOT receive broader authorization than its material supporting information merely because it is transformed.

7.18 Revocation linkage

Where authorization can be revoked, derived or dependent states SHOULD retain sufficient linkage for affected downstream use to be reassessed.

7.19 Deletion and forgetting linkage

Where information is deleted or forgotten under applicable memory rules, downstream derivatives SHALL NOT silently preserve prohibited influence.

7.20 Temporal context

A material handoff SHOULD preserve the temporal context required to distinguish when something occurred, when it was observed, when it was interpreted, and when it became applicable.

7.21 Event time and processing time MAY differ

The architecture SHOULD permit distinction between the time of the member event and the time at which the system processed, stored, or interpreted it.

7.22 Historical applicability

A state that was valid historically but is no longer presently applicable SHOULD remain distinguishable from a state that was always invalid.

7.23 Staleness

Where currentness affects downstream use, the handoff SHOULD permit the receiving component to determine whether the information may be stale.

7.24 Temporal precision SHALL NOT be invented

A component SHALL NOT manufacture more precise timing than the source or architecture supports.

7.25 Confidence and uncertainty context

Where a handed-off state carries evidentiary confidence, uncertainty, or maturity relevant to downstream use, those properties SHOULD remain available to the receiving responsibility.

7.26 Confidence SHALL retain its domain

A confidence value or qualitative confidence state SHALL NOT be interpreted outside the proposition, scope, context, or domain for which it was established.

7.27 Maturity SHALL retain its purpose

A readiness or maturity state SHALL remain associated with the downstream purpose and consequence level for which it was assessed.

7.28 Uncertainty SHALL survive transformation

Summarization or handoff SHALL NOT silently remove material uncertainty merely to produce a simpler downstream representation.

7.29 Contradiction context

Where unresolved contradiction materially affects the handed-off state, the receiving component SHOULD be able to discover that contradiction or the limitation it creates.

7.30 Dependency identity

A derived or synthesized state SHOULD preserve sufficient dependency identity to determine which material upstream states support it.

7.31 Dependency identity supports correction

Dependency linkage SHOULD be sufficient for material correction, invalidation, revocation, or integrity findings to trigger reassessment of affected downstream states.

7.32 Dependency identity supports anti-duplication

Where multiple states derive from the same underlying source, the architecture SHOULD permit that relationship to be recognized so apparent volume is not mistaken for independent evidence.

7.33 Circular dependency SHALL remain detectable

Inter-component handoff SHALL NOT obscure dependency relationships in a manner that makes circular support appear independent.

7.34 Integrity context

Where an upstream component has identified a material integrity limitation, the handoff SHOULD preserve or expose that limitation to downstream responsibilities affected by it.

7.35 Integrity signal is not an error verdict

A receiving component SHALL NOT treat the presence of an integrity signal as automatic proof that the information is false.

7.36 Absence of an integrity signal is not proof

A receiving component SHALL NOT treat the absence of a known integrity warning as proof that the handed-off information is correct or complete.

7.37 Correction linkage

A material handoff SHOULD permit identification of whether the state is current, corrected, superseded, disputed, reconciled, revoked, or otherwise affected by a correction lifecycle where applicable.

7.38 Supersession SHALL NOT erase history

A corrected state MAY supersede a prior state for present use while the architecture preserves the historical relationship necessary for accountability.

7.39 Correction propagation SHALL be dependency-aware

Correction SHALL propagate according to actual material dependency rather than indiscriminately invalidating unrelated downstream states.

7.40 Recovery context

Where information has been reconstructed after corruption, incomplete recovery, migration, or other loss, material recovery limitations SHOULD remain available downstream.

7.41 Scope context

A handoff SHOULD preserve the domain, context, subject, or applicability boundaries required to prevent unsupported generalization.

7.42 Cross-domain use requires support

Availability of information at a component boundary SHALL NOT itself justify using that information in another domain.

7.43 Member identity and subject identity

Where information may concern multiple people, relationships, profiles, or subjects, the architecture SHOULD preserve sufficient subject identity to prevent attribution leakage.

7.44 Relationship information requires bounded attribution

Information about a relationship SHALL NOT silently become a claim about either participant independent of the relationship context.

7.45 Intervention provenance

Where system behavior materially shaped the circumstances that produced later observations, the handoff SHOULD preserve that intervention relationship when relevant.

7.46 Practice Engine intervention

If a Practice Engine selected an experience because of prior continuity, later observations arising from that experience SHOULD remain attributable to the system-selected context.

7.47 Recommendation intervention

If a recommendation materially influenced later behavior, the architecture SHOULD preserve enough provenance to avoid treating the resulting behavior as fully independent confirmation of the recommendation's basis.

7.48 Reflection intervention

Member agreement with a system-generated reflection MAY be meaningful member evidence, but the architecture SHOULD preserve that the reflection preceded and may have influenced the response.

7.49 Purpose and intended use

A material handoff SHOULD identify or permit recovery of the intended downstream purpose when that purpose constrains authorization, maturity, consequence, or interpretation.

7.50 Purpose SHALL NOT silently broaden

A component receiving information for one purpose SHALL NOT automatically reuse it for a materially different purpose without satisfying applicable authorization and readiness requirements.

7.51 Consequence context

Where downstream consequence affects eligibility or maturity, the handoff SHOULD preserve enough context for the receiving component to apply the appropriate threshold.

7.52 Readiness is not authority

A state marked ready for a downstream purpose SHALL NOT be interpreted as authoritative, certain, permanent, diagnostic, or universally applicable.

7.53 Minimal sufficient envelope

Implementations SHOULD transmit or resolve only the envelope properties necessary for accountable downstream use rather than indiscriminately exposing all available member information.

7.54 Data minimization applies to handoff

Inter-component architecture SHALL respect data minimization even when all components are operated by the same implementation.

7.55 Selective disclosure

A component MAY expose a bounded derivative rather than underlying sensitive source material when the derivative is sufficient for the authorized downstream purpose and remains accountable.

7.56 Selective disclosure SHALL preserve reviewability

Where source material is withheld from a downstream component, the architecture SHOULD still preserve an authorized path for appropriate review or correction where required.

7.57 Envelope completeness is use-relative

Not every handoff requires every envelope property. Required context depends on the information type, downstream responsibility, consequence, authorization, and material risk of losing that property.

7.58 Material omission SHALL fail safely

If a receiving component lacks context necessary to determine authorized or responsible use, it SHOULD narrow, defer, reject, or request resolution rather than silently assume permissive defaults.

7.59 Unknown provenance

Information with materially unknown provenance SHOULD NOT be treated as equivalent to otherwise comparable information with accountable provenance.

7.60 Unknown authorization

Information with unresolved authorization SHALL NOT be used for a purpose requiring authorization until that authorization is established.

7.61 Unknown temporal applicability

Where present applicability is necessary and cannot be established, the receiving component SHOULD avoid representing historical information as current.

7.62 Unknown dependency state

Where material dependency identity is unavailable, the architecture SHOULD avoid claiming evidence independence that cannot be demonstrated.

7.63 Unknown integrity state

Lack of completed integrity review MAY constrain downstream use where the consequence or purpose requires such review.

7.64 Component receipt does not imply acceptance

A component MAY receive information while declining to admit it into its own normative state because eligibility requirements are not satisfied.

7.65 Rejection SHOULD be accountable

Material rejection, deferral, or narrowing SHOULD be attributable to a reason such as authorization, provenance, scope, maturity, integrity, temporal applicability, or component eligibility.

7.66 Transformation responsibility

A component that materially transforms information SHOULD be accountable for the transformation it performs rather than attributing the transformed meaning entirely to the upstream source.

7.67 Lossy transformation

Where a transformation intentionally discards detail, the architecture SHOULD preserve enough indication of that loss to prevent the derivative from being mistaken for the complete source.

7.68 Aggregation

Aggregated states SHOULD preserve enough dependency and scope information to avoid hiding material contradiction, source concentration, or authorization differences.

7.69 Summarization

Summaries SHOULD remain attributable as summaries and SHALL NOT silently acquire greater confidence or authority than the information summarized.

7.70 Translation and representation changes

Changing language, modality, storage representation, or transport representation SHALL NOT intentionally alter normative meaning.

7.71 Handoff acknowledgements MAY be implementation-defined

Implementations MAY use acknowledgements, transactions, queues, retries, or other reliability mechanisms, but LGT-ARCH-001 does not prescribe them.

7.72 Delivery failure SHALL NOT fabricate continuity

A failed or delayed handoff SHALL NOT be represented as though the receiving component successfully incorporated the information.

7.73 Duplicate delivery

Implementations SHOULD prevent duplicate transport from becoming duplicate evidence, memory, or continuity support.

7.74 Ordering

Where order materially affects interpretation, the architecture SHOULD preserve or reconstruct the relevant ordering rather than relying on incidental processing order.

7.75 Partial handoff

A partial handoff SHOULD remain identifiable as partial where omitted context could materially affect downstream use.

7.76 Cross-component auditability

The architecture SHOULD support review of material handoffs sufficient to reconstruct which component supplied a state, which component consumed it, and which accountable transformations occurred.

7.77 Auditability SHALL respect privacy

Cross-component auditability SHALL NOT require unrestricted exposure of member information to every component or operator.

7.78 Conformance without centralized transport

A conforming implementation MAY use direct calls, shared storage, events, queues, local functions, model context, or other mechanisms provided the logical handoff contract remains satisfied.

7.79 Conformance without persistent envelopes

A conforming implementation need not persist a literal envelope object if equivalent accountable properties remain reliably recoverable throughout the required lifecycle.

7.80 Conformance with monolithic implementations

A single-process or single-model implementation SHALL still preserve logical handoff boundaries where material information changes architectural responsibility.

7.81 Conformance with distributed implementations

A distributed implementation SHALL NOT treat network or service boundaries as substitutes for normative component boundaries.

7.82 Interoperability

Where independent LGT components interoperate, the implementation SHOULD establish a shared interpretation of envelope properties sufficient to prevent semantic drift.

7.83 Semantic compatibility

Interoperability requires compatible meaning, not merely compatible field names or transport syntax.

7.84 Versioning

Where component or envelope semantics change, implementations SHOULD preserve enough version context to prevent incompatible interpretation of previously produced states.

7.85 Backward compatibility is not absolute

LGT-ARCH-001 does not require indefinite technical backward compatibility when doing so would preserve unsafe, unauthorized, or constitutionally invalid behavior.

7.86 Migration

Migration between implementations or storage systems SHOULD preserve material provenance, authorization, correction, temporal, dependency, and integrity relationships.

7.87 Migration SHALL NOT reset accountability

Moving information to a new system SHALL NOT convert derived information into original evidence or erase prior limitations.

7.88 Export and portability

Where member-authorized portability is supported, exported information SHOULD preserve sufficient context to avoid materially misleading reinterpretation by a receiving LGT system.

7.89 Import

Imported information SHOULD be evaluated for provenance, authorization, compatibility, and integrity before being admitted to equivalent internal authority.

7.90 Security boundaries

Security controls MAY enforce architectural boundaries, but access control alone SHALL NOT be treated as equivalent to purpose authorization, evidence eligibility, or downstream readiness.

7.91 Privacy boundaries

Privacy controls SHOULD operate across component handoffs, including derived states, rather than applying only to raw source storage.

7.92 Failure containment

A component unable to establish required envelope context SHOULD fail within its own responsibility where possible rather than contaminating downstream state with unsupported assumptions.

7.93 Graceful degradation

Where some envelope properties are unavailable, a component MAY continue with a narrower use if that use remains constitutionally and normatively justified.

7.94 No silent degradation

A component SHALL NOT silently continue at the same confidence, scope, or consequence when missing envelope context materially weakens its basis.

7.95 Human review

Where human review is part of an implementation, the reviewer **SHOULD** receive sufficient accountable context for the decision being reviewed without requiring unrestricted access to unrelated member information.

7.96 Explainability support

The handoff contract **SHOULD** preserve the information necessary for downstream explanations to identify material evidence, uncertainty, scope, corrections, and limitations without exposing hidden chain-of-thought.

7.97 Member correction support

The architecture **SHOULD** preserve enough linkage that a member correction can reach materially dependent states rather than stopping at the visible reflection where the correction was expressed.

7.98 Member disagreement

Member disagreement with a reflection **SHALL NOT** automatically invalidate all upstream evidence, but it **SHOULD** be eligible to create new evidence, correction, or scope information as governed by the applicable components.

7.99 Member agency

No envelope property **SHALL** convert an architectural inference into a binding decision about the member's identity, future, or permitted choices.

7.100 Implementation-independent closing principle

The accountable information envelope exists so that meaning, limits, and responsibility survive architectural movement. LGT conformance depends on preserving those properties, not on adopting any particular payload format or technology stack.

8. Cross-Component Lifecycle Coordination and State Propagation

8.1 Purpose

LGT components **SHALL** coordinate material lifecycle changes sufficiently to prevent downstream states from silently outliving the accountable conditions that justified them.

8.2 Lifecycle coordination is logical

This section defines normative lifecycle relationships rather than a required event bus, transaction model, workflow engine, callback system, queue, database trigger, or orchestration technology.

8.3 Local state remains locally governed

Each component remains authoritative for its own normative state. Cross-component coordination **SHALL NOT** permit one component to directly redefine another component's internal eligibility rules.

8.4 Admission

Receipt of information **SHALL NOT** imply admission into a component's normative state. Admission occurs only when the receiving component's applicable eligibility, authorization, provenance, integrity, and scope requirements are satisfied.

8.5 Admission result **SHOULD** be accountable

Material admission, rejection, deferral, or narrowing **SHOULD** remain attributable to the basis on which the receiving component acted.

8.6 Update

A component **MAY** update its state when accountable new information changes a material property without invalidating the underlying state.

8.7 Update **SHALL** preserve provenance

An update **SHALL NOT** silently sever the relationship between the updated state and the information that justified the change.

8.8 Correction

A correction addresses information that was materially inaccurate, misattributed, improperly interpreted, or otherwise wrong under the applicable component rules.

8.9 Correction is distinct from member change

The architecture **SHALL** distinguish correction of an earlier error from evidence that the member genuinely changed after an earlier state was valid.

8.10 Correction propagation

A material correction **SHOULD** propagate to dependent states whose basis is affected.

8.11 Correction SHALL be dependency-aware

Correction SHALL NOT indiscriminately invalidate states that do not materially depend on the corrected information.

8.12 Supersession

A state MAY be superseded when a newer accountable state should govern present use while the prior state remains historically meaningful.

8.13 Supersession SHALL preserve history

Supersession SHALL NOT rewrite the prior state as though it never existed when historical accountability requires its preservation.

8.14 Supersession SHALL NOT imply error

A superseded state MAY have been accurate for its earlier temporal context.

8.15 Revocation

Revocation removes permission or eligibility for a state to continue being used for one or more purposes.

8.16 Revocation SHALL propagate through material use

A revoked source SHALL NOT remain influential for a prohibited purpose merely because its influence has been copied, summarized, synthesized, cached, or transformed downstream.

8.17 Revocation MAY be purpose-specific

Revocation for one purpose SHALL NOT automatically imply revocation for every purpose unless the governing authorization requires it.

8.18 Deletion and forgetting

Deletion or forgetting governed by LGT-MEM-001 SHOULD trigger reassessment of materially dependent states where continued influence would defeat the applicable memory requirement.

8.19 Forgetting SHALL NOT fabricate history

Where historical accountability permits retention of a bounded record that forgetting occurred, the architecture MAY preserve that fact without preserving prohibited substantive influence.

8.20 Staleness

A state MAY become stale when temporal change makes its present applicability uncertain even though it has not been corrected or revoked.

8.21 Staleness SHALL be purpose-sensitive

The same state MAY remain adequate for historical explanation while becoming inadequate for current personalization or recommendation.

8.22 Staleness SHALL NOT automatically mean false

A stale state SHALL NOT be represented as erroneous merely because current applicability requires reassessment.

8.23 Integrity degradation

A state MAY enter a degraded integrity condition when provenance, dependency, authorization, recovery, contradiction, duplication, or other integrity concerns materially weaken responsible use.

8.24 Integrity degradation SHALL affect dependent readiness

Where integrity degradation weakens the basis for downstream use, affected readiness SHOULD be reassessed.

8.25 Integrity recovery

A degraded state MAY recover when accountable review resolves the condition that caused degradation.

8.26 Integrity recovery SHALL NOT erase the degradation history

Where material to accountability, the architecture SHOULD preserve that the state passed through an integrity-limited condition.

8.27 Readiness change

A continuity or derived state MAY become more or less ready for a specified downstream purpose as evidence, uncertainty, consequence, authorization, integrity, or temporal applicability changes.

8.28 Readiness increase SHALL require accountable basis

Elapsed time, repeated retrieval, repeated system assertion, or repeated downstream use SHALL NOT independently increase readiness.

8.29 Readiness decrease SHALL be permitted

A mature state SHALL remain capable of becoming less ready when new evidence, contradiction, staleness, correction, authorization change, or integrity findings warrant it.

8.30 Readiness SHALL remain purpose-bound

A state becoming ready for one use SHALL NOT automatically become ready for another use with different consequence or evidentiary requirements.

8.31 Dependency-aware reconciliation

When an upstream lifecycle change occurs, the architecture SHOULD identify materially dependent downstream states and reassess them according to actual dependency.

8.32 Direct dependency

A state directly derived from a changed source SHOULD be considered for reconciliation when that source materially contributed to its meaning or readiness.

8.33 Transitive dependency

Where a changed source materially contributed through intermediate derived states, reconciliation SHOULD be capable of reaching affected transitive dependents.

8.34 Dependency depth SHALL NOT create immunity

A material source SHALL NOT become immune from correction merely because multiple transformations separate it from a downstream state.

8.35 Dependency depth SHALL NOT require indiscriminate cascade

The existence of a distant dependency SHALL NOT require invalidation where the changed source was not material to the downstream conclusion or use.

8.36 Reconciliation outcomes

Reconciliation MAY retain, update, narrow, qualify, suspend, supersede, revoke, or retire a dependent state according to the governing component rules.

8.37 Reconciliation SHALL NOT manufacture replacement meaning

If affected understanding cannot be responsibly reconstructed after a lifecycle change, the architecture SHOULD preserve uncertainty rather than invent a convenient replacement.

8.38 Contradiction propagation

New contradiction SHOULD propagate far enough for dependent confidence, maturity, synthesis, or readiness to be reassessed where materially affected.

8.39 Contradiction SHALL NOT automatically invalidate

Contradiction MAY reduce confidence, narrow scope, or require review without proving that either side is wholly false.

8.40 Authorization change

Changes to authorization SHOULD propagate to materially dependent uses and derivatives sufficiently to prevent unauthorized continuation.

8.41 Authorization expansion

New authorization MAY permit new downstream use but SHALL NOT retroactively convert previously unauthorized processing into authorized processing.

8.42 Authorization reduction

Reduced authorization SHALL constrain future use and SHOULD trigger review of active dependent states where continued use would exceed the new boundary.

8.43 Purpose change

A component seeking to use an existing state for a materially new purpose SHALL reassess authorization, scope, readiness, and consequence rather than relying on prior admission alone.

8.44 Subject correction

Where a state is discovered to concern the wrong member, profile, relationship, or subject, affected dependencies SHOULD be treated as materially compromised until attribution is corrected.

8.45 Relationship lifecycle

Changes in a relationship MAY alter the applicability of relationship-derived continuity without converting historical relationship evidence into error.

8.46 Domain lifecycle

A state valid within one domain SHALL NOT silently broaden into another domain through repeated reuse or synthesis.

8.47 Cross-domain reconciliation

Where cross-domain synthesis depends on a changed domain-specific state, the synthesis SHOULD be reassessed without assuming that unrelated contributing domains are invalid.

8.48 Intervention lifecycle

Where a system intervention materially shaped later observations, changes to the interpretation that selected the intervention SHOULD remain reviewable alongside those observations.

8.49 Intervention outcome SHALL remain distinct

A successful or unsuccessful downstream experience SHALL return as new accountable evidence opportunity rather than directly changing the historical fact that the intervention was selected.

8.50 Reflection lifecycle

A member-facing reflection MAY remain historically recorded while its present applicability, confidence, or basis changes.

8.51 Reflection correction SHALL reach upstream when appropriate

A member correction expressed at the reflection layer SHOULD be capable of reaching the upstream evidence, memory, or continuity state actually implicated by the correction.

8.52 Companion lifecycle

CMP MAY adapt conversational and relational behavior after correction, refusal, changed preference, or changed understanding without treating relational continuity as a reason to preserve obsolete interpretation.

8.53 Duplicate-state lifecycle

When duplicate states are identified, deduplication SHOULD prevent duplicated representation from continuing to inflate evidence, confidence, maturity, or salience.

8.54 Merge lifecycle

Where duplicate or overlapping states are merged, the merged state SHOULD preserve material provenance and dependency relationships rather than appearing as a new independent source.

8.55 Split lifecycle

Where one state is later determined to contain multiple materially distinct claims, the architecture MAY split it so each claim can carry appropriate provenance, confidence, authorization, and correction status.

8.56 Partial invalidation

Where only part of a composite or synthesized state is invalidated, unaffected portions SHOULD remain eligible for independent reassessment rather than being automatically discarded.

8.57 Partial recovery

Where only part of a damaged or incomplete state can be recovered, the recovered portion SHALL NOT be represented as complete.

8.58 Lifecycle ordering

Where lifecycle order affects meaning, the architecture SHOULD preserve sufficient ordering to distinguish, for example, correction before reflection from correction after reflection.

8.59 Concurrent change

Implementations SHOULD account for materially concurrent updates or corrections sufficiently to avoid silently overwriting accountable state.

8.60 Conflict resolution SHALL remain accountable

Where concurrent lifecycle changes conflict, resolution SHOULD preserve the competing inputs and the basis for the resolved state where material.

8.61 Failed propagation

A failed propagation SHALL NOT be represented as successful reconciliation.

8.62 Propagation retry MAY be implementation-defined

Implementations MAY use retries, queues, transactions, reconciliation jobs, or other reliability mechanisms without affecting conformance provided lifecycle accountability is preserved.

8.63 Delayed propagation

Where propagation is delayed, downstream use SHOULD reflect the risk of stale state when that delay is material.

8.64 Propagation boundaries

A lifecycle change need propagate only to states and uses materially dependent on the changed information and within the applicable authorization boundary.

8.65 Propagation across persistence boundaries

Moving a state between transient context, durable memory, caches, indexes, archives, or other persistence layers SHALL NOT sever its lifecycle obligations.

8.66 Propagation across model boundaries

A change in model, model provider, prompt architecture, or reasoning implementation SHALL NOT erase lifecycle relationships attached to accountable information.

8.67 Propagation across service boundaries

Distributed services SHALL preserve lifecycle coordination even when no single service holds the complete dependency graph.

8.68 Propagation in monolithic implementations

A monolithic implementation SHALL preserve equivalent logical lifecycle coordination even when changes occur within one process or model context.

8.69 Lifecycle observability

Implementations SHOULD provide sufficient observability to determine whether material lifecycle changes were admitted, propagated, reconciled, deferred, or failed.

8.70 Lifecycle observability SHALL respect privacy

Lifecycle review SHALL NOT require indiscriminate exposure of unrelated member information.

8.71 Lifecycle auditability

Material lifecycle transitions SHOULD preserve sufficient history to explain why a state became current, stale, superseded, revoked, degraded, or otherwise changed.

8.72 Audit history SHALL NOT become active evidence

The existence of a lifecycle record SHALL NOT independently strengthen the evidentiary status of the state it describes.

8.73 Member-visible correction

Where appropriate, an implementation SHOULD be able to communicate that prior understanding changed without requiring the member to understand internal lifecycle machinery.

8.74 Member-visible uncertainty

Where lifecycle reconciliation leaves material uncertainty, presentation SHOULD preserve that uncertainty rather than implying seamless certainty.

8.75 Member agency during reconciliation

Lifecycle processing SHALL NOT prevent the member from disagreeing, correcting, refusing, or providing new evidence while reconciliation is incomplete.

8.76 Safety-sensitive lifecycle change

Where stale, corrected, revoked, or degraded information could materially affect safety-sensitive reflection, the affected use SHOULD be reassessed before authoritative-sounding continuation.

8.77 No lifecycle authority escalation

No lifecycle status - including current, mature, stable, reconciled, or repeatedly confirmed - SHALL by itself grant professional, diagnostic, coercive, or personal authority.

8.78 No permanence by maturity

A mature continuity state SHALL remain revisable, correctable, revocable, and capable of becoming less applicable.

8.79 No destiny by recurrence

Repeated historical recurrence SHALL NOT exempt a state from lifecycle reassessment when the member changes or new evidence emerges.

8.80 No identity fixation

Lifecycle continuity SHALL preserve historical understanding without converting a long-lived pattern into a permanent definition of the member.

8.81 Graceful degradation

When lifecycle coordination is incomplete, a component MAY continue with a narrower use if the remaining accountable basis supports that use.

8.82 No silent degradation

A component SHALL NOT continue at unchanged scope, confidence, or consequence when an unresolved lifecycle condition materially weakens its basis.

8.83 Safe suspension

A component MAY suspend downstream use while awaiting correction, authorization resolution, integrity review, or dependency reconciliation.

8.84 Suspension SHALL NOT imply guilt or falsity

Suspension is a state of insufficient readiness or unresolved accountability, not proof that the underlying information is false or improper.

8.85 Retirement

A state MAY be retired from active use when it is no longer relevant, authorized, recoverable, or appropriate for continued operational influence.

8.86 Retirement MAY preserve bounded history

A retired state MAY remain historically visible where authorized and necessary for accountability while being excluded from active inference or personalization.

8.87 Re-entry

A retired, stale, suspended, or previously inapplicable state MAY re-enter active consideration if new accountable evidence and authorization justify it.

8.88 Re-entry SHALL be reassessed

Re-entry SHALL NOT automatically restore prior confidence, maturity, salience, or readiness without current review.

8.89 Migration lifecycle

System migration SHOULD preserve lifecycle states and relationships sufficiently to prevent corrected, revoked, stale, or superseded information from returning as current.

8.90 Version lifecycle

Changes in component specification or implementation semantics SHOULD trigger compatibility review where previously produced states may be interpreted differently.

8.91 Constitutional change

Where a future constitutional revision materially changes permissible use, implementations SHOULD reassess affected architectural behavior rather than grandfathering incompatible active states indefinitely.

8.92 Component specification change

Where a normative component specification changes eligibility or lifecycle meaning, affected implementations SHOULD define a bounded migration or reconciliation strategy.

8.93 Downstream system removal

Removing a downstream experience system SHALL NOT require deleting accountable upstream evidence unless the applicable memory or authorization rules independently require deletion.

8.94 Downstream system replacement

Replacing a Practice Engine, interface, or other downstream system SHALL NOT reset the provenance of member experiences previously shaped by the earlier system.

8.95 Component unavailability

Temporary unavailability of a component SHALL NOT authorize another component to permanently assume that component's normative authority.

8.96 Emergency fallback

A fallback MAY provide reduced functionality but SHALL preserve constitutional boundaries and clearly avoid claiming unavailable evidence, memory, continuity, reflection, or companion functions were completed when they were not.

8.97 Recovery after outage

After an outage or partial failure, the architecture SHOULD reconcile lifecycle changes that may have occurred or failed to propagate before resuming materially dependent downstream use.

8.98 Conformance

A conforming implementation SHALL preserve material lifecycle distinctions and dependency-aware reconciliation even if its internal state names, mechanisms, and topology differ from this section.

8.99 Implementation independence

LGT-ARCH-001 does not prescribe a universal state machine. Implementations MAY model lifecycle differently provided admission, update, correction, supersession, revocation, forgetting, staleness, integrity change, readiness change, and dependency-aware reconciliation remain materially preserved.

8.100 Lifecycle closing principle

Continuity is trustworthy only when it can change responsibly. LGT lifecycle coordination exists so that accountable understanding can mature without becoming irreversible, propagate without becoming detached from its basis, and preserve history without imprisoning the member inside it.

9. Architectural Failure Containment and Degraded Operation

9.1 Purpose

LGT SHALL respond to material component, dependency, integrity, authorization, or propagation failures by containing their effects and narrowing behavior rather than silently fabricating unavailable understanding.

9.2 Failure is not a single state

Implementations SHOULD distinguish among unavailability, timeout, stale dependency, incomplete retrieval, corrupted state, unresolved authorization, failed propagation, incompatible semantics, integrity limitation, and other materially different failure conditions.

9.3 Failure classification SHALL be accountable

Where failure type affects downstream behavior, the architecture SHOULD preserve enough information to explain which capability or dependency was unavailable or limited.

9.4 Local failure SHOULD remain local where possible

A failure in one component SHOULD NOT automatically invalidate unrelated components or states that remain independently accountable.

9.5 Failure containment follows dependency

The scope of degradation SHOULD follow material dependency rather than implementation proximity.

9.6 Shared infrastructure is not shared authority

Multiple components failing because they share infrastructure SHALL NOT collapse their normative responsibilities into one substitute authority.

9.7 No silent authority substitution

If a normative component is unavailable, another component SHALL NOT silently assume that component's authority merely to preserve apparent functionality.

9.8 Observation fallback

If accountable observation cannot be established, the system SHALL NOT fabricate an observation from inference, expectation, prior memory, or model completion.

9.9 Evidence fallback

If evidence evaluation is unavailable or materially incomplete, downstream components SHALL NOT represent unevaluated information as though evidentiary sufficiency had been established.

9.10 Memory fallback

If authorized memory retrieval is unavailable, the Guide MAY operate with narrower present-context capability but SHALL NOT claim to remember information it cannot accountably retrieve.

9.11 CIN fallback

If Continuity Intelligence is unavailable or integrity-limited, downstream systems MAY use independently eligible present-context information but SHALL NOT manufacture continuity from isolated context.

9.12 Mirror fallback

If MIR responsibilities cannot be satisfied, the system SHOULD avoid presenting synthesis as an accountable member-facing reflection.

9.13 Companion fallback

If CMP continuity is unavailable, the system MAY continue a bounded interaction without pretending relational history or familiarity that cannot be accountably established.

9.14 Downstream experience fallback

A Practice Engine or other experience system MAY fall back to non-personalized or broadly eligible experiences when personalized continuity is unavailable, provided the fallback is not represented as continuity-derived personalization.

9.15 Fallback SHALL be bounded

Fallback behavior SHALL be limited to functions whose accountable prerequisites remain satisfied.

9.16 Fallback SHALL NOT broaden consequence

A degraded system SHALL NOT compensate for missing evidence or continuity by making higher-consequence decisions with a weaker basis.

9.17 Fallback SHALL NOT broaden domain

A failure in one domain SHALL NOT justify importing unsupported assumptions from another domain.

9.18 Fallback SHALL NOT broaden authorization

Technical necessity or service degradation SHALL NOT create new authorization to access or use member information.

9.19 Fallback SHALL NOT erase uncertainty

Degraded operation SHOULD preserve or increase uncertainty where the accountable basis is weakened.

9.20 Capability disclosure

Where material to the member's interpretation of an output, the system SHOULD communicate that a relevant capability is unavailable, incomplete, stale, or operating in a limited mode.

9.21 Disclosure SHALL be proportionate

Implementations need not expose internal infrastructure details when a concise capability-level limitation is sufficient.

9.22 No false continuity

The system SHALL NOT present generic, cached, reconstructed, or inferred content as though it arose from current authorized continuity when it did not.

9.23 No false freshness

Cached or stale information SHALL NOT be represented as current when currentness is material to the use.

9.24 No false completeness

Partial retrieval SHALL NOT be represented as complete retrieval when omitted information could materially change the result.

9.25 No false successful propagation

A lifecycle change that failed to reach a dependent component SHALL NOT be recorded or presented as fully reconciled.

9.26 Stale dependency handling

A component consuming stale upstream state SHOULD reassess whether the intended use remains permissible at the known age and consequence level.

9.27 Stale state MAY remain historically useful

Staleness affecting current personalization does not necessarily invalidate historical explanation.

9.28 Stale state SHOULD be bounded

Where present applicability cannot be established, the system SHOULD narrow claims to the historical or limited context that remains supported.

9.29 Partial availability

When only part of a required dependency set is available, the receiving component **SHOULD** determine whether a narrower accountable result can be produced.

9.30 Partial availability SHALL NOT imply full confidence

Confidence, scope, maturity, and readiness **SHOULD** reflect material missing dependencies.

9.31 Missing dependency identity

If a component knows that a material dependency is missing but cannot identify it sufficiently for review, the affected state **SHOULD** be treated as integrity-limited.

9.32 Dependency ambiguity

Where multiple possible upstream states could have produced a derivative and the material dependency cannot be resolved, downstream use **SHOULD** avoid unsupported certainty about provenance.

9.33 Corrupted state

Information known or reasonably suspected to be corrupted **SHALL NOT** be silently repaired by model inference and then treated as original accountable state.

9.34 Reconstruction

A reconstructed state **MAY** be used when permitted by the governing component, but reconstruction **SHALL** remain attributable and **SHALL** preserve material uncertainty or recovery limitations.

9.35 Incomplete recovery

Partially recovered information **SHALL** remain distinguishable from complete original information.

9.36 Authorization service failure

If authorization cannot be established for a use that requires it, the system **SHALL** fail closed for that use.

9.37 Authorization cache

A previously established authorization state **MAY** be used during temporary unavailability only when its scope, lifetime, revocation model, and consequence make such use explicitly acceptable.

9.38 Authorization uncertainty

Uncertainty about authorization **SHALL NOT** be resolved by assuming permission.

9.39 Provenance service failure

If material provenance cannot be recovered, the system **SHOULD** narrow or defer uses that require accountable source identity.

9.40 Provenance uncertainty SHALL survive output

A derivative produced under unresolved provenance limitations **SHALL NOT** later appear as fully accountable simply because it was successfully stored.

9.41 Integrity service failure

If an integrity review mechanism is unavailable, the absence of a warning **SHALL NOT** be interpreted as a successful integrity finding.

9.42 Integrity-limited operation

A component **MAY** continue at a lower consequence or narrower scope when unresolved integrity does not invalidate that bounded use.

9.43 Evidence service failure

When evidence independence, contradiction, or sufficiency cannot be evaluated, the architecture **SHOULD** avoid increasing confidence or maturity.

9.44 Memory service failure

Temporary inability to retrieve memory **SHALL NOT** be interpreted as evidence that no memory exists.

9.45 Memory write failure

A failed memory write **SHALL NOT** be represented to the member or downstream components as successfully remembered.

9.46 Memory deletion failure

A failed deletion or forgetting operation **SHALL** remain unresolved and **SHOULD** prevent claims that the information has been fully removed where such claims would be false.

9.47 CIN service failure

Failure to synthesize continuity SHALL NOT convert isolated evidence into a continuity conclusion by shortcut.

9.48 MIR service failure

Failure to produce a governed reflection SHALL NOT justify exposing raw internal synthesis as an equivalent member-facing reflection.

9.49 CMP service failure

Loss of companion-state continuity SHALL NOT justify reconstructing personal familiarity from unsupported stereotypes or generic assumptions.

9.50 Model provider failure

A model outage or provider change SHALL NOT alter the normative authority hierarchy.

9.51 Model fallback

An alternate model MAY be used if it can satisfy the same applicable architectural responsibilities and constraints for the bounded function being performed.

9.52 Model fallback SHALL be capability-aware

A fallback model SHALL NOT be assumed equivalent merely because it can generate text or complete the same API call.

9.53 Provider transition provenance

Where model/provider differences materially affect interpretation or reproducibility, the implementation SHOULD preserve sufficient operational provenance for review.

9.54 Storage failure

Storage unavailability SHALL NOT authorize components to invent persistent state from transient context.

9.55 Cache failure

Cache loss SHALL NOT be interpreted as deletion of the authoritative state unless the cache itself is the governed authoritative store.

9.56 Cache fallback

Cached state MAY support bounded operation when its age, authorization, provenance, and purpose remain acceptable.

9.57 Queue or transport failure

A transport failure SHALL NOT be treated as successful delivery to a receiving component.

9.58 Duplicate retry

Retry mechanisms SHOULD prevent repeated delivery from creating duplicated evidence, memory, salience, or lifecycle effects.

9.59 Out-of-order delivery

Where delivery order affects meaning, the receiving component SHOULD reconcile ordering before applying lifecycle changes whose interpretation depends on sequence.

9.60 Network partition

Distributed components operating during partition SHOULD avoid making irreversible cross-component assumptions that require unavailable authoritative state.

9.61 Partition recovery

After connectivity returns, materially divergent states SHOULD be reconciled before high-consequence downstream use resumes.

9.62 Clock uncertainty

Distributed time disagreement SHALL NOT create false precision about event order or temporal applicability.

9.63 External source failure

Failure of an external source SHALL NOT be replaced with fabricated source content.

9.64 External source staleness

Previously retrieved external information MAY remain usable for historical or bounded purposes when its age and source limitations remain visible to the consuming responsibility.

9.65 External source substitution

A substitute source MAY be used when its identity, authority, compatibility, and evidentiary role are independently evaluated.

9.66 Human-review unavailability

If a profile requires human review before a use, temporary reviewer unavailability SHALL NOT be bypassed merely to preserve throughput.

9.67 Human override

Human intervention SHALL remain accountable and SHALL NOT silently erase component provenance, uncertainty, or constitutional boundaries.

9.68 Human correction

A human correction SHOULD propagate through the same dependency-aware reconciliation principles as other accountable corrections.

9.69 Configuration failure

Missing or invalid configuration SHALL NOT default to broader authority, broader data access, or higher consequence.

9.70 Safe configuration defaults

Where defaults are necessary, they SHOULD prefer narrower capability, lower consequence, and stronger preservation of authorization and uncertainty.

9.71 Version incompatibility

Components detecting materially incompatible semantics SHOULD reject, defer, translate accountably, or degrade rather than silently reinterpret incompatible state.

9.72 Translation layer

A compatibility or translation layer MAY mediate versions but SHALL be accountable for any semantic transformation it performs.

9.73 Unknown field or state

An unknown lifecycle or envelope property SHALL NOT be silently interpreted as the most permissive known value.

9.74 Feature flag failure

Failure to resolve a capability or policy flag SHALL NOT default to enabling a higher-risk or unauthorized behavior.

9.75 Resource exhaustion

Compute, token, storage, or latency pressure SHALL NOT justify dropping material provenance, authorization, correction, or uncertainty context without corresponding degradation of use.

9.76 Truncation

If context or state must be truncated, the architecture SHOULD preserve enough indication that the resulting view is incomplete.

9.77 Summarization under resource pressure

A compressed representation SHALL remain attributable as a lossy derivative and SHALL NOT acquire greater evidentiary authority than its sources.

9.78 Degraded confidence

A component SHOULD reduce or qualify confidence when material accountable inputs are unavailable.

9.79 Confidence floor

Where remaining support is insufficient for the intended use, the component SHOULD defer or suspend rather than forcing a minimum confidence output.

9.80 Degraded readiness

Readiness SHOULD be reassessed when failures weaken the evidence, authorization, integrity, freshness, or dependency basis for a downstream use.

9.81 Readiness suspension

A previously ready state MAY become temporarily unready during unresolved failure conditions.

9.82 Maturity is not failure immunity

A mature continuity state SHALL NOT bypass failure containment merely because it was previously stable.

9.83 Stable history MAY remain available

A failure affecting current inference does not require erasing unaffected historical continuity.

9.84 Degraded personalization

When personalized continuity is unavailable, the system MAY provide clearly bounded generic functionality rather than fabricating personalization.

9.85 Generic fallback SHALL remain generic

A generic fallback SHALL NOT use language implying that it was selected because of the member's accumulated journey unless that basis remains accountable.

9.86 Member correction during degraded mode

The system SHOULD accept and preserve member corrections when possible even if full reconciliation must wait for unavailable components.

9.87 Deferred reconciliation

Corrections or lifecycle changes received during degraded operation MAY be queued or marked pending, but SHALL NOT be represented as fully propagated until reconciliation completes.

9.88 Recovery begins with state validation

Restoration of component availability SHALL NOT automatically imply that all dependent state is current, reconciled, or safe to resume.

9.89 Recovery reconciliation

After a material outage, the architecture SHOULD identify missed, delayed, duplicated, or conflicting lifecycle changes and reconcile them before materially dependent use resumes.

9.90 Recovery SHALL preserve failure history

Where relevant to accountability, recovery SHOULD preserve that a period of degraded operation occurred.

9.91 Recovery SHALL NOT duplicate evidence

Replaying missed events or rebuilding indexes SHALL NOT cause previously accounted information to become new independent evidence.

9.92 Recovery SHALL NOT restore revoked influence

Reconstruction from backup, cache, replica, or archive SHALL NOT reactivate information whose authorization or lifecycle status no longer permits active use.

9.93 Backup restoration

Backup recovery SHOULD restore lifecycle and authorization context alongside substantive state.

9.94 Disaster recovery

A disaster-recovery environment SHALL remain subject to the same constitutional and normative boundaries as the primary environment.

9.95 Failover

Failover MAY change infrastructure but SHALL NOT change the meaning or authority of the state being processed.

9.96 Split-brain recovery

Where multiple replicas accepted divergent changes, reconciliation SHOULD preserve material competing history rather than selecting a winner without accountable basis.

9.97 Re-entry after degradation

A component returning to normal operation SHOULD re-establish required dependencies, authorization, integrity, and compatibility before resuming full consequence.

9.98 Progressive restoration

Implementations MAY restore capabilities incrementally as accountable prerequisites become available.

9.99 No cosmetic normality requirement

An implementation SHOULD prefer visible bounded degradation over maintaining a seamless interface that falsely implies full capability.

9.100 Member-facing language

Degraded-mode communication SHOULD describe capability limitations without overstating technical certainty or burdening the member with unnecessary infrastructure detail.

9.101 Failure telemetry

Implementations SHOULD maintain operational telemetry sufficient to detect material failures and propagation gaps.

9.102 Telemetry is not member evidence

Operational failure telemetry SHALL NOT become evidence about the member merely because it occurs during member interaction.

9.103 Monitoring data minimization

Failure monitoring SHOULD avoid collecting unrelated member information beyond what is necessary for operational accountability.

9.104 Failure auditability

Material degraded-operation decisions SHOULD be reviewable enough to determine what was unavailable, what fallback occurred, and which downstream uses were narrowed or suspended.

9.105 Security incident

A suspected security incident affecting authorization, provenance, integrity, or confidentiality SHOULD trigger appropriate containment of affected LGT uses.

9.106 Security containment SHALL NOT invent conclusions

A security alert MAY justify suspension or review but SHALL NOT automatically prove that member information is false, malicious, or compromised.

9.107 Privacy incident

A privacy-related failure SHOULD constrain further use according to applicable authorization and privacy requirements rather than preserving functionality at the expense of member control.

9.108 Isolation

Implementations SHOULD be capable of isolating materially compromised components or data paths where continued operation would spread unaccountable state.

9.109 Quarantine

Questionable state MAY be quarantined from active use while preserving bounded evidence necessary for review.

9.110 Quarantine is not deletion

Quarantine SHALL remain distinct from forgetting, deletion, correction, or revocation.

9.111 Circuit breaking MAY be implementation-defined

Implementations MAY use circuit breakers or equivalent mechanisms to stop repeated failing calls, but such mechanisms do not alter normative component authority.

9.112 Rate limiting MAY narrow service

Rate limiting MAY reduce availability but SHALL NOT justify inventing missing results or silently increasing use of stale state beyond acceptable bounds.

9.113 High-consequence profiles

Profiles governing higher-consequence uses SHOULD require stricter failure containment, freshness, authorization resolution, integrity review, and recovery reconciliation.

9.114 Low-consequence bounded continuation

Lower-consequence functionality MAY continue under more limited dependencies when the remaining basis is sufficient and the limitation is not misleading.

9.115 Consequence escalation during failure

A degraded system SHALL NOT escalate to a higher-consequence action because a lower-risk preferred component is unavailable.

9.116 Professional authority

Failure or fallback SHALL NOT convert the Guide into a professional authority, diagnostic system, therapist, oracle, or coercive decision-maker.

9.117 Emergency claims

The architecture SHALL NOT claim emergency knowledge, certainty, or situational awareness it does not accountably possess.

9.118 Member agency

Degraded operation SHALL preserve the member's ability to refuse, correct, disengage, or choose a non-personalized path where such options are applicable.

9.119 Non-determinism

Failure recovery SHALL NOT reconstruct missing state by assuming that the member necessarily followed a predicted or previously recurring path.

9.120 Genuine change during outage

If the member changes while continuity services are unavailable, recovery SHALL permit new evidence to revise prior understanding rather than forcing the member back into the pre-outage model.

9.121 No failure-based identity fixation

Historical state used during fallback SHALL NOT become a permanent identity claim merely because fresher information was temporarily unavailable.

9.122 Conformance

A conforming implementation SHALL preserve failure containment, bounded fallback, honest degradation, and recovery reconciliation even if its technical resilience mechanisms differ from this section.

9.123 Implementation independence

LGT-ARCH-001 does not prescribe active-active deployment, replicas, queues, caches, retries, circuit breakers, model routing, or any other specific resilience architecture.

9.124 Failure-containment closing principle

When accountable capability weakens, the Guide should become narrower before it becomes fictional. Resilience in LGT is not the appearance of uninterrupted intelligence; it is the preservation of truth, authorization, correction, and member agency when the system cannot operate normally.

10. Constitutional and Component Conformance Mapping

10.1 Purpose

LGT conformance SHALL demonstrate accountable satisfaction of governing constitutional principles, this Reference Architecture, and applicable normative component specifications rather than resemblance to a particular implementation.

10.2 Conformance hierarchy

Conformance evaluation SHALL preserve the authority hierarchy: Constitution, Reference Architecture, applicable normative component specifications, conformance profiles where adopted, and implementation-specific design.

10.3 Constitutional primacy

An implementation SHALL NOT claim architectural conformance by satisfying a lower-level requirement in a manner that materially violates the Constitution.

10.4 Architectural coordination

LGT-ARCH-001 conformance concerns whether component responsibilities interact accountably across boundaries, lifecycles, failures, and downstream uses.

10.5 Component conformance

A component claiming conformance SHALL satisfy the normative specification governing the responsibility it performs.

10.6 Implementation freedom

Implementations MAY use materially different languages, models, databases, interfaces, services, deployment patterns, and operational mechanisms while remaining conformant.

10.7 ConjuresUp is not the template

ConjuresUp is the first reference implementation of LGT but SHALL NOT define the mandatory technical form of a conforming implementation.

10.8 Reference implementation evidence

Behavior demonstrated by ConjuresUp MAY provide implementation evidence, examples, or test cases but SHALL NOT replace normative requirements.

10.9 No conformance by branding

Use of LGT terminology, naming, documentation, or visual identity SHALL NOT establish technical or constitutional conformance.

10.10 No conformance by feature count

An implementation SHALL NOT establish conformance merely by implementing similarly named features if the accountable responsibilities and boundaries are absent.

10.11 Requirement traceability

A conforming implementation SHOULD maintain traceability from material architectural behavior to the governing normative requirement or requirements.

10.12 Traceability direction

Traceability SHOULD permit review from requirement to implementation evidence and from material implementation behavior back to its governing requirement.

10.13 Traceability granularity

Traceability MAY group closely related requirements when the grouping does not obscure a material difference in authority, consequence, authorization, or failure behavior.

10.14 Constitutional mapping

Architectural requirements SHOULD be mappable to the constitutional principles they operationalize where such mapping materially assists review.

10.15 Component mapping

Cross-component requirements SHOULD identify the component responsibilities materially implicated without redefining those component specifications.

10.16 Multiple governing requirements

A single behavior MAY satisfy or implicate multiple constitutional, architectural, and component requirements.

10.17 Conflict identification

Where an implementation identifies apparent conflict between requirements, it SHOULD record the conflict and resolve it according to the governing authority hierarchy rather than silently selecting the easier requirement.

10.18 Ambiguity

Material ambiguity SHOULD be documented and resolved conservatively until the governing specification or profile clarifies it.

10.19 Mandatory requirements

Normative SHALL and SHALL NOT requirements are mandatory for the scope in which they apply unless an explicitly governing profile lawfully narrows applicability without weakening constitutional requirements.

10.20 Recommended requirements

Normative SHOULD and SHOULD NOT requirements establish expected behavior from which an implementation MAY deviate only with accountable justification appropriate to the consequence.

10.21 Permitted behavior

MAY requirements identify permitted options and SHALL NOT be interpreted as mandatory implementation features.

10.22 Applicability

A requirement need not be implemented where its triggering capability, data type, downstream use, or architectural condition does not exist.

10.23 Non-applicability SHALL be genuine

An implementation SHALL NOT avoid a requirement by renaming or obscuring a capability that materially performs the governed function.

10.24 Responsibility over component name

Conformance follows the responsibility performed, not the software module name assigned by the implementation.

10.25 Shared implementation

Where one service performs multiple normative responsibilities, each responsibility SHALL remain separately reviewable for conformance.

10.26 Distributed implementation

Where one normative responsibility is distributed across multiple services, the combined behavior SHALL satisfy the applicable responsibility.

10.27 External dependency

Use of an external provider SHALL NOT transfer away the implementation's responsibility to satisfy applicable LGT requirements.

10.28 Model dependency

A model provider's capabilities or limitations SHALL NOT redefine LGT authority boundaries.

10.29 Evidence of conformance

Conformance evidence MAY include tests, configuration, architecture records, logs, traces, policy enforcement, code review, data-flow review, operational exercises, failure simulations, or other reproducible evidence.

10.30 Evidence SHALL match the claim

Evidence used to support conformance SHOULD demonstrate the actual behavior claimed rather than only the intended design.

10.31 Documentation alone is insufficient

Documentation stating that a requirement is satisfied SHALL NOT be treated as sufficient evidence when the behavior can materially differ at runtime.

10.32 Runtime evidence

Runtime or integration evidence SHOULD be used for requirements involving propagation, authorization, lifecycle coordination, degraded operation, or cross-component behavior where static review alone cannot establish the result.

10.33 Static evidence

Static design or code evidence MAY be sufficient where the requirement concerns an invariant that can be reliably established without runtime execution.

10.34 Negative requirements

Requirements prohibiting behavior SHOULD include evidence that the prohibited path is prevented or safely contained, not merely that the preferred path exists.

10.35 Failure evidence

Failure-containment conformance SHOULD include evidence from degraded or failed conditions rather than only normal-operation tests.

10.36 Correction evidence

Correction and reconciliation conformance SHOULD demonstrate that material upstream changes can affect dependent downstream state appropriately.

10.37 Authorization evidence

Authorization conformance SHOULD demonstrate both permitted use and denied or revoked use where materially applicable.

10.38 Provenance evidence

Provenance conformance SHOULD demonstrate that material source and transformation relationships remain recoverable across relevant handoffs.

10.39 Anti-duplication evidence

Where duplicate transport, caching, summarization, or replay can occur, conformance SHOULD demonstrate that duplication does not manufacture independent evidence or salience.

10.40 Anti-self-reinforcement evidence

Where system interventions can influence later member behavior, conformance SHOULD demonstrate that resulting observations remain attributable to the intervention context.

10.41 Member-change evidence

Where continuity persists over time, conformance SHOULD demonstrate that new evidence can revise or reduce prior understanding rather than only strengthen it.

10.42 No hidden permanence

Conformance SHALL NOT depend on an implementation assumption that mature continuity is immutable.

10.43 Test determinism is not epistemic determinism

Deterministic tests MAY verify system behavior but SHALL NOT imply that the member's future or identity is deterministic.

10.44 Conformance profile

A conformance profile MAY define a bounded set of capabilities, consequence levels, deployment assumptions, or interoperability requirements for a specific implementation class.

10.45 Profile authority

A profile SHALL remain subordinate to the Constitution, Reference Architecture, and applicable normative component specifications.

10.46 Profile narrowing

A profile MAY declare capabilities out of scope where genuinely absent, but SHALL NOT weaken requirements for capabilities it includes.

10.47 Profile strengthening

A profile MAY impose stricter requirements for higher-consequence, regulated, interoperable, or specialized environments.

10.48 Profile transparency

A conformance claim SHOULD identify the profile and version, if any, against which the implementation was evaluated.

10.49 Partial conformance

An implementation MAY claim conformance for a clearly bounded component or profile scope when it does not claim conformance for capabilities outside that scope.

10.50 Partial conformance SHALL be explicit

A partial claim SHALL identify its boundaries sufficiently to avoid implying full-framework conformance.

10.51 Component-only claim

A component MAY claim conformance to its normative component specification without claiming that the complete system conforms to LGT-ARCH-001.

10.52 Architecture claim

A system claiming Reference Architecture conformance SHALL account for every normative responsibility and boundary applicable to the capabilities it actually performs.

10.53 Full-framework claim

A full LGT conformance claim SHOULD identify the Constitution version, Reference Architecture version, applicable component specification versions, and adopted profiles.

10.54 Version-specific conformance

Conformance is evaluated against identified normative versions rather than an unspecified moving target.

10.55 Newer specification versions

Release of a newer specification SHALL NOT automatically make an earlier accurately scoped conformance claim false, but continued current conformance claims SHOULD identify whether reassessment has occurred.

10.56 Migration period

A profile or governance process MAY define a bounded migration period for newly introduced requirements where immediate adoption is impracticable and constitutional safety is not compromised.

10.57 No indefinite grandfathering

A migration mechanism SHALL NOT preserve materially unconstitutional or unsafe behavior indefinitely merely because it existed before a newer requirement.

10.58 Extension

An implementation MAY add capabilities beyond the normative framework.

10.59 Extension SHALL preserve boundaries

An extension SHALL NOT silently acquire the authority of OBS, EVD, MEM, CIN, MIR, CMP, or another normative responsibility unless it satisfies the requirements of that responsibility.

10.60 Extension SHALL preserve authorization

Additional capability SHALL NOT broaden member-data use beyond applicable authorization.

10.61 Extension SHALL preserve provenance

Extension-generated derivatives SHOULD remain attributable and SHALL NOT be converted into independent member evidence merely through internal reuse.

10.62 Extension SHALL preserve correction

Where an extension materially influences normative state, its influence SHOULD remain reachable by correction and reconciliation.

10.63 Extension naming

Implementations SHOULD avoid naming extensions in a manner that falsely implies they are normative LGT components.

10.64 Experimental capability

Experimental capabilities MAY coexist with conforming capabilities when their status and boundaries are sufficiently isolated and they do not weaken normative behavior.

10.65 Experimental output

Experimental output SHALL NOT silently enter normative evidence, memory, continuity, reflection, or companion state without satisfying the applicable admission requirements.

10.66 Deviation

A deviation is an intentional departure from a SHOULD or other permitted implementation expectation, or a temporary bounded inability to satisfy a requirement where governance explicitly allows remediation.

10.67 Mandatory violation is not a harmless deviation

Failure to satisfy an applicable SHALL or SHALL NOT requirement SHALL be treated as non-conformance unless the governing normative text explicitly provides an exception.

10.68 Deviation record

Material deviations SHOULD identify the affected requirement, rationale, scope, consequence, compensating controls, review owner, and intended resolution where applicable.

10.69 Compensating control

A compensating control MAY reduce risk but SHALL NOT be described as satisfying a mandatory requirement unless it actually preserves the requirement's normative outcome.

10.70 Temporary exception

A temporary exception SHOULD be time-bounded and reviewed before expiration.

10.71 Exception SHALL NOT become silent baseline

Expired or repeatedly renewed exceptions SHOULD trigger explicit governance review rather than becoming invisible permanent architecture.

10.72 Non-conformance

Non-conformance exists when an applicable normative requirement is materially unsatisfied.

10.73 Non-conformance SHALL be scoped

A finding SHOULD identify the affected component, boundary, use, requirement, and consequence rather than declaring unrelated system behavior invalid.

10.74 Non-conformance severity

Implementations MAY classify findings by consequence, exposure, likelihood, member impact, authorization impact, integrity impact, or other accountable dimensions.

10.75 Severity SHALL NOT change truth

A low-severity classification SHALL NOT convert a real normative violation into conformance.

10.76 Containment

Material non-conformance SHOULD be contained so that affected behavior does not continue to propagate unsupported authority or state.

10.77 Remediation

Remediation SHOULD address the normative cause of the finding rather than merely suppressing its visible symptom.

10.78 Re-verification

A remediated finding SHOULD be re-evaluated with evidence sufficient to demonstrate that the affected requirement is now satisfied.

10.79 Regression

A previously conforming behavior that ceases to satisfy an applicable requirement SHALL be treated as a regression rather than assumed conformant because of prior approval.

10.80 Baseline promotion

An implementation governance process SHOULD promote a new conformance baseline only after required evidence and regression checks pass.

10.81 Rollback

Where a new implementation introduces material non-conformance, rollback to the last verified baseline MAY be an appropriate containment measure.

10.82 Rollback SHALL preserve newer member state where required

Technical rollback SHALL NOT silently discard or overwrite valid member corrections, authorization changes, or other accountable state created after the earlier software baseline.

10.83 Conformance report

A conformance report SHOULD identify evaluated versions, scope, profiles, evidence, findings, deviations, unresolved limitations, and evaluation date.

10.84 Report reproducibility

Material conformance findings SHOULD be reproducible or independently reviewable to the extent permitted by privacy and security constraints.

10.85 Privacy of conformance evidence

Conformance evidence SHOULD minimize exposure of member information and use synthetic, redacted, or bounded evidence where it can establish the same behavior.

10.86 Security of conformance evidence

Evidence collection SHALL NOT require weakening production security controls merely to make conformance easier to demonstrate.

10.87 Member data in tests

Production member data SHOULD NOT be used in conformance testing when equivalent synthetic or properly authorized data can establish the behavior.

10.88 Audit independence

Independent review MAY strengthen assurance but is not itself a substitute for satisfying the normative requirements.

10.89 Self-assessment

An implementation MAY self-assess conformance when no profile or governance regime requires independent assessment.

10.90 Certification terminology

An implementation SHALL NOT represent self-assessment as independent certification when no independent certification occurred.

10.91 Conformance mark

Any future LGT conformance mark or certification program SHOULD identify its scope and governing versions to avoid ambiguous claims.

10.92 Interoperability conformance

Two independently conforming components are not necessarily interoperable unless they also share compatible handoff semantics, versions, and profile requirements.

10.93 Semantic interoperability

Interoperability conformance SHOULD evaluate preservation of meaning, authority, provenance, authorization, lifecycle, and correction semantics rather than transport compatibility alone.

10.94 Imported conformance claims

An implementation consuming another component's conformance claim SHOULD NOT treat that claim as proof beyond its stated scope and version.

10.95 Supply-chain responsibility

Third-party components SHOULD be evaluated for the LGT responsibilities they materially perform, even when they are not marketed as LGT components.

10.96 Black-box component

A black-box dependency MAY be used when the integrating implementation can establish the required externally observable behavior and contain material unknowns.

10.97 Unverifiable internal claims

A provider's unverifiable internal claim SHALL NOT substitute for evidence of externally required LGT behavior.

10.98 Architecture decision record

Material design decisions affecting normative boundaries SHOULD be recorded sufficiently to explain why the implementation considers the design conformant.

10.99 Requirement evolution

When normative requirements evolve, architecture records SHOULD support identifying which implementation behaviors require reassessment.

10.100 Conformance matrix

Implementations SHOULD maintain a conformance matrix or equivalent traceability mechanism for material requirements when system complexity makes informal tracking unreliable.

10.101 Matrix status

A conformance matrix MAY classify requirements as satisfied, not applicable, partially satisfied, non-conforming, pending verification, or subject to an approved bounded deviation where such deviation is normatively permitted.

10.102 Not-applicable rationale

A not-applicable status SHOULD include enough rationale to demonstrate that the governed responsibility or condition is genuinely absent.

10.103 Pending verification

Pending verification SHALL NOT be represented as verified conformance.

10.104 Unknown status

Where conformance cannot be established, the implementation SHOULD report the status as unknown or unverified rather than assuming satisfaction.

10.105 Continuous conformance

Implementations with frequent change SHOULD integrate material conformance and regression checks into their release process rather than relying solely on occasional document review.

10.106 Operational drift

Configuration, model, provider, data-flow, policy, or infrastructure changes that can alter normative behavior SHOULD trigger proportionate conformance reassessment.

10.107 Model update drift

A model update SHOULD be reassessed where it can materially alter evidence handling, reflection behavior, companion behavior, tool use, authorization compliance, or failure behavior.

10.108 Prompt and policy drift

Prompt, policy, or orchestration changes SHOULD be treated as architectural changes when they materially affect normative responsibilities.

10.109 Data-schema drift

Data-model changes SHOULD be reassessed where they can sever provenance, authorization, correction, dependency, temporal, or lifecycle relationships.

10.110 Conformance and quality

Conformance establishes satisfaction of normative LGT requirements; it SHALL NOT be represented as proof of general product quality, accuracy in every interaction, or fitness for every possible purpose.

10.111 Conformance and safety

Conformance supports LGT safety principles but SHALL NOT be represented as a guarantee that no failure, harm, or error can occur.

10.112 Conformance and truth

A conforming architecture remains capable of being wrong; conformance requires that uncertainty, correction, evidence, and accountability remain operative when it is wrong.

10.113 Conformance and relationship

A conforming companion relationship SHALL NOT be treated as proof that the Guide understands the member with certainty.

10.114 Conformance and member agency

No conformance status SHALL grant the implementation authority over the member's identity, choices, beliefs, future, or lived interpretation.

10.115 Conformance and professional roles

LGT conformance SHALL NOT be represented as professional licensure, diagnosis, therapy, legal authority, medical authority, or spiritual authority.

10.116 Public claims

Public conformance claims SHOULD be specific enough for a reasonable reviewer to identify the evaluated scope and governing versions.

10.117 Material limitation disclosure

Known material limitations affecting a conformance claim SHOULD be disclosed rather than hidden behind a broad compliant/non-compliant label.

10.118 Revocation of claim

An implementation SHOULD suspend or narrow a conformance claim when a material regression or unresolved non-conformance makes the prior claim misleading.

10.119 Restoration of claim

A suspended claim MAY be restored after remediation and proportionate re-verification.

10.120 No conformance laundering

Moving a non-conforming behavior into an extension, external service, prompt, model, or downstream system SHALL NOT remove it from conformance scope when it materially performs or influences a governed responsibility.

10.121 No implementation monopoly

No vendor, model provider, database, hosting platform, interface, or reference implementation SHALL be required solely because it was used by an earlier conforming implementation.

10.122 Innovation compatibility

Conformance SHOULD permit architectural innovation when the innovation preserves or strengthens the normative outcomes and constitutional principles.

10.123 Stronger controls

An implementation MAY adopt controls stricter than the minimum requirements when those controls do not violate member agency, authorization, or other governing principles.

10.124 Conformance closing principle

LGT conformance is accountability to principles and observable responsibilities, not imitation of a product. A system qualifies by preserving evidence, memory, continuity, correction, authorization, uncertainty, reflection, companionship, and member agency according to the governing specifications - regardless of the technology used to achieve them.

11. Interoperability and Semantic Compatibility**11.1 Purpose**

LGT interoperability SHALL enable independently implemented components to exchange accountable information without silently changing its meaning, authority, authorization, uncertainty, lifecycle status, or constitutional constraints.

11.2 Interoperability is semantic before technical

Two components SHALL NOT be considered interoperable merely because they can exchange bytes, fields, objects, messages, files, or API calls. They SHALL share sufficiently compatible meaning for the information and responsibilities exchanged.

11.3 Transport neutrality

LGT-ARCH-001 SHALL NOT prescribe HTTP, queues, events, files, shared databases, function calls, model context, RPC, or any other transport mechanism.

11.4 Serialization neutrality

LGT-ARCH-001 SHALL NOT prescribe JSON, XML, relational schemas, graph structures, binary formats, or any other serialization representation.

11.5 Independent implementation

Interoperability SHALL permit components implemented by different organizations, vendors, programming languages, models, storage systems, or deployment architectures when normative meaning remains compatible.

11.6 ConjuresUp is not the wire standard

ConjuresUp interfaces, schemas, database structures, prompts, plugin boundaries, and service conventions SHALL NOT become mandatory interoperability requirements merely because ConjuresUp is the first reference implementation.

11.7 Capability declaration

An interoperating component SHOULD be able to declare the normative responsibilities, versions, profiles, lifecycle behaviors, envelope properties, and optional capabilities it supports.

11.8 Capability claims SHALL be bounded

A capability declaration SHALL identify only capabilities the component can accountably perform under the applicable version and profile.

11.9 Capability discovery SHALL NOT grant authority

Learning that another component supports a capability SHALL NOT authorize its use or expand the receiving component's normative authority.

11.10 Version identity

Material inter-component exchanges SHOULD preserve sufficient specification and semantic version identity to determine how the exchanged state should be interpreted.

11.11 Version negotiation

Where versions differ, components SHOULD determine whether their relevant semantics are compatible before exchanging state whose interpretation depends on those semantics.

11.12 Version negotiation MAY be static or dynamic

Compatibility MAY be established by deployment configuration, certification, profile agreement, runtime negotiation, or another accountable mechanism.

11.13 Version equality is not always required

Different component versions MAY interoperate when the exchanged normative semantics remain compatible for the intended use.

11.14 Same version does not guarantee interoperability

Two implementations claiming the same version SHALL NOT be assumed interoperable if they interpret required semantics differently or fail required responsibilities.

11.15 Semantic compatibility

Semantic compatibility exists when sender and receiver preserve materially equivalent meaning for the exchanged information and the normative consequences attached to it.

11.16 Semantic drift

Implementations SHOULD detect or prevent semantic drift in which a shared field, state, label, or lifecycle term acquires materially different meaning across components.

11.17 Labels are not semantics

Identical names SHALL NOT be treated as proof of semantic compatibility.

11.18 Different labels MAY be compatible

Different internal names MAY represent compatible semantics when an accountable mapping preserves normative meaning.

11.19 Semantic mapping

A translation or compatibility layer MAY map between representations provided it remains accountable for the transformation and does not broaden meaning or authority.

11.20 Lossy mapping

Where a mapping cannot preserve all material semantics, the receiving component **SHOULD** narrow, qualify, defer, or reject affected uses rather than silently discarding required meaning.

11.21 Accountable envelope compatibility

Interoperating components **SHOULD** preserve the accountable information envelope properties required by the receiving use, including provenance, authorization, temporal context, uncertainty, dependency, integrity, correction, scope, intervention, purpose, and readiness where material.

11.22 Envelope extensibility

Implementations **MAY** add envelope properties beyond the normative minimum provided extensions do not redefine required properties incompatibly.

11.23 Unknown envelope property

A receiving component **MAY** ignore an unknown optional property only when doing so cannot materially alter accountable interpretation or required behavior.

11.24 Unknown required property

A property identified as required for the intended use **SHALL NOT** be silently ignored.

11.25 Missing material property

If a receiving component lacks a material envelope property required for responsible use, it **SHOULD** narrow, defer, reject, or request resolution.

11.26 Default values

An omitted property **SHALL NOT** silently receive a permissive default when absence and an explicit value have materially different meaning.

11.27 Explicit unknown

Interoperability **SHOULD** permit explicit representation of unknown, unresolved, unavailable, or not-applicable states where those distinctions affect accountable use.

11.28 Unknown is not false

A receiver **SHALL NOT** interpret unknown as false unless the governing semantic contract explicitly establishes that equivalence.

11.29 Unknown is not authorized

A receiver **SHALL NOT** interpret unresolved authorization as permission.

11.30 Unknown is not current

A receiver **SHALL NOT** interpret unknown temporal applicability as current applicability.

11.31 Provenance preservation

Interoperability **SHALL** preserve sufficient provenance to prevent transferred derivatives from appearing as independent original evidence.

11.32 Provenance translation

A receiver **MAY** translate provenance identifiers into local representations while preserving the accountable relationship to their origin.

11.33 Source identity collision

Implementations **SHOULD** prevent identifier collisions from causing unrelated sources or states to be treated as the same dependency.

11.34 Stable correlation

Where cross-component correction or reconciliation depends on identity, implementations **SHOULD** provide a stable means to correlate materially related states.

11.35 Authorization preservation

Authorization constraints **SHALL** survive inter-component exchange and **SHALL NOT** be broadened by transport, translation, caching, replication, or import.

11.36 Purpose preservation

Where authorization is purpose-limited, the intended or permitted purpose **SHOULD** remain available to the receiving component.

11.37 Receiver authorization

A receiver SHALL independently satisfy applicable authorization requirements even when the sender was authorized to process the information.

11.38 Sender authorization does not transfer automatically

Permission granted to one component, organization, or processing context SHALL NOT automatically become permission for another.

11.39 Revocation propagation

Interoperability SHOULD support propagation or discoverability of authorization revocation where continued downstream use would otherwise exceed the member's current boundary.

11.40 Deletion and forgetting propagation

Cross-component systems SHALL NOT use replication or interoperability as a means to preserve prohibited influence after applicable deletion or forgetting requirements take effect.

11.41 Temporal semantics

Interoperating components SHOULD preserve distinctions among event time, observation time, interpretation time, processing time, and present applicability where material.

11.42 Time-zone and clock representation

Technical differences in time representation SHALL NOT silently alter temporal meaning.

11.43 Precision preservation

A receiver SHALL NOT infer greater temporal precision than the sender accountably supplied.

11.44 Ordering semantics

Where lifecycle or evidentiary meaning depends on order, interoperability SHOULD preserve or reconstruct the accountable order rather than relying on incidental arrival order.

11.45 Confidence semantics

Confidence exchanged between components SHALL remain tied to the proposition, domain, scope, and method for which it was established.

11.46 Confidence scale compatibility

Numerically similar confidence values SHALL NOT be treated as equivalent unless their semantics and calibration are compatible.

11.47 Confidence translation

A receiver MAY translate a confidence representation into a local scale only when the mapping preserves material meaning and uncertainty.

11.48 Uncertainty preservation

Translation SHALL NOT silently remove material uncertainty to fit a receiver's simpler representation.

11.49 Maturity semantics

Maturity or readiness exchanged between components SHALL remain purpose- and consequence-bound.

11.50 Readiness translation

A receiver SHALL NOT treat readiness for one downstream purpose as readiness for another merely because both systems use the same label.

11.51 Dependency semantics

Interoperability SHOULD preserve sufficient dependency identity for correction, anti-duplication, circularity review, and reconciliation.

11.52 Independent evidence semantics

A receiver SHALL NOT count multiple interoperable derivatives of one underlying source as independent evidence merely because they arrived from different components.

11.53 Circular support across components

Cross-component exchange SHALL NOT permit a conclusion to return through another component and appear as independent support for itself.

11.54 Lifecycle vocabulary

Interoperating components SHOULD establish compatible meaning for lifecycle states material to exchange, including current, corrected, superseded, revoked, stale, degraded, suspended, retired, and pending where applicable.

11.55 Lifecycle translation

A component MAY map lifecycle states into a different local state model provided no material distinction required for downstream accountability is lost.

11.56 Correction propagation

A correction originating in one component SHOULD be capable of reaching materially dependent interoperating components.

11.57 Correction identity

Correction exchange SHOULD identify the state or dependency being corrected sufficiently to prevent unrelated information from being modified.

11.58 Supersession compatibility

A receiver SHOULD preserve the distinction between supersession and correction when that distinction affects historical validity.

11.59 Revocation compatibility

A receiver SHALL NOT reinterpret revocation as mere staleness when the difference affects permitted use.

11.60 Pending lifecycle state

Where propagation is incomplete, a receiver SHOULD be able to represent unresolved or pending reconciliation rather than falsely reporting final consistency.

11.61 Integrity semantics

Integrity warnings, degradation, recovery limitations, provenance uncertainty, and unresolved contradiction SHOULD retain compatible meaning across component boundaries.

11.62 Integrity warning is not falsity

A receiver SHALL NOT interpret an integrity limitation as proof that the underlying member information is false.

11.63 Integrity absence is not validation

A receiver SHALL NOT treat absence of an integrity warning as proof that a sender completed every integrity review the receiver requires.

11.64 Intervention provenance

Interoperability SHOULD preserve material system-intervention provenance so downstream outcomes do not become falsely independent confirmation.

11.65 Practice Engine interoperability

A Practice Engine MAY consume continuity from an independently implemented CIN component when the required semantics, authorization, readiness, and provenance are compatible.

11.66 Practice outcomes

Outcomes returned from a Practice Engine SHALL re-enter LGT through accountable observation/evidence pathways rather than directly increasing the maturity of the continuity that selected the practice.

11.67 Mirror interoperability

MIR MAY consume independently produced continuity when it can establish the continuity's applicable scope, confidence, maturity, provenance, authorization, and correction state.

11.68 Companion interoperability

CMP MAY consume independently produced member-facing state without acquiring authority to redefine upstream evidence or continuity semantics.

11.69 Model interoperability

Different model providers MAY participate in interoperable LGT components when normative responsibilities remain satisfied.

11.70 Model identity is not semantic compatibility

Using the same model SHALL NOT be treated as proof that two components interpret LGT states compatibly.

11.71 Different models are not automatically incompatible

Using different models SHALL NOT by itself prevent interoperability.

11.72 Determinism is not required

Interoperability SHALL NOT require identical generated wording or identical internal reasoning when accountable normative outcomes remain compatible.

11.73 Reproducibility scope

Where exact reproduction is not possible, implementations SHOULD preserve enough operational and semantic provenance to review the accountable basis of material outcomes.

11.74 Capability downgrade

When a receiver supports only a subset of the sender's capabilities, the exchange MAY proceed for the compatible subset if unsupported semantics cannot contaminate the bounded use.

11.75 Capability upgrade

A more capable receiver SHALL NOT infer missing higher-order semantics from a less capable sender merely because the receiver could represent them.

11.76 Profile compatibility

Conformance profiles MAY define additional interoperability requirements for particular domains, consequences, or deployment contexts.

11.77 Profile identity

Interoperating components SHOULD identify applicable profiles where profile-specific semantics affect exchange.

11.78 Profile mismatch

A profile mismatch SHOULD trigger compatibility assessment rather than automatic acceptance or rejection.

11.79 Higher-consequence profile

A component operating under a stricter profile SHALL NOT weaken its requirements merely because an upstream component operates under a less strict profile.

11.80 Extension compatibility

Extensions MAY interoperate when required LGT semantics remain intact and extension-specific meaning is explicitly negotiated or safely ignorable.

11.81 Extension namespace or identity

Implementations SHOULD provide a means to distinguish extension semantics from normative LGT semantics.

11.82 Extension collision

An extension SHALL NOT reuse a normative term with materially incompatible meaning.

11.83 Proprietary extension

A proprietary extension MAY exist without preventing LGT conformance, provided conformance does not depend on other implementations adopting that proprietary extension unless a declared profile requires it.

11.84 Safe incompatibility handling

When material incompatibility is detected, components SHOULD reject, defer, narrow, quarantine, or translate accountably rather than silently reinterpret the state.

11.85 Incompatibility SHALL be explicit internally

Material incompatibility SHOULD remain observable enough for operators or conformance review to determine why exchange was limited or rejected.

11.86 Member-facing incompatibility

Where incompatibility materially affects the member's experience, the implementation SHOULD communicate the resulting capability limitation without unnecessary technical detail.

11.87 No permissive fallback

Unknown or incompatible semantics SHALL NOT default to the interpretation that grants the broadest authority, highest confidence, or widest data use.

11.88 No cosmetic interoperability

An implementation SHALL NOT claim interoperability merely because a connector exists when required normative semantics are not preserved.

11.89 No vendor lock-in by conformance

A conformance profile SHOULD NOT require a proprietary vendor technology when equivalent normative behavior can be demonstrated independently.

11.90 Interoperability evidence

Claims of interoperability SHOULD be supported by evidence appropriate to the claim, such as semantic mapping, compatibility tests, lifecycle propagation tests, authorization tests, or cross-implementation conformance results.

11.91 Test vectors MAY be standardized

Future profiles MAY define technology-neutral test vectors representing normative states and expected accountable outcomes.

11.92 Test vectors SHALL NOT become implementation prescriptions

Passing a test vector demonstrates bounded semantic behavior; it SHALL NOT require a particular internal algorithm, prompt, database, or model.

11.93 Round-trip testing

Where translation occurs in both directions, implementations SHOULD test whether material semantics survive a round trip without unauthorized broadening or silent loss.

11.94 Correction testing

Interoperability testing SHOULD include correction, supersession, revocation, and state-state propagation rather than testing only successful forward exchange.

11.95 Failure testing

Interoperability testing SHOULD include unknown fields, missing required semantics, incompatible versions, delayed propagation, duplicate delivery, and partial availability.

11.96 Authorization testing

Interoperability testing SHOULD verify that sender authorization does not silently broaden at the receiver and that revocation can constrain downstream use.

11.97 Anti-circularity testing

Interoperability testing SHOULD verify that information traversing multiple components does not return as falsely independent evidence.

11.98 Migration interoperability

Moving member state between conforming implementations SHOULD preserve material provenance, authorization, temporal, lifecycle, dependency, integrity, and intervention semantics.

11.99 Import trust boundary

Imported state SHALL be evaluated before receiving equivalent local authority; portability does not require blind trust.

11.100 Export accountability

An exporting implementation SHOULD provide sufficient accountable context for a receiving implementation to avoid materially misleading reinterpretation.

11.101 Partial export

A partial export SHOULD be identifiable as partial where missing context could affect interpretation.

11.102 Privacy-preserving interoperability

Interoperability SHOULD exchange only information necessary for the authorized purpose rather than requiring unrestricted replication of member state.

11.103 Selective disclosure

A component MAY provide a bounded accountable derivative instead of underlying sensitive source material when the derivative is sufficient for the authorized use.

11.104 Selective disclosure compatibility

A receiver SHALL NOT demand raw source information merely because its own implementation normally stores that information when a sufficient authorized derivative satisfies the normative requirement.

11.105 Security independence

Authentication, encryption, signing, and transport security MAY support interoperability but SHALL NOT substitute for evidence, authorization, semantic compatibility, or conformance.

11.106 Trust establishment

Technical trust in a sender SHALL NOT automatically establish epistemic trust in every state the sender provides.

11.107 Component reputation

Operational reputation or certification MAY inform trust decisions but SHALL NOT transform unsupported member information into evidence.

11.108 Certification portability

A component's conformance evidence MAY be reusable across integrations only within the versions, profiles, capabilities, and conditions actually assessed.

11.109 Continuous compatibility

Interoperability SHOULD be reassessed when component versions, profiles, semantic mappings, authorization behavior, or material dependencies change.

11.110 Regression

An integration that was previously compatible MAY become non-compatible after a material change and SHALL be capable of narrowing or suspending affected exchange.

11.111 Backward compatibility

Backward compatibility SHOULD be preserved where reasonable but SHALL NOT override correction, authorization, privacy, integrity, or constitutional requirements.

11.112 Forward compatibility

A receiver MAY tolerate unknown optional future semantics when safe, but SHALL NOT invent meaning for unknown required semantics.

11.113 Deprecation

Deprecated semantics SHOULD remain identifiable during transition and SHOULD NOT silently change meaning before removal.

11.114 Sunset

A component MAY cease support for an older semantic version provided affected integrations can detect the incompatibility and avoid silent misinterpretation.

11.115 Interoperability governance

Future LGT profiles MAY define shared semantic registries, identifiers, test suites, or compatibility declarations provided they remain subordinate to the Constitution and normative component specifications.

11.116 Registry authority

A registry or catalog SHALL NOT become an independent source of normative authority beyond the specifications it represents.

11.117 Open implementation possibility

The architecture SHOULD remain implementable without privileged access to ConjuresUp internals or any single vendor's proprietary system.

11.118 Cross-organization accountability

When interoperating components are operated by different organizations, each organization remains accountable for the normative responsibilities it performs.

11.119 Responsibility boundary

Contracts or service boundaries SHALL NOT be used to obscure which component or operator performed a material transformation or decision.

11.120 End-to-end accountability

No individual component need possess every detail, but the interoperating system SHOULD preserve an accountable path sufficient to trace material state from origin through downstream use and correction.

11.121 Member agency across implementations

Interoperability SHALL NOT weaken the member's ability to correct, revoke, refuse, or change merely because their journey crosses implementation boundaries.

11.122 Member change across migration

A receiving implementation SHALL remain capable of recognizing new evidence and genuine change rather than freezing the member into the continuity imported from another system.

11.123 No identity portability mandate

Portability of accountable state SHALL NOT be interpreted as portability of a fixed identity claim about the member.

11.124 Interoperability closing principle

LGT components are interoperable when accountable meaning survives the boundary. The goal is not universal sameness of software; it is preservation of evidence, authorization, uncertainty, correction, continuity, and member agency across independently built systems.

12. Architectural Privacy, Consent, and Member-Control Coordination**12.1 Purpose**

LGT SHALL coordinate privacy, consent, authorization, correction, revocation, deletion, forgetting, portability, and selective disclosure across component boundaries so that continuity does not become a mechanism for escaping member control.

12.2 Architectural scope

This section defines cross-component coordination responsibilities. It SHALL NOT replace the detailed memory eligibility, retention, correction, or forgetting rules governed by LGT-MEM-001.

12.3 Member control follows information

Where member-controlled information crosses an LGT component boundary, applicable control constraints SHALL remain attached to its use rather than ending at the originating component.

12.4 Consent is not blanket authority

Consent for one purpose, component, interaction, domain, or consequence SHALL NOT automatically authorize materially different uses.

12.5 Purpose limitation

Each material use of member information SHOULD be attributable to an authorized purpose sufficiently specific to determine whether downstream processing remains within scope.

12.6 Purpose expansion

A component seeking a materially new purpose SHALL reassess authorization rather than treating prior collection or availability as continuing permission.

12.7 Purpose inheritance

A derivative SHALL NOT silently acquire broader permissible purposes than the information and authorization from which it was derived.

12.8 Purpose narrowing

A receiving component MAY voluntarily operate within a narrower purpose than the sender permitted.

12.9 Minimum necessary information flow

Components SHOULD exchange only the member information materially necessary for the authorized downstream responsibility.

12.10 Architectural convenience is not necessity

A component SHALL NOT justify unrestricted information transfer solely because broader replication is easier to implement.

12.11 Selective disclosure

Where a bounded derivative can satisfy an authorized downstream need, the architecture SHOULD permit disclosure of that derivative instead of unnecessary underlying detail.

12.12 Selective disclosure SHALL remain accountable

A selectively disclosed derivative SHOULD preserve enough provenance, scope, authorization, uncertainty, and lifecycle context for responsible use.

12.13 Selective disclosure SHALL NOT imply hidden certainty

A receiver SHALL NOT infer that undisclosed underlying information necessarily supports a stronger conclusion than the disclosed derivative establishes.

12.14 Raw-source minimization

A receiving component SHOULD NOT require raw observations, journal content, reflections, or other sensitive source material when a sufficient authorized abstraction can perform the required function.

12.15 Data locality

An implementation MAY keep sensitive information within a component or trust boundary while exposing only accountable derivatives required elsewhere.

12.16 Locality SHALL NOT break correction

Data locality mechanisms SHOULD still permit material correction, revocation, lifecycle, and dependency signals to reach affected downstream states.

12.17 Authorization is component-specific

Authorization to process information in one component SHALL NOT automatically authorize another component to access the same underlying information.

12.18 Authorization is operator-specific where applicable

Where different organizations or operators control interoperating components, each SHALL satisfy the authorization responsibilities applicable to its own processing.

12.19 Authorization is purpose-specific

A component authorized to use information for continuity synthesis SHALL NOT automatically be authorized to use it for unrelated analytics, marketing, profiling, or external disclosure.

12.20 Authorization is temporally bounded where applicable

Where authorization has an expiration, withdrawal condition, or lifecycle boundary, downstream use SHALL respect that boundary.

12.21 Authorization uncertainty

If authorization required for a use cannot be established, the system SHALL narrow, defer, or reject that use rather than assume permission.

12.22 Consent withdrawal

Withdrawal SHOULD propagate sufficiently to prevent continued use for purposes no longer authorized.

12.23 Withdrawal is not historical falsification

Withdrawal of permission for future use SHALL NOT automatically mean that historically authorized processing never occurred.

12.24 Withdrawal and active influence

Where continued active influence would violate the withdrawn authorization, derived states SHOULD be reassessed even if a bounded historical record remains permissible.

12.25 Revocation propagation

Revocation SHALL reach materially dependent active uses sufficiently to prevent revoked information from continuing to influence them through copies, summaries, embeddings, caches, indexes, or derived continuity.

12.26 Revocation scope

Revocation SHOULD propagate according to the purpose, information, domain, component, or authorization scope actually withdrawn.

12.27 Revocation SHALL NOT indiscriminately erase

A scoped revocation SHALL NOT automatically remove unrelated information or uses that remain independently authorized.

12.28 Revocation acknowledgment

Where material, implementations SHOULD be able to determine whether affected components received and applied a revocation.

12.29 Failed revocation propagation

A failed propagation SHALL remain unresolved and SHALL NOT be represented as completed member control.

12.30 Correction reach

A member correction expressed in any appropriate member-facing layer SHOULD be capable of reaching the component or state whose accountable meaning is actually implicated.

12.31 Correction routing

The architecture SHOULD route corrections according to dependency and normative responsibility rather than requiring the member to know which internal component produced the affected understanding.

12.32 Correction SHALL NOT stop at presentation

Changing only the visible wording SHALL NOT satisfy correction when an incorrect upstream state continues to influence future behavior.

12.33 Correction SHALL NOT overreach

A correction to one proposition SHALL NOT automatically rewrite unrelated memories, evidence, continuity, or historical events.

12.34 Correction of attribution

Where information is attributed to the wrong member, profile, relationship, or subject, affected downstream states SHOULD be treated as compromised until corrected.

12.35 Correction history

Where authorized and materially necessary for accountability, the architecture MAY preserve that a correction occurred without continuing to treat the corrected proposition as active truth.

12.36 Member disagreement

A member's disagreement with a reflection or synthesis SHOULD remain distinguishable from a verified factual correction when the evidentiary implications differ.

12.37 Disagreement still matters

Even where disagreement does not establish factual falsity, it MAY materially affect confidence, scope, relational presentation, or the appropriateness of continued use.

12.38 Refusal

A member MAY refuse a reflection, practice, personalization path, or request for additional information without that refusal being interpreted as evidence that the underlying interpretation is correct.

12.39 Silence

Failure to object SHALL NOT automatically be treated as consent, confirmation, agreement, or evidentiary support.

12.40 Engagement

Clicking, completing, viewing, or continuing an experience SHALL NOT automatically constitute consent to materially broader data use.

12.41 Practice participation

Participation in a Practice Engine experience SHALL NOT automatically authorize unrelated use of resulting reflections or outcomes.

12.42 Journal sensitivity

Journal content SHOULD remain subject to particularly careful purpose limitation and minimum-necessary disclosure because reflective material may contain context not intended for every component.

12.43 Reflection sensitivity

Member-facing reflections SHOULD NOT require unrestricted exposure of underlying private source material when the reflection can be responsibly produced from bounded continuity.

12.44 Companion access

CMP SHOULD receive only the member context necessary for its governed relational function rather than unrestricted access to all stored information by default.

12.45 CIN access

CIN MAY require broad longitudinal context for synthesis, but access SHALL remain bounded by authorization, purpose, domain, and applicable memory rules.

12.46 MIR access

MIR SHOULD receive sufficient continuity to produce accountable reflection while avoiding unnecessary disclosure of unrelated source detail.

12.47 Practice Engine access

A Practice Engine SHOULD receive only the continuity, readiness, preferences, constraints, and other context necessary to select or adapt the authorized experience.

12.48 Outcome return

Practice outcomes returning to LGT SHOULD be limited to information necessary for accountable observation, learning, correction, and continuity rather than indiscriminate capture of the entire experience.

12.49 Domain boundaries

Authorization in one domain SHALL NOT silently authorize disclosure into another domain.

12.50 Cross-domain synthesis

Where cross-domain synthesis is authorized, the resulting derivative SHOULD preserve enough domain provenance to prevent inappropriate downstream generalization.

12.51 Sensitive-domain narrowing

Profiles MAY impose stricter disclosure, retention, review, or consequence requirements for domains whose information warrants greater protection.

12.52 Privacy profile

Future conformance profiles MAY define stricter privacy and consent coordination requirements without weakening this architectural baseline.

12.53 Deletion coordination

When applicable deletion requirements remove information from active retention, components SHOULD reconcile materially dependent copies and derivatives sufficiently to prevent prohibited continued influence.

12.54 Deletion is not always forgetting

Deletion of a stored representation and removal of its learned or derived influence MAY be distinct responsibilities and SHOULD NOT be conflated.

12.55 Forgetting coordination

Where LGT-MEM-001 requires forgetting, architectural coordination SHOULD prevent downstream systems from reconstructing the forgotten information as active continuity from retained derivatives.

12.56 Forgetting SHALL follow dependency

Forgetting SHOULD address material dependent influence rather than merely deleting one originating record while leaving equivalent active derivatives intact.

12.57 Forgetting SHALL respect independent evidence

Information independently and accountably established from another permitted source SHALL NOT automatically be erased merely because one contributing source is forgotten.

12.58 Forgetting uncertainty

If the architecture cannot determine whether prohibited influence remains in a dependent state, that state SHOULD be narrowed, reassessed, or suspended for affected uses.

12.59 Bounded accountability after forgetting

Where permitted, a system MAY retain a minimal record that a forgetting action occurred without retaining the substantive information whose active influence was removed.

12.60 Backup and archive coordination

Backups and archives SHALL NOT become a route for routinely restoring deleted, revoked, or forgotten information into active use contrary to current controls.

12.61 Recovery from backup

Restored state SHOULD be reconciled against current authorization, correction, deletion, forgetting, and lifecycle status before returning to active influence.

12.62 Cache coordination

Caches SHOULD inherit applicable privacy, authorization, revocation, deletion, and staleness constraints from the state they represent.

12.63 Index coordination

Indexes, embeddings, search structures, and retrieval aids SHOULD NOT preserve prohibited active influence merely because they are technically derived artifacts.

12.64 Logging minimization

Operational logs SHOULD avoid unnecessary inclusion of member content where identifiers, status codes, bounded metadata, or other less revealing information can support accountability.

12.65 Telemetry minimization

Telemetry SHALL NOT become an ungoverned secondary memory of the member.

12.66 Debugging access

Debugging convenience SHALL NOT justify unrestricted exposure or retention of member information beyond an authorized operational need.

12.67 Test data

Production member information SHOULD NOT be copied into testing or development contexts unless specifically authorized and appropriately protected.

12.68 Model context minimization

Components using generative models SHOULD provide only context necessary for the governed task rather than maximizing available personal context by default.

12.69 Model provider boundary

Sending information to an external model provider constitutes a processing boundary and SHALL remain subject to applicable authorization, purpose, and minimum-necessary requirements.

12.70 Model output does not erase input controls

A derivative generated by a model SHALL NOT automatically escape the authorization or purpose constraints materially inherited from its inputs.

12.71 Training and improvement use

Use of member information for model training, generalized product improvement, or unrelated system development SHALL require authorization appropriate to that distinct purpose where applicable.

12.72 No implied training consent

Use of LGT functionality SHALL NOT by itself be interpreted as consent to unrelated model training or generalized data use.

12.73 External service boundary

Third-party analytics, storage, messaging, search, or other services SHALL NOT receive member information merely because they are convenient implementation dependencies.

12.74 Service substitution

Replacing one external service with another SHOULD trigger reassessment of authorization and information-flow implications where the processing boundary materially changes.

12.75 Privacy-preserving interoperability

Independent LGT implementations SHOULD be able to interoperate without requiring wholesale replication of a member's complete journey.

12.76 Portability purpose

Portability SHOULD enable a member to carry accountable continuity or relevant state between conforming implementations without converting continuity into a fixed identity dossier.

12.77 Member-directed export

Where portability is supported, the member SHOULD be able to initiate or authorize export within the applicable product, legal, and technical boundaries.

12.78 Export minimization

An export SHOULD contain the information necessary for the requested portability purpose rather than automatically including every internal artifact.

12.79 Export transparency

Where practical, the member SHOULD be able to understand the broad categories of information included in a portability export.

12.80 Export provenance

Portable state SHOULD retain sufficient provenance, temporal, authorization, lifecycle, confidence, and dependency context to avoid misleading reinterpretation.

12.81 Exported uncertainty

Portability SHALL NOT convert probabilistic or provisional understanding into categorical member facts.

12.82 Imported state

A receiving implementation SHALL evaluate imported state before granting it equivalent local authority.

12.83 Import authorization

Member direction to import state does not require the receiver to use every imported item for every purpose.

12.84 Import correction

The receiving implementation SHOULD preserve applicable correction and supersession relationships in imported state.

12.85 Import refusal

A receiving implementation MAY refuse or narrow imported state that lacks sufficient provenance, authorization, semantic compatibility, or integrity for the intended use.

12.86 Portability and forgetting

Export or migration SHALL NOT create an obligation for the originating implementation to retain information that is otherwise subject to valid deletion or forgetting.

12.87 Portability and revocation

Where feasible and applicable, revocation or correction after export SHOULD be representable so that interoperating systems can avoid treating known obsolete state as current.

12.88 No universal member identifier requirement

LGT SHALL NOT require a universal cross-service identity identifier as a condition of portability or interoperability.

12.89 Identity correlation

Where identity correlation is necessary, implementations SHOULD use the least expansive mechanism sufficient for the authorized purpose.

12.90 Relationship privacy

Information about another person appearing in a member's journey SHOULD NOT automatically become portable, shareable, or reusable as though the member possessed unrestricted authority over that person's information.

12.91 Multi-person observations

Components SHOULD distinguish information about the member from information about third parties where that distinction affects privacy, authorization, or portability.

12.92 Shared experiences

A shared experience MAY create information relevant to multiple people, but each person's control and authorization interests SHOULD be considered according to the applicable profile and context.

12.93 Derived relational state

Relationship-derived continuity SHALL NOT automatically authorize disclosure of one participant's private source information to another participant.

12.94 Member-facing access

Implementations SHOULD provide appropriate means for members to review or understand material continuity used to shape their experience, subject to safety, privacy, security, and system-integrity constraints.

12.95 Access is not raw internals

Member transparency does not require disclosure of hidden chain-of-thought, security-sensitive internals, proprietary model parameters, or unrelated third-party information.

12.96 Actionable transparency

Member-facing access SHOULD prioritize understandable observations, sources/categories, confidence, corrections, and consequences rather than opaque internal representations.

12.97 Control discoverability

Material correction, refusal, revocation, deletion, forgetting, or portability controls SHOULD be reasonably discoverable when the implementation supports them.

12.98 No dark-pattern control

Member controls SHOULD NOT be intentionally designed to make refusal, correction, revocation, or deletion materially harder than corresponding participation or authorization.

12.99 No retaliation for correction

A member correction or refusal SHALL NOT be treated as a reason to reduce dignity, punish the member, or intentionally degrade unrelated service.

12.100 No coercive personalization

The architecture SHALL NOT use accumulated continuity to pressure a member into disclosure, continued participation, purchase, or acceptance of an interpretation.

12.101 No authority from intimacy

Greater longitudinal familiarity SHALL NOT create greater entitlement to member information beyond authorized purposes.

12.102 No privacy erosion by maturity

As continuity matures, information controls SHALL NOT silently weaken merely because the Guide has accumulated more history.

12.103 No consent inference from recurrence

Repeated participation SHALL NOT automatically broaden consent.

12.104 No consent inference from prediction

A prediction that the member would probably agree SHALL NOT substitute for required authorization.

12.105 No consent inference from benefit

A system SHALL NOT assume broader permission merely because it believes additional data use would benefit the member.

12.106 Member change

Privacy preferences and authorization choices MAY change over time and SHALL remain revisable.

12.107 Preference history

Historical privacy preferences MAY remain relevant to accountability but SHALL NOT override a current valid preference or authorization boundary.

12.108 Conflicting controls

Where member controls conflict across components or versions, the architecture SHOULD resolve the conflict according to applicable authority, scope, recency, purpose, and legal/product requirements rather than choosing the most permissive state.

12.109 Unresolved control conflict

If a material control conflict cannot be resolved, affected processing SHOULD narrow or defer.

12.110 Control propagation latency

Implementations SHOULD minimize the period during which a valid correction, revocation, deletion, or forgetting action has not yet reached materially dependent active uses.

12.111 High-consequence control propagation

Higher-consequence profiles SHOULD require stricter propagation and verification expectations.

12.112 Offline components

Components temporarily offline SHOULD reconcile member-control changes before resuming materially affected active use.

12.113 Failed component

A failed component SHALL NOT permanently prevent a member-control action from being represented as pending and reconciled when the component returns.

12.114 Degraded operation

Failure containment SHALL preserve privacy and authorization boundaries; degraded mode SHALL NOT broaden information access to compensate for unavailable capabilities.

12.115 Emergency operation

An emergency or safety-sensitive profile MAY define exceptional processing rules, but those rules SHALL be explicit, bounded, auditable, and subordinate to applicable law and constitutional limits rather than inferred ad hoc.

12.116 Security and privacy distinction

Security controls protect information from unauthorized access; privacy and member-control rules govern whether otherwise secure processing is permitted. One SHALL NOT be treated as a substitute for the other.

12.117 Encryption is not consent

Encrypted processing or storage SHALL NOT make an otherwise unauthorized purpose permissible.

12.118 Anonymization claims

Information SHALL NOT be treated as anonymous merely because obvious identifiers were removed if the implementation can still reasonably associate it with the member.

12.119 Aggregation

Aggregated information MAY reduce privacy exposure, but aggregation SHALL NOT be represented as independent evidence about an individual member unless individual-level support remains accountable.

12.120 Privacy auditability

Material privacy and member-control actions SHOULD preserve sufficient operational evidence to determine whether authorized purpose, propagation, and lifecycle responsibilities were followed.

12.121 Audit minimization

Privacy auditability SHOULD avoid creating a second unnecessarily detailed repository of member content.

12.122 Conformance evidence

A conformance claim SHOULD be able to demonstrate representative flows for purpose limitation, selective disclosure, correction, revocation, deletion/forgetting coordination, and portability where those capabilities are in scope.

12.123 Extension responsibility

An extension handling member information SHALL remain subject to applicable privacy and member-control requirements even if the extension itself is non-normative.

12.124 Downstream responsibility

A downstream experience system SHALL NOT escape privacy coordination merely because it sits outside the core LGT intelligence components.

12.125 Operator responsibility

Organizational boundaries SHALL NOT be used to obscure which operator is responsible for a material information use or member-control action.

12.126 Architecture and law

LGT-ARCH-001 defines architectural responsibilities and SHALL NOT be represented as a substitute for jurisdiction-specific privacy, consumer-protection, health, employment, child-safety, or other legal obligations.

12.127 Stricter obligations

Where applicable law, contract, platform policy, or a conformance profile imposes stricter requirements, an implementation MAY need to exceed this architectural baseline.

12.128 No legal inference

Architectural conformance SHALL NOT by itself be represented as proof of legal compliance.

12.129 Constitutional alignment

Privacy coordination SHALL preserve the Constitution's observation-first, evidence-loyal, non-coercive, member-agency, and revisability principles.

12.130 Member source of truth

The member's lived journey remains the source of truth about the member; accumulated data SHALL NOT become an ownership claim over that journey.

12.131 Continuity without possession

Continuity Intelligence SHOULD preserve accountable relationship over time without treating accumulated member information as something the system is entitled to keep or use indefinitely.

12.132 Conformance

A conforming implementation SHALL preserve purpose limitation, authorization boundaries, correction reach, revocation propagation, minimum-necessary flow, and applicable deletion/forgetting coordination across component boundaries.

12.133 Implementation independence

LGT-ARCH-001 does not prescribe a consent database, privacy dashboard, identity provider, encryption system, policy engine, portability format, or deletion technology.

12.134 Privacy closing principle

A Living Guide may remember deeply without claiming ownership of what it remembers. Continuity earns trust only when the member can remain more authoritative over their participation, correction, boundaries, and future than the system's accumulated history is.

13. Architectural Observability, Auditability, and Accountable Explanation

13.1 Purpose

LGT SHALL make material transformations, dependencies, controls, and consequences sufficiently observable to support accountable review without requiring disclosure of hidden model reasoning.

13.2 Observability is not surveillance

Operational observability SHALL be limited to information necessary to understand system behavior, conformance, failures, and material state transitions rather than becoming an unrestricted secondary record of the member.

13.3 Auditability is end-to-end

Where a material outcome depends on multiple components, the architecture SHOULD preserve an accountable path from relevant source state through material transformations to downstream use.

13.4 Explanation is not chain-of-thought

A conforming implementation SHALL NOT require disclosure of private chain-of-thought, hidden reasoning traces, model activations, proprietary parameters, or security-sensitive internals in order to provide an accountable explanation.

13.5 Accountable explanation

An accountable explanation SHOULD identify the material evidence or continuity basis, provenance, applicable scope, uncertainty or confidence, relevant lifecycle state, material transformations, governing constraints, and resulting consequence where those factors affected the outcome.

13.6 Explanation SHALL match actual basis

A component SHALL NOT present a plausible post-hoc rationale as though it were the accountable basis for an outcome when that rationale was not supported by the recorded state and governing process.

13.7 Explanation may be reconstructed from accountable records

A member-facing or operator-facing explanation MAY be generated after the fact from preserved accountable state when it faithfully represents the material basis and limitations of the outcome.

13.8 Explanation is audience-specific

The architecture MAY provide different levels of explanation to members, operators, auditors, developers, or conformance reviewers provided each remains truthful and does not materially distort the underlying basis.

13.9 Member-facing explanation

Member-facing explanation SHOULD prioritize understandable reasons, relevant observations or themes, confidence, limitations, and available correction paths rather than internal implementation detail.

13.10 Operator-facing explanation

Operator-facing explanation MAY include component identity, dependency state, lifecycle transitions, failure conditions, policy decisions, and technical provenance necessary for diagnosis and accountability.

13.11 Conformance explanation

Conformance evidence SHOULD demonstrate how normative responsibilities are satisfied without requiring disclosure of unrelated proprietary implementation details.

13.12 Materiality

Implementations SHOULD prioritize observability for transformations and decisions capable of materially affecting continuity, personalization, member-facing reflection, authorization, privacy, readiness, or downstream consequence.

13.13 Trivial operations

Purely mechanical operations that cannot materially alter meaning need not generate the same depth of audit record as interpretive or authority-bearing transformations.

13.14 Transformation identity

A material derivative SHOULD identify the component responsibility that produced it sufficiently for later review.

13.15 Transformation provenance

A derivative SHOULD preserve enough relationship to its inputs to distinguish transformation from independent observation or evidence.

13.16 Transformation type

Where useful to accountability, the system SHOULD distinguish observation capture, evidence evaluation, memory admission, synthesis, reflection, recommendation, correction, translation, migration, and other materially different transformations.

13.17 Input scope

An accountable record SHOULD identify the material input scope used for a transformation without requiring unnecessary duplication of sensitive source content.

13.18 Omitted input

Where an expected or normally available input was absent and that absence materially affected the result, the limitation SHOULD be observable.

13.19 Dependency identity

Material dependencies SHOULD remain traceable sufficiently to support correction, anti-circularity review, revocation, failure reconciliation, and explanation.

13.20 Dependency graph

An implementation MAY represent dependencies as a graph, ledger, event lineage, relational structure, or other mechanism; LGT does not prescribe the representation.

13.21 Dependency minimization

Auditability SHOULD NOT require retaining unnecessary substantive member content when stable dependency identifiers or bounded metadata are sufficient.

13.22 Provenance continuity

Provenance SHOULD survive storage, caching, serialization, translation, migration, summarization, and cross-component exchange when it remains material to accountable use.

13.23 Provenance loss

If material provenance is lost, affected downstream states SHOULD be marked, narrowed, reassessed, or suspended according to consequence rather than silently treated as fully accountable.

13.24 Provenance uncertainty

Uncertain provenance SHALL remain distinguishable from verified provenance.

13.25 Confidence observability

Where confidence materially affects downstream use, the proposition, domain, scope, and basis of that confidence SHOULD be reviewable.

13.26 Confidence change

Material increases or decreases in confidence SHOULD be attributable to new evidence, contradiction, correction, staleness, integrity changes, or another accountable cause.

13.27 Confidence SHALL NOT rise invisibly

A component SHOULD NOT materially increase confidence without an observable accountable basis.

13.28 Maturity observability

Where continuity maturity affects readiness or consequence, the basis for maturity SHOULD be reviewable at an appropriate level.

13.29 Readiness observability

A readiness decision SHOULD identify the applicable purpose or consequence and the material conditions supporting or limiting readiness.

13.30 Authorization observability

Material authorization decisions SHOULD be reviewable sufficiently to establish the purpose, scope, relevant boundary, and whether permission was resolved.

13.31 Privacy observability

Selective disclosure, purpose limitation, revocation, deletion, forgetting, portability, and other material member-control actions SHOULD leave sufficient operational evidence to demonstrate coordination without creating an unnecessary copy of private content.

13.32 Correction observability

A correction SHOULD be traceable from the corrected proposition or state through materially dependent active derivatives where propagation is required.

13.33 Correction completion

The architecture SHOULD distinguish requested, pending, partially propagated, completed, failed, and otherwise unresolved correction states when those distinctions affect truthful representation.

13.34 Revocation observability

A revocation SHOULD be capable of demonstrating whether materially dependent uses were constrained or remain pending reconciliation.

13.35 Forgetting observability

Where forgetting is required, the system SHOULD preserve enough bounded operational evidence to demonstrate that the action occurred without preserving the substantive information solely for audit convenience.

13.36 Failure observability

Material failures SHOULD be observable at the capability or dependency level necessary to explain degraded behavior and support recovery reconciliation.

13.37 Recovery observability

Recovery SHOULD preserve whether reconciliation was completed before materially dependent full operation resumed.

13.38 Interoperability observability

Cross-component exchange SHOULD retain enough version, profile, capability, mapping, and compatibility information to explain material translation or rejection decisions.

13.39 Translation observability

A semantic translation SHOULD identify the mapping or transformation responsible where the mapping could materially affect interpretation.

13.40 Lossy translation

If a translation is materially lossy, that limitation SHOULD remain observable downstream.

13.41 Model provenance

Where model identity, configuration, or provider materially affects reproducibility or conformance review, sufficient operational provenance SHOULD be preserved.

13.42 Model provenance SHALL be bounded

Model provenance need not include private chain-of-thought, hidden prompts unrelated to accountable behavior, proprietary weights, or security-sensitive implementation detail.

13.43 Prompt accountability

Where a system instruction, policy template, or prompt materially governs a normative transformation, the implementation SHOULD be able to demonstrate the applicable governing behavior or version without necessarily disclosing protected prompt text.

13.44 Policy provenance

Material policy or rule decisions SHOULD be attributable to the applicable policy version or profile where practical.

13.45 Configuration provenance

Configuration capable of materially changing normative behavior SHOULD be versioned or otherwise reviewable.

13.46 Feature-flag provenance

A feature flag that changes a material LGT responsibility SHOULD be observable enough to determine which behavior was active for the affected outcome.

13.47 Code provenance

Implementations MAY preserve software version or deployment identity where useful to reproducibility, but code identity alone SHALL NOT be treated as evidence that normative behavior was correct.

13.48 Temporal observability

Material records SHOULD preserve temporal context sufficient to distinguish event, observation, interpretation, processing, correction, and present applicability where those distinctions matter.

13.49 Clock limitation

Known clock uncertainty or ordering ambiguity SHOULD remain observable when it could affect lifecycle interpretation.

13.50 Member intervention provenance

Member corrections, refusals, preferences, confirmations, or other material interventions SHOULD remain distinguishable from system-generated inferences.

13.51 System intervention provenance

Practices, prompts, recommendations, reflections, reminders, or other system interventions SHOULD remain distinguishable from independent member-originated evidence.

13.52 Outcome provenance

A member response following a system intervention MAY be valid evidence, but its relationship to the intervention SHOULD remain observable to prevent false independence.

13.53 Human intervention provenance

Material human review, correction, override, or adjudication SHOULD remain attributable as human intervention.

13.54 Human authority boundary

Human intervention SHALL NOT erase the underlying provenance or transform opinion into member evidence merely because a human supplied it.

13.55 External-source provenance

Material external information SHOULD preserve source identity and retrieval context sufficient for appropriate evidentiary interpretation.

13.56 Source update

Where an external source changes materially, the architecture SHOULD be capable of distinguishing newly retrieved information from older cached or historical information.

13.57 Audit record integrity

Audit records SHOULD be protected against unauthorized alteration sufficient to support the consequence level of the behavior they evidence.

13.58 Audit record correction

Incorrect audit metadata SHOULD be correctable without silently rewriting the substantive history of what occurred.

13.59 Append-only is not universally required

LGT does not require a universal append-only ledger; implementations MAY use other accountable mechanisms that preserve necessary history and correction.

13.60 Tamper evidence MAY be profile-dependent

Higher-consequence profiles MAY require stronger tamper-evidence, signing, attestation, or independent audit controls.

13.61 Audit retention

Audit information SHOULD be retained only as long as necessary for applicable accountability, security, conformance, product, or legal purposes.

13.62 Audit retention SHALL NOT override forgetting automatically

The existence of an audit function SHALL NOT become blanket justification to retain substantive member information that should otherwise be forgotten.

13.63 Audit minimization

Where a hash, identifier, category, status, or bounded derivative is sufficient, the architecture SHOULD avoid duplicating full member content into audit systems.

13.64 Audit access

Access to audit information SHOULD be restricted according to role, purpose, sensitivity, and applicable authorization.

13.65 Member access to explanations

Where a material outcome shapes the member's experience, the implementation SHOULD provide an appropriate means to understand the accountable basis or limitation of that outcome.

13.66 Member explanation is not forensic disclosure

Member transparency does not require exposing security-sensitive logs, third-party private information, proprietary internals, or hidden model reasoning.

13.67 Explanation should support correction

Where practical, an explanation SHOULD help the member identify what they can correct, clarify, refuse, or update if the Guide's understanding is wrong or outdated.

13.68 Explanation should expose uncertainty

A member-facing explanation SHOULD NOT hide material uncertainty merely to sound confident or coherent.

13.69 Explanation should expose limitation

When missing, stale, incompatible, or degraded information materially limited an outcome, the explanation SHOULD communicate that limitation at an appropriate level.

13.70 Explanation should expose personalization basis

When a recommendation or reflection is described as personalized, the system SHOULD be able to identify the categories of accountable continuity that materially shaped it.

13.71 Generic output explanation

A generic fallback SHOULD be explainable as generic rather than retroactively attributed to the member's journey.

13.72 No fabricated trace

An implementation SHALL NOT generate fictional citations, nonexistent source identifiers, fabricated confidence history, or invented audit events to make an outcome appear accountable.

13.73 No explanation laundering

A fluent explanation SHALL NOT be used to conceal missing provenance, unsupported certainty, authorization failure, or absent conformance evidence.

13.74 No chain-of-thought dependency

Conformance SHALL NOT depend on storing or exposing a model's private reasoning transcript.

13.75 Reason codes

Implementations MAY use bounded reason codes, decision records, structured rationales, or evidence summaries to support explanation.

13.76 Structured rationale

A structured rationale SHOULD reflect material accountable factors rather than every computational step.

13.77 Counterfactual explanation

A system MAY explain what material condition would have changed an outcome when that statement is supported by the governing rules or accountable state.

13.78 Counterfactual limits

A counterfactual SHALL NOT be presented as certain when model nondeterminism, unknown dependencies, or other limitations prevent that certainty.

13.79 Competing evidence

Where materially competing evidence exists, explanation SHOULD avoid presenting only the supporting side if doing so would materially misrepresent confidence.

13.80 Superseded understanding

A historical explanation MAY state why an earlier understanding was reasonable at the time while also identifying why it was later superseded.

13.81 Correction explanation

A correction SHOULD permit the system to explain that its understanding changed because new or corrected information altered the accountable basis.

13.82 Genuine member change

Where the member genuinely changed, explanation SHOULD avoid falsely characterizing the newer state as merely a correction of an earlier system error.

13.83 Explanation consistency

Different interfaces SHOULD NOT provide materially contradictory explanations for the same accountable state without a legitimate audience, scope, or timing difference.

13.84 Explanation versioning

Where explanation rules materially change, implementations **SHOULD** preserve enough version context to review historical explanations appropriately.

13.85 Dashboard observability

An implementation **MAY** provide dashboards for operators or members, but dashboards **SHALL** remain views over accountable state rather than new sources of authority.

13.86 Metrics

Operational metrics **MAY** summarize system behavior but **SHALL NOT** replace case-level provenance where case-level accountability is required.

13.87 Aggregate metrics

Aggregate performance **SHALL NOT** prove that a specific member-facing outcome was accountably supported.

13.88 Case-level evidence

Where a particular outcome is challenged, the architecture **SHOULD** be capable of reviewing the accountable basis for that outcome within applicable privacy and retention limits.

13.89 Sampling

Conformance or quality review **MAY** use representative sampling where appropriate, but sampling **SHALL NOT** conceal known material failures.

13.90 Alerting

Implementations **SHOULD** alert operators to material integrity, propagation, authorization, or conformance failures at a level appropriate to consequence.

13.91 Alert fatigue

Observability design **SHOULD** prioritize material signals rather than producing excessive low-value alerts that obscure meaningful failures.

13.92 Diagnostic separation

Operational diagnostics **SHOULD** remain distinguishable from member evidence and continuity state.

13.93 Debug data

Temporary debug instrumentation **SHOULD** be removed, minimized, or governed when no longer required.

13.94 Trace identifiers

Implementations **MAY** use trace identifiers to correlate cross-component activity without embedding unnecessary member content in those identifiers.

13.95 Correlation privacy

Trace or correlation identifiers **SHOULD NOT** become universal member identifiers unless such identity linkage is independently justified and authorized.

13.96 Cross-organization tracing

Interoperating organizations **MAY** exchange bounded trace references sufficient for accountability without exposing unrelated internal logs.

13.97 Trust boundary observability

Material crossings of organizational, provider, model, storage, or external-service boundaries **SHOULD** be reviewable where they affect authorization, privacy, provenance, or conformance.

13.98 Selective disclosure in audit

An auditor **MAY** receive evidence sufficient to assess a requirement without receiving all underlying member content.

13.99 Redacted conformance evidence

Conformance evidence **MAY** be redacted or abstracted when necessary to protect member privacy or security, provided the remaining evidence is sufficient for the claim.

13.100 Evidence insufficiency

If privacy-preserving evidence is insufficient to substantiate a conformance claim, the claim **SHOULD** be narrowed rather than filled with unsupported assurance.

13.101 Independent audit

Profiles MAY require independent audit for particular consequence classes, but LGT-ARCH-001 does not mandate a universal external auditor.

13.102 Self-assessment

Self-assessment MAY support conformance where appropriate, but the implementation SHALL remain responsible for truthful scope and evidence.

13.103 Certification claims

Certification or audit status SHALL identify the version, profile, scope, and conditions actually assessed.

13.104 Expired evidence

Old conformance evidence SHALL NOT automatically substantiate materially changed architecture, models, policies, or component behavior.

13.105 Continuous reassessment

Material changes SHOULD trigger reassessment of affected observability, auditability, and explanation responsibilities.

13.106 Regression detection

The architecture SHOULD support detection of regressions that remove provenance, break correction propagation, broaden authorization, hide uncertainty, or otherwise weaken accountable behavior.

13.107 Regression evidence

A previously passing implementation SHALL NOT treat historical QA as proof that a changed release remains conforming.

13.108 Reference implementation telemetry

ConjuresUp MAY demonstrate observability patterns, but its logging stack, analytics schema, dashboards, or tracing technology SHALL NOT become mandatory LGT architecture.

13.109 Technology neutrality

LGT does not prescribe OpenTelemetry, a particular log system, database, SIEM, analytics service, tracing platform, or audit ledger.

13.110 Explainability technology neutrality

LGT does not prescribe a particular explainable-AI technique, saliency method, reasoning model, or model-specific introspection mechanism.

13.111 Outcome-focused explanation

The architectural requirement is that material outcomes can be accountably explained from governed evidence and state, not that every internal computation can be narrated.

13.112 Epistemic explanation

When explaining an understanding about the member, the system SHOULD distinguish observation, evidence, inference, synthesis, uncertainty, and contradiction where those distinctions are material.

13.113 Authority explanation

When explaining an action or restriction, the system SHOULD distinguish what information supported the decision from what policy or authorization rule permitted the action.

13.114 Recommendation explanation

A recommendation SHOULD be explainable in terms of relevant continuity, readiness, preferences, constraints, prior outcomes, or other accountable factors that actually shaped selection.

13.115 Reflection explanation

A reflection SHOULD be explainable in terms of the patterns or continuity it is reflecting without claiming certainty about the member's identity or destiny.

13.116 Memory explanation

Where appropriate, the Guide SHOULD be able to explain broadly why something is remembered, updated, corrected, or no longer actively used without exposing unnecessary private internals.

13.117 Failure explanation

A degraded result SHOULD be explainable as degraded when a material dependency or capability was unavailable.

13.118 Refusal explanation

Where the system refuses or narrows an action because a prerequisite is missing, the explanation SHOULD identify the relevant limitation without inventing unsupported content.

13.119 Security refusal

Security-sensitive refusal explanations MAY remain intentionally bounded to avoid revealing exploitable implementation detail.

13.120 Privacy refusal

A system MAY decline to expose another person's private information even when that limits the completeness of a member-facing explanation.

13.121 Explanation and dignity

Explanations SHOULD avoid framing uncertainty, disagreement, correction, or refusal in ways that diminish member dignity or agency.

13.122 Explanation and non-coercion

An explanation SHALL NOT use accumulated personal knowledge to manipulate the member into accepting an interpretation or action.

13.123 Explanation and destiny

The Guide SHALL NOT convert probabilistic continuity into deterministic claims merely because a deterministic explanation sounds simpler.

13.124 Explanation and free will

Where future-oriented guidance is explained, the system SHOULD preserve the role of member choice, chance, changing circumstances, and incomplete knowledge.

13.125 Explanation of absence

Absence of evidence, memory, or continuity SHOULD NOT be explained as evidence that an event, preference, or pattern never existed.

13.126 Explanation of stale state

Historical information SHOULD be identified as historical when current applicability is not established.

13.127 Explanation of imported state

Imported continuity SHOULD be identifiable as imported where origin affects confidence, authorization, or interpretation.

13.128 Explanation of human input

Human-authored or reviewed state SHOULD remain distinguishable from member-originated evidence.

13.129 Explanation of system-generated content

System-generated reflections, summaries, or recommendations SHALL NOT be cited back as independent member evidence merely because they appear in an audit trail.

13.130 Audit anti-circularity

Audit systems SHALL NOT create new evidentiary authority merely by recording an existing inference multiple times.

13.131 Record duplication

Replicated logs or traces SHALL NOT be counted as multiple independent confirmations of the same underlying event.

13.132 Audit conflict

Conflicting audit records SHOULD be reconciled or represented as unresolved rather than selecting the record most favorable to system certainty.

13.133 Missing audit record

Absence of an expected audit record MAY indicate a system limitation but SHALL NOT automatically prove that the underlying member event did not occur.

13.134 Audit failure

Failure of the observability system SHALL trigger appropriate degradation of claims that depend on unavailable accountability rather than silent continuation as though auditability remained intact.

13.135 Observability failure containment

A telemetry outage SHALL NOT automatically halt unrelated low-consequence functionality, but uses requiring unavailable accountability SHOULD narrow accordingly.

13.136 Recovery of auditability

After an observability outage, the system SHOULD distinguish reconstructed records from records captured contemporaneously.

13.137 Reconstructed audit state

Reconstruction MAY use accountable sources but SHALL preserve uncertainty about details that cannot be recovered.

13.138 Conformance

A conforming implementation SHALL provide sufficient observability, auditability, and accountable explanation for its material normative responsibilities at the applicable consequence level.

13.139 Implementation independence

LGT-ARCH-001 does not prescribe the storage format, tracing system, audit technology, explanation UI, model introspection technique, or operational tooling used to satisfy this section.

13.140 Observability closing principle

A Living Guide should be able to account for what shaped a material outcome without pretending that accountability requires exposing every hidden computation. Trust comes from traceable evidence, boundaries, uncertainty, correction, and consequence - not from a manufactured transcript of thought.

14. Architectural Security and Trust-Boundary Coordination**14.1 Purpose**

LGT SHALL protect the integrity, confidentiality, availability, authorization boundaries, and accountable operation of its components without confusing technical trust with epistemic authority.

14.2 Security is not evidence

Successful authentication, encryption, signature validation, network trust, or component certification SHALL NOT establish that information is true, evidentially sufficient, current, or representative of the member.

14.3 Security is not consent

Secure processing SHALL NOT make an otherwise unauthorized purpose permissible.

14.4 Security is not constitutional authority

A privileged component SHALL NOT gain authority to redefine observations, evidence, memory, continuity, reflection, or member agency merely because it possesses elevated technical access.

14.5 Trust boundaries

Implementations SHOULD identify material trust boundaries among components, operators, model providers, storage systems, external services, administrative interfaces, member interfaces, and downstream experience systems.

14.6 Boundary crossings

Information crossing a trust boundary SHOULD be subject to controls appropriate to its sensitivity, purpose, authorization, provenance, integrity requirements, and consequence.

14.7 Least privilege

Components and operators SHOULD receive only the permissions materially necessary to perform their governed responsibilities.

14.8 Privilege separation

Administrative, operational, analytical, member-facing, and normative processing privileges SHOULD be separated where combining them would create unnecessary risk.

14.9 Privilege SHALL NOT imply evidence authority

An administrator's ability to edit storage SHALL NOT cause administrator-authored content to become member-originated evidence.

14.10 Service identity

Components communicating across material trust boundaries SHOULD establish the identity of the service or responsibility they are communicating with at a level appropriate to consequence.

14.11 Member authentication

Where an action depends on member identity, the implementation SHOULD establish identity with controls appropriate to the consequence of the action.

14.12 Authentication strength

Higher-consequence actions MAY require stronger authentication than low-consequence viewing or generic interaction.

14.13 Authentication uncertainty

If required identity cannot be established, the system SHOULD narrow or defer identity-dependent actions rather than guess.

14.14 Authentication does not prove authorship

A valid authenticated session MAY support attribution, but SHALL NOT by itself prove that every event in that session expresses the member's considered belief, preference, or intent.

14.15 Authorization after authentication

Authentication SHALL NOT substitute for authorization. A known actor may still lack permission for a particular purpose, component, record, or action.

14.16 Component authorization

Each component SHOULD enforce the authorization relevant to its own responsibility rather than relying exclusively on upstream enforcement.

14.17 Defense in depth

Implementations SHOULD use multiple appropriate controls where failure of a single control could materially compromise member information or normative state.

14.18 Secrets

Credentials, API keys, signing keys, tokens, and other secrets SHOULD be protected from unnecessary exposure and SHALL NOT be embedded into member-facing outputs.

14.19 Secret rotation

Implementations SHOULD support rotation or replacement of material secrets without requiring loss of accountable member continuity.

14.20 Secret compromise

Suspected compromise SHOULD trigger containment appropriate to the affected capability and SHALL NOT automatically invalidate unrelated member evidence.

14.21 Key separation

Where cryptographic keys serve materially different trust purposes, implementations SHOULD avoid unnecessary reuse.

14.22 Encryption in transit

Sensitive member information crossing untrusted or external transport boundaries SHOULD be protected against unauthorized interception appropriate to the deployment context.

14.23 Encryption at rest

Sensitive persisted information SHOULD receive storage protections appropriate to its sensitivity, access model, and consequence.

14.24 Encryption limitations

Encryption SHALL NOT be represented as a substitute for purpose limitation, retention limits, member control, provenance, or evidentiary evaluation.

14.25 Integrity protection

Material normative state SHOULD be protected against unauthorized or accidental alteration appropriate to consequence.

14.26 Integrity verification

Where integrity verification is required, failure to verify SHALL NOT be treated as successful verification.

14.27 Signed state

Digital signatures MAY support origin and integrity claims but SHALL NOT prove the substantive truth of signed member information.

14.28 Trusted sender

A technically trusted sender MAY still provide stale, mistaken, incomplete, unauthorized, or semantically incompatible information.

14.29 Input validation

Components SHOULD validate incoming structures, types, ranges, versions, and required semantics before allowing them to affect governed state.

14.30 Semantic validation

Syntactically valid input SHALL NOT automatically be treated as semantically compatible or evidentially sufficient.

14.31 Untrusted content

Member text, imported files, external content, model output, and third-party data SHOULD be treated as untrusted content where they may influence executable, administrative, retrieval, or instruction pathways.

14.32 Instruction/data separation

Implementations using generative models SHOULD preserve a meaningful distinction between governing instructions and untrusted member or external content.

14.33 Prompt injection resistance

Untrusted content SHALL NOT be allowed to silently redefine constitutional authority, authorization boundaries, component responsibilities, or security controls.

14.34 Retrieved-content resistance

Information retrieved from external sources, memory, journals, documents, or interoperability partners SHALL NOT automatically become governing system instruction.

14.35 Model output validation

Model-generated structured actions or state changes SHOULD be validated before execution when they can materially affect member information, authorization, continuity, or external systems.

14.36 Tool authorization

A model's ability to request a tool action SHALL NOT by itself authorize that action.

14.37 Tool scoping

Tools SHOULD expose only the capabilities and data scope necessary for the component responsibility invoking them.

14.38 High-impact tool actions

Actions capable of deletion, disclosure, account change, payment, external communication, or other material consequence SHOULD receive controls appropriate to that consequence.

14.39 Indirect action

A downstream component SHALL NOT bypass a security or authorization control by requesting another component to perform the same prohibited action indirectly.

14.40 Confused deputy prevention

A more privileged component SHOULD verify that a requesting component or actor is authorized for the requested use rather than blindly lending its privilege.

14.41 Administrative interfaces

Administrative interfaces SHOULD be separated and protected according to the sensitivity and consequence of the controls they expose.

14.42 Administrative observability

Material administrative changes SHOULD remain attributable sufficiently for security review and correction.

14.43 Administrative edits to member state

An administrator MAY correct or maintain state when authorized, but the intervention SHALL remain distinguishable from member-originated information.

14.44 Operator access

Human operator access to sensitive member information SHOULD be purpose-limited and minimized.

14.45 Break-glass access

Emergency administrative access MAY exist but SHOULD be narrowly scoped, time-bounded where practical, and auditable.

14.46 Break-glass authority

Emergency access SHALL NOT grant authority to create unsupported continuity or override constitutional member-agency principles.

14.47 Network segmentation

Implementations MAY segment components or data stores to limit compromise propagation.

14.48 Isolation

Materially compromised components SHOULD be isolatable from unaffected components where continued interaction could spread unaccountable state or unauthorized access.

14.49 Quarantine

Suspect information or components MAY be quarantined pending review without treating quarantine as proof of falsity.

14.50 Compromise containment

A compromise in one component SHOULD be contained according to dependency and access rather than automatically invalidating the entire member journey.

14.51 Compromise scope

Security response SHOULD distinguish affected confidentiality, integrity, availability, authorization, and provenance dimensions.

14.52 Epistemic impact assessment

After a security incident, the system SHOULD assess which observations, evidence, memories, or derivatives may have lost accountable integrity rather than assuming all or none are trustworthy.

14.53 Incident uncertainty

Where compromise scope cannot be established, affected states SHOULD carry appropriate integrity limitation until reconciled.

14.54 Incident response

Implementations SHOULD provide procedures for detection, containment, investigation, recovery, and accountable reconciliation appropriate to their deployment.

14.55 Incident response is not evidence correction

Security remediation SHALL NOT silently rewrite member evidence unless an accountable correction is independently established.

14.56 Recovery

Returning infrastructure to service SHALL NOT automatically restore the trustworthiness of state that may have been altered during compromise.

14.57 Recovery reconciliation

Affected state SHOULD be reconciled against backups, provenance, lifecycle history, authorization changes, member corrections, and other accountable sources before full consequence resumes.

14.58 Backup security

Backups SHOULD receive protections appropriate to the sensitivity and authority of the information they contain.

14.59 Backup restoration

Restoration SHALL preserve current revocation, deletion, forgetting, correction, and authorization boundaries rather than resurrecting obsolete active influence.

14.60 Availability

Security architecture SHOULD protect reasonable availability without sacrificing truthfulness, authorization, or member control merely to keep an interface responsive.

14.61 Denial-of-service degradation

Under resource attack or exhaustion, the system SHOULD narrow capability predictably rather than dropping essential authorization, provenance, or integrity checks.

14.62 Rate limiting

Rate limiting MAY protect availability but SHALL NOT be used to infer member intent or evidentiary significance from request frequency alone.

14.63 Replay resistance

Where repeated messages or actions could duplicate evidence, lifecycle effects, or external consequences, implementations SHOULD prevent unauthorized or accidental replay.

14.64 Idempotency

Material state-changing operations SHOULD use idempotent or equivalent safeguards where duplicate delivery is reasonably foreseeable.

14.65 Freshness

Security-sensitive tokens, assertions, or messages SHOULD include freshness protections appropriate to the risk of replay or stale authorization.

14.66 Session boundaries

Session continuity SHALL NOT automatically become psychological or longitudinal continuity about the member.

14.67 Session termination

Ending a technical session SHOULD terminate applicable session authority without necessarily deleting authorized long-term continuity.

14.68 Device trust

A trusted device MAY reduce authentication friction but SHALL NOT become evidence about the member's beliefs, identity development, or current intent.

14.69 Multi-user devices

Implementations SHOULD account for the possibility that a device or browser may be used by more than one person where attribution matters.

14.70 Account recovery

Account recovery SHOULD restore access without silently rewriting member continuity or treating recovery signals as substantive member evidence.

14.71 Identity merge

Merging accounts or profiles SHOULD require safeguards against combining unrelated people's continuity.

14.72 Identity split

Where information was incorrectly combined across people, the architecture SHOULD support correction and downstream reconciliation.

14.73 Cross-tenant isolation

Multi-tenant implementations SHOULD prevent one member or organization from accessing or influencing another tenant's protected state except through explicitly authorized shared functions.

14.74 Tenant identifiers

Tenant or member identifiers SHOULD be designed to avoid unnecessary exposure and collision across trust boundaries.

14.75 Data exfiltration resistance

Components SHOULD minimize opportunities for model output, logs, errors, exports, tools, or integrations to disclose information beyond authorized scope.

14.76 Error messages

Error handling SHOULD avoid leaking secrets, unnecessary member content, internal credentials, or exploitable implementation detail.

14.77 Logging security

Security logs SHOULD protect sensitive operational information while remaining sufficiently available for accountable incident review.

14.78 Logging privacy

Security logging SHALL remain subject to privacy minimization and SHALL NOT become an unrestricted archive of member content.

14.79 Audit-log access

Access to security and audit records SHOULD be separated from ordinary application access where appropriate.

14.80 Supply-chain trust

Implementations SHOULD account for the security of dependencies, libraries, plugins, models, containers, build systems, and external services capable of affecting normative behavior.

14.81 Dependency provenance

Material software or model dependencies SHOULD be identifiable sufficiently to support vulnerability response and conformance review.

14.82 Dependency update

A security update MAY change implementation behavior and SHOULD trigger regression review where normative responsibilities could be affected.

14.83 Vulnerable dependency

Discovery of a vulnerability SHOULD lead to risk assessment based on actual exposure and consequence rather than automatic epistemic invalidation of all processed state.

14.84 Build integrity

Production artifacts SHOULD be produced and deployed through controls appropriate to the risk of unauthorized modification.

14.85 Deployment integrity

Implementations SHOULD be able to determine which material software/configuration version was active for a relevant outcome where necessary for incident or conformance review.

14.86 Configuration security

Security-sensitive configuration SHOULD be protected from unauthorized change.

14.87 Policy configuration

A security administrator SHALL NOT be able to silently redefine constitutional or evidentiary semantics merely by changing an unrelated infrastructure policy.

14.88 Model supply chain

Model files, endpoints, adapters, fine-tunes, and routing configurations capable of materially changing behavior SHOULD be treated as supply-chain dependencies.

14.89 Model substitution

Substituting a model for security or availability reasons SHOULD preserve the applicable normative responsibilities or trigger degraded operation.

14.90 Model compromise

Suspected model or provider compromise SHOULD trigger containment of affected uses without treating all model-generated historical state as automatically false.

14.91 External provider trust

A contractual or technical relationship with an external provider SHALL NOT make that provider an independent source of epistemic authority over the member.

14.92 External provider minimization

External services SHOULD receive only the information necessary for their authorized function.

14.93 Secure interoperability

Interoperating components SHOULD authenticate and protect exchanges appropriate to consequence while preserving semantic, authorization, provenance, and lifecycle requirements.

14.94 Transport security is not semantic compatibility

A mutually authenticated encrypted connection SHALL NOT prove that the exchanged LGT state has compatible meaning.

14.95 Partner compromise

A compromised interoperability partner SHOULD be isolatable without requiring abandonment of unaffected local continuity.

14.96 Partner revocation

Trust in a partner component or credential SHOULD be revocable independently of the member's underlying evidence and memories.

14.97 Trust establishment

Technical trust SHOULD be explicit enough to determine which component, operator, or credential is being trusted and for what function.

14.98 Trust scope

Trust SHALL be scoped rather than treated as a universal property of an organization, model, component, or person.

14.99 Transitive trust

Trust in one component SHALL NOT automatically extend to every component or service that component trusts.

14.100 Zero-trust compatibility

LGT MAY be implemented using zero-trust security principles, but no particular security architecture is mandated.

14.101 Trust downgrade

A component's technical trust may be reduced or revoked without requiring the architecture to erase independently validated member information previously received from it.

14.102 Trust restoration

Restoring technical trust SHOULD require appropriate security evidence and SHALL NOT automatically restore the epistemic status of disputed state.

14.103 Member-controlled integrations

A member MAY authorize integrations, but authorization SHOULD remain scoped to the information and actions necessary for the integration.

14.104 Integration removal

Removing an integration SHOULD revoke future access and trigger appropriate reconciliation of cached credentials and active data flows.

14.105 Integration history

Historical outcomes produced while an integration was valid MAY remain historically attributable where permitted, without implying continued authorization.

14.106 Export security

Member-directed exports SHOULD be protected against unintended disclosure appropriate to their sensitivity.

14.107 Import security

Imported state SHOULD be treated as untrusted until its structure, provenance, authorization, and integrity are sufficiently evaluated.

14.108 File handling

Uploaded or imported files SHOULD be processed with controls appropriate to malicious content risk and SHALL NOT automatically be executed or treated as governing instruction.

14.109 Content sanitization

Sanitization MAY protect technical systems but SHALL NOT silently alter substantive member meaning without preserving the relevant transformation or limitation.

14.110 Vulnerability disclosure

Implementations SHOULD maintain an appropriate path for reporting material security vulnerabilities.

14.111 Security testing

Implementations SHOULD test controls appropriate to their threat model, including authorization boundaries, tenant isolation, injection resistance, secrets handling, and recovery.

14.112 Security testing SHALL NOT become member evidence

Penetration tests, synthetic probes, and security events SHALL remain operational data rather than evidence about a member.

14.113 Adversarial testing

Model-enabled components SHOULD be tested for attempts to override governing instructions, exfiltrate protected context, broaden tools, or cross authorization boundaries.

14.114 Conformance testing

Security conformance evidence SHOULD demonstrate preservation of normative boundaries under representative attack or failure conditions where applicable.

14.115 Higher-consequence profiles

Higher-consequence profiles MAY require stronger authentication, isolation, signing, audit, recovery, human review, or independent security assessment.

14.116 Low-consequence profiles

Lower-consequence profiles MAY use simpler controls when risk is genuinely lower, but SHALL still preserve core authorization, confidentiality, integrity, and trust distinctions.

14.117 Security exception

A temporary security exception SHOULD be explicit, scoped, time-bounded where practical, and reviewed rather than becoming an invisible permanent bypass.

14.118 Fail secure

Where a required security control cannot be established for a material action, the system SHOULD fail closed for that action unless an explicit bounded profile defines another safe behavior.

14.119 Fail narrow

Security failure SHOULD narrow affected capability rather than broadening data access or component authority.

14.120 No security theater

Visible security features SHALL NOT be represented as proof of overall LGT trustworthiness when material authorization, privacy, evidence, or integrity responsibilities remain unmet.

14.121 No epistemic laundering

Information SHALL NOT gain evidentiary status merely because it passed through a secure, signed, certified, or privileged system.

14.122 No consent laundering

Information use SHALL NOT become authorized merely because it occurs inside a secure environment.

14.123 No privilege laundering

A request SHALL NOT become permissible merely because a more privileged component can technically perform it.

14.124 No compromise-based identity claim

Security anomalies SHALL NOT be used to infer enduring traits, motives, or identity claims about the member without independent accountable evidence.

14.125 Member dignity

Security controls SHOULD avoid unnecessary accusatory or coercive treatment of members when uncertainty about identity or compromise can be handled through bounded verification.

14.126 Member correction after compromise

Members SHOULD retain appropriate means to correct continuity affected by account takeover, misattribution, or compromised input.

14.127 Security and genuine change

Post-incident reconciliation SHALL remain capable of recognizing genuine member change rather than restoring an old model simply because it predates the incident.

14.128 Security and forgetting

Security retention needs MAY justify bounded operational records where permitted, but SHALL NOT become blanket authority to preserve substantive forgotten member information.

14.129 Security and observability

Security events SHOULD integrate with the observability architecture while remaining distinguishable from member evidence and continuity.

14.130 Security and privacy

Security and privacy controls SHOULD reinforce one another while remaining conceptually distinct: security protects against unauthorized access or alteration; privacy governs permissible use and member control.

14.131 Security and interoperability

Technical trust, semantic compatibility, authorization, and evidentiary sufficiency SHALL each be evaluated according to their own responsibility.

14.132 Security and failure containment

A security incident SHALL use the degraded-operation principles of this architecture: contain, narrow, preserve uncertainty, reconcile, and restore progressively.

14.133 Security and explanation

Member-facing security explanations MAY be bounded to avoid exposing exploitable detail but SHALL NOT fabricate reasons or certainty.

14.134 Security and conformance

A conforming implementation SHALL demonstrate security controls appropriate to the normative responsibilities and consequence profile it claims.

14.135 Threat modeling

Implementations SHOULD maintain a threat model proportionate to their architecture, trust boundaries, member information, external dependencies, and consequence.

14.136 Threat-model evolution

Threat models SHOULD be revisited when new components, tools, integrations, models, data flows, or consequence profiles are introduced.

14.137 Reference implementation independence

ConjuresUp security mechanisms MAY illustrate implementation choices but SHALL NOT define the mandatory security stack for LGT.

14.138 Technology neutrality

LGT-ARCH-001 does not prescribe an identity provider, encryption suite, secrets manager, network architecture, SIEM, WAF, sandbox, container platform, signing system, or vulnerability scanner.

14.139 Conformance

A conforming implementation SHALL preserve least privilege, scoped trust, secure boundaries, compromise containment, integrity protection, and the separation of technical trust from epistemic and consent authority.

14.140 Security closing principle

A secure Living Guide protects the conditions under which accountable understanding can exist. Security can establish who or what may participate and whether information was protected in transit or storage; it cannot decide what is true about the member, what the member consented to, or what the Guide is entitled to conclude.

15. Architectural Performance, Scalability, and Resource Governance**15.1 Purpose**

LGT SHALL support increasing members, observations, memories, practices, relationships, integrations, and longitudinal history without sacrificing accountable meaning, member-specific continuity, correction, privacy, authorization, or evidence quality merely to achieve performance.

15.2 Performance is subordinate to correctness

An implementation SHALL NOT knowingly improve latency or throughput by silently weakening normative evidence, authorization, correction, provenance, lifecycle, or member-control requirements.

15.3 Scalability is not simplification of the member

Scaling the system SHALL NOT require reducing a member's lived journey to a fixed persona, permanent archetype, static score, or context-free summary.

15.4 Resource governance

Implementations SHOULD govern compute, storage, retrieval, model context, network, queue, and external-service resources according to the consequence and purpose of the work being performed.

15.5 Consequence-aware allocation

Higher-consequence operations MAY justify more expensive retrieval, verification, reconciliation, explanation, or human review than low-consequence interactions.

15.6 Low-consequence efficiency

Low-consequence operations MAY use bounded shortcuts where those shortcuts do not materially distort the member's continuity or violate another normative requirement.

15.7 Longitudinal growth

The architecture SHOULD assume that a member's relevant history may grow for years rather than treating continuity as a short-lived session artifact.

15.8 Historical depth

Increasing history SHOULD remain addressable without requiring every historical item to occupy active model context for every interaction.

15.9 Active context

Active context SHOULD contain the information materially necessary for the present governed task rather than the maximum information technically available.

15.10 Deep context

Information outside active context MAY remain retrievable when its historical depth is relevant to a later interpretation, correction, contradiction, relationship, milestone, or member request.

15.11 Context selection

Context selection SHOULD be accountable to purpose, relevance, recency, confidence, domain, relationship, lifecycle, member preference, and consequence where those factors matter.

15.12 Context selection is a transformation

When selection materially shapes an outcome, the fact that some continuity was selected and other continuity omitted SHOULD remain explainable at an appropriate level.

15.13 Context omission

Absence from active context SHALL NOT automatically mean that information was forgotten, disproven, irrelevant to all future purposes, or removed from long-term continuity.

15.14 Context overflow

When relevant context exceeds an implementation's processing capacity, the system SHOULD narrow, stage, retrieve iteratively, summarize accountably, or defer rather than silently truncate material evidence.

15.15 Silent truncation

Material evidence SHALL NOT be dropped solely because it appears after an implementation-specific token, record, page, or retrieval limit without an accountable handling strategy.

15.16 Summarization

Summaries MAY reduce resource use, but SHALL remain derivatives rather than replacements for all underlying provenance where deeper recovery remains necessary.

15.17 Summary provenance

A material summary SHOULD preserve sufficient linkage to the information from which it was derived.

15.18 Summary uncertainty

A summary SHALL NOT increase certainty beyond the evidence it summarizes.

15.19 Summary compression loss

Where compression removes nuance material to future use, the limitation SHOULD be recoverable or observable.

15.20 Summary refresh

Long-lived summaries SHOULD be revisable when new evidence, contradiction, correction, forgetting, or genuine member change materially alters their basis.

15.21 Summary anti-circularity

A summary SHALL NOT later be counted as independent confirmation of the underlying information it summarized.

15.22 Hierarchical continuity

Implementations MAY maintain multiple levels of continuity representation, such as observations, memories, bounded summaries, themes, chapters, and higher-order synthesis.

15.23 Hierarchy SHALL preserve dependency

Higher-order continuity SHOULD remain traceable to lower-order accountable sources sufficiently for correction and review.

15.24 Hierarchy SHALL preserve revisability

A higher-order representation SHALL NOT become immutable merely because it is computationally convenient.

15.25 Retrieval

Retrieval SHOULD prioritize accountable relevance rather than only lexical similarity, recency, popularity, or embedding distance.

15.26 Retrieval diversity

Where materially competing or contradictory evidence exists, retrieval SHOULD avoid selecting only evidence that reinforces the currently dominant interpretation.

15.27 Retrieval anti-confirmation bias

Performance optimizations SHALL NOT systematically favor evidence that agrees with cached conclusions merely because it is easier to retrieve.

15.28 Retrieval provenance

Retrieved information SHOULD retain enough provenance and lifecycle state to support responsible downstream interpretation.

15.29 Retrieval freshness

Caches and indexes SHOULD distinguish current, superseded, corrected, stale, or otherwise lifecycle-limited information where those states affect use.

15.30 Retrieval failure

If a material retrieval capability is unavailable, the system SHOULD represent the limitation rather than fabricate continuity from incomplete context.

15.31 Retrieval budget

Implementations MAY use retrieval budgets, but budgets SHOULD be allocated according to task consequence and uncertainty rather than uniformly.

15.32 Iterative retrieval

A component MAY retrieve additional context when initial evidence is insufficient, contradictory, or unexpectedly uncertain.

15.33 Stop conditions

Iterative retrieval SHOULD stop when additional retrieval is unlikely to materially improve the governed outcome relative to its resource cost and consequence.

15.34 Resource cost is not evidence

Greater compute expenditure SHALL NOT itself increase confidence in a proposition.

15.35 Compute budget

Compute budgets MAY vary by member interaction, component, consequence, uncertainty, and operational conditions.

15.36 Compute degradation

When compute is constrained, the system SHOULD narrow sophistication before weakening core evidence, authorization, privacy, or correction responsibilities.

15.37 Model routing

Implementations MAY route tasks among models or algorithms with different cost, latency, or capability characteristics.

15.38 Model routing accountability

A routing decision that materially affects normative behavior SHOULD preserve the applicable capability and consequence requirements.

15.39 Smaller models

A smaller or less expensive model MAY perform bounded tasks when it can satisfy the applicable normative responsibility.

15.40 Model escalation

A task SHOULD be eligible for escalation when uncertainty, ambiguity, consequence, incompatibility, or missing capability exceeds the bounded model's responsibility.

15.41 Model fallback

Fallback to another model SHALL NOT silently broaden authorization, weaken privacy, or erase relevant limitations.

15.42 Model context limits

Model context-window limitations SHALL be treated as implementation constraints rather than truths about what is relevant to the member.

15.43 Batch processing

Non-interactive work MAY be batched where delay does not materially harm member experience, correction propagation, safety, or consequence.

15.44 Interactive work

Member-facing interactions SHOULD prioritize responsiveness while preserving truthful representation of work that remains pending.

15.45 Deferred work

If deeper synthesis or reconciliation is deferred, the system SHOULD distinguish provisional output from completed accountable processing where the distinction matters.

15.46 Queuing

Queued operations SHOULD preserve identity, authorization, provenance, lifecycle, and relevant version context until execution.

15.47 Queue staleness

Before executing materially delayed work, the system SHOULD reassess whether authorization, member state, dependencies, or purpose have changed.

15.48 Queue replay

Duplicate queue delivery SHOULD NOT create duplicate evidence, repeated interventions, or unintended external consequences.

15.49 Backpressure

Components SHOULD support backpressure or equivalent controls so overload in one subsystem does not silently corrupt or discard accountable state.

15.50 Overload behavior

Under overload, implementations SHOULD degrade predictably according to consequence rather than on an arbitrary first-failure basis.

15.51 Admission control

An implementation MAY defer new work when accepting it would threaten the integrity or availability of already-authorized material work.

15.52 Fairness of resources

Resource allocation SHOULD avoid systematically deprioritizing particular members merely because their continuity is longer, more complex, contradictory, or computationally expensive.

15.53 Complexity is not member fault

A complex member history SHALL NOT be treated as evidence that the member is inconsistent, difficult, or less deserving of accurate continuity.

15.54 Per-member isolation

A resource-intensive member workflow SHOULD be containable so it does not unnecessarily degrade unrelated members.

15.55 Tenant scaling

Multi-tenant scaling SHOULD preserve isolation, authorization, and member-specific continuity across partitions, replicas, or service boundaries.

15.56 Horizontal scaling

Components MAY scale horizontally provided replicated or partitioned state preserves applicable semantics, provenance, lifecycle, and correction behavior.

15.57 Partitioning

State MAY be partitioned by member, domain, time, component, tenant, or another accountable boundary.

15.58 Partition boundary

A partitioning strategy SHALL NOT create false independence between records that share the same underlying evidence.

15.59 Cross-partition synthesis

Where synthesis spans partitions, the architecture **SHOULD** preserve dependency identity and correction reach across them.

15.60 Sharding

Sharding is an implementation choice and **SHALL NOT** alter the normative meaning of member continuity.

15.61 Replication

Replicas **SHALL NOT** be counted as independent evidentiary confirmations.

15.62 Replica consistency

Where replicas may temporarily diverge, the implementation **SHOULD** define which operations can tolerate eventual consistency and which require stronger coordination.

15.63 Strong-consistency candidates

Authorization revocation, correction of high-consequence state, identity separation, deletion/forgetting coordination, and irreversible external actions **MAY** require stronger coordination than ordinary low-consequence reads.

15.64 Eventual consistency

Eventual consistency **MAY** be appropriate when temporary divergence cannot materially mislead, violate member control, or create irreversible consequence.

15.65 Stale replica

A stale replica **SHOULD NOT** present obsolete continuity as current when staleness is known and material.

15.66 Conflict resolution

Conflicting replicated state **SHOULD** be reconciled according to normative authority, provenance, correction, lifecycle, and temporal context rather than simple last-write-wins where that would distort meaning.

15.67 Last-write-wins limitation

Last-write-wins **SHALL NOT** be used for member evidence or correction when ordering alone cannot determine semantic authority.

15.68 Caching

Caching **MAY** improve latency but **SHALL** preserve applicable authorization, privacy, correction, lifecycle, and staleness semantics.

15.69 Cache keying

Cache boundaries **SHOULD** prevent one member's protected state from being returned to another member or tenant.

15.70 Cache invalidation

Material correction, revocation, deletion, forgetting, profile change, or semantic-version change **SHOULD** invalidate or reconcile affected cached derivatives.

15.71 Cache miss

A cache miss **SHALL NOT** be interpreted as absence of member history.

15.72 Cache warming

Precomputation **MAY** warm caches when authorized, but speculative processing **SHALL** remain purpose-limited.

15.73 Precomputation

Frequently needed derivatives **MAY** be precomputed if their dependencies, lifecycle, and correction paths remain accountable.

15.74 Precomputed staleness

A precomputed derivative **SHOULD** be reassessed when material dependencies change.

15.75 Indexing

Indexes **MAY** accelerate retrieval but **SHALL** preserve lifecycle and authorization boundaries applicable to indexed content.

15.76 Index lag

Known index lag **SHOULD** be considered when recent correction, revocation, or new evidence could materially change retrieval.

15.77 Embeddings

Embeddings MAY support retrieval but SHALL remain derivatives subject to applicable privacy, deletion, forgetting, and provenance responsibilities.

15.78 Embedding similarity

Similarity SHALL NOT be treated as proof of semantic equivalence, member identity, or evidentiary support.

15.79 Vector-store scaling

Partitioned or distributed vector retrieval SHALL preserve member and tenant isolation.

15.80 Search ranking

Ranking optimizations SHOULD NOT systematically hide contradictory or corrective information material to the governed task.

15.81 Storage tiers

Implementations MAY use hot, warm, cold, archival, or equivalent storage tiers.

15.82 Storage tier is not lifecycle state

Moving information to cheaper storage SHALL NOT itself mean the information is forgotten, superseded, inactive, or no longer governed.

15.83 Archival retrieval

Archived information SHOULD remain retrievable when required for authorized historical reconstruction, correction, or member-directed review.

15.84 Archival cost

Higher retrieval cost MAY affect latency but SHALL NOT justify falsely claiming the historical information does not exist.

15.85 Retention efficiency

Storage efficiency SHOULD be achieved through accountable lifecycle and compression mechanisms rather than indiscriminate deletion of inconvenient history.

15.86 Deduplication

Duplicate storage MAY be deduplicated, but deduplication SHALL preserve distinct provenance, context, authorization, or event identity where those distinctions matter.

15.87 Deduplication is not evidence merging

Similar observations SHALL NOT be merged into one proposition merely because their text or embedding is similar.

15.88 Compression

Technical compression MAY reduce storage without changing normative meaning.

15.89 Semantic compression

Semantic compression SHALL preserve the distinction between source evidence and derivative interpretation.

15.90 Progressive synthesis

Longitudinal continuity MAY be progressively synthesized as history grows.

15.91 Progressive synthesis SHALL remain revisable

Earlier synthesis SHOULD be capable of change when later evidence alters its support.

15.92 Chaptering

Implementations MAY organize long journeys into chapters, phases, milestones, or eras to support retrieval and member comprehension.

15.93 Chapters are not fixed identities

A chapter SHALL NOT become a permanent label that overrides later evidence or genuine change.

15.94 Milestones

Milestones MAY serve as retrieval anchors but SHALL remain grounded in accountable member experience.

15.95 Relationship scaling

Relationship continuity MAY require separate retrieval and synthesis boundaries for different people or relationship contexts.

15.96 Relationship isolation

Information relevant to one relationship SHOULD NOT automatically populate another relationship context merely for retrieval convenience.

15.97 Domain scaling

Domains MAY maintain specialized indexes, summaries, or models while preserving cross-domain provenance and authorization boundaries.

15.98 Cross-domain cost

Expensive cross-domain synthesis SHOULD be performed when materially useful rather than by default for every interaction.

15.99 Practice scaling

Practice recommendation systems MAY precompute candidate pools, scores, or readiness signals if final selection remains accountable to current member state.

15.100 Recommendation cache

Cached recommendations SHOULD be invalidated or reassessed when material member evidence, readiness, constraints, or recent outcomes change.

15.101 Outcome learning

Outcome-learning pipelines MAY process asynchronously provided delayed learning is not falsely represented as already incorporated.

15.102 Sequence learning

Sequence-level learning MAY aggregate across experiences while preserving member-specific evidence and avoiding false certainty from sparse outcomes.

15.103 Population-level optimization

Aggregate patterns MAY improve system efficiency or candidate generation where authorized, but SHALL NOT override the member's own evidence when personalizing their journey.

15.104 Population prior

A population prior MAY initialize uncertainty but SHOULD yield to sufficient member-specific evidence.

15.105 Popularity bias

Resource optimizations SHALL NOT reduce personalization to the most common or cheapest experience merely because it performs well on average.

15.106 Cold start

A new member SHOULD receive appropriately bounded guidance without pretending that longitudinal understanding already exists.

15.107 Warm start

Imported or previously established continuity MAY reduce cold-start limitations when provenance, authorization, and semantic compatibility are sufficient.

15.108 Cost-aware personalization

Cost MAY influence implementation strategy but SHALL NOT be represented as a member preference unless the member actually expressed such a preference.

15.109 External API budgets

Rate limits or costs imposed by external providers SHOULD be handled without fabricating unavailable data.

15.110 External-service degradation

When an external service is unavailable, affected capabilities SHOULD narrow or use accountable cached information with appropriate staleness disclosure.

15.111 Retry

Retries SHOULD avoid duplicate member-facing interventions, evidence events, charges, messages, or irreversible actions.

15.112 Timeout

A timeout SHALL NOT be interpreted as a negative member response or evidence that an event did not occur.

15.113 Circuit breaking

Circuit breakers MAY contain failing dependencies and SHOULD integrate with degraded-operation semantics.

15.114 Resource exhaustion

Memory, disk, token, queue, connection, or compute exhaustion SHOULD fail predictably and preserve accountable state where possible.

15.115 Partial completion

A partially completed multi-stage operation SHOULD preserve which stages completed and which remain unresolved.

15.116 Transaction boundaries

Material state transitions SHOULD use transaction or equivalent reconciliation mechanisms appropriate to their consequence.

15.117 Distributed transactions

LGT does not require global distributed transactions where compensating, idempotent, or reconciled workflows can preserve normative correctness.

15.118 Compensation

A compensating action SHOULD preserve the historical fact that the original action occurred when that fact remains relevant.

15.119 Latency

Latency targets SHOULD be defined by user experience and consequence rather than by a universal LGT threshold.

15.120 Latency transparency

If a deeper accountable result requires additional time, the interface MAY provide a bounded provisional response rather than silently substituting a shallower conclusion.

15.121 Throughput

Throughput improvements SHALL NOT convert member-specific continuity into undifferentiated bulk processing where individual authorization or evidence matters.

15.122 Concurrency

Concurrent operations affecting the same material continuity SHOULD reconcile conflicts according to normative semantics.

15.123 Race conditions

Race conditions SHALL NOT cause corrected, revoked, or superseded information to regain active authority merely because an older operation completed later.

15.124 Ordering

Where event ordering affects meaning, implementations SHOULD preserve or reconstruct ordering sufficiently for the applicable consequence.

15.125 Clock independence

Distributed clock differences SHOULD NOT be resolved solely by trusting the numerically latest timestamp when provenance or causal order establishes a different sequence.

15.126 Causal context

Causal or dependency context MAY be more important than wall-clock order for longitudinal interpretation.

15.127 Resource observability

Implementations SHOULD observe resource pressure sufficiently to distinguish performance degradation from evidence, model, or member-behavior changes.

15.128 Performance telemetry

Performance telemetry SHALL remain operational data and SHALL NOT become evidence about the member's character, motivation, or intent.

15.129 Capacity planning

Implementations SHOULD plan capacity using realistic longitudinal growth, retrieval complexity, correction propagation, interoperability, and peak interaction patterns.

15.130 Capacity testing

Load testing SHOULD include representative long-history members and correction/revocation scenarios rather than only empty or newly created profiles.

15.131 Scale testing

Scale tests SHOULD verify semantic integrity, isolation, retrieval quality, and lifecycle behavior in addition to raw latency and throughput.

15.132 Regression testing

Performance optimizations SHOULD be regression-tested for changes in evidence selection, personalization, contradiction retrieval, correction reach, and privacy boundaries.

15.133 Benchmarking

Benchmarks SHOULD identify what workload, history depth, component profile, model, and infrastructure they actually measure.

15.134 Benchmark limitation

A benchmark SHALL NOT be generalized into a universal LGT performance claim when deployment conditions materially differ.

15.135 Service-level objectives

Implementations MAY define service-level objectives for latency, availability, freshness, propagation, or recovery.

15.136 SLO hierarchy

Service-level objectives SHALL NOT override normative correctness merely because meeting a metric would otherwise be difficult.

15.137 Error budgets

Error budgets MAY govern operational tradeoffs but SHALL NOT authorize systematic violation of member control or evidentiary integrity.

15.138 Graceful degradation

Graceful degradation SHOULD preserve the most foundational responsibilities first: identity/tenant isolation, authorization, member control, evidence integrity, correction, and truthful uncertainty.

15.139 Feature shedding

Optional or expensive capabilities MAY be temporarily shed before foundational continuity responsibilities are weakened.

15.140 Personalization degradation

If personalized continuity cannot be retrieved reliably, the system SHOULD fall back to appropriately generic behavior and identify the limitation rather than invent personalization.

15.141 Explanation degradation

A system SHOULD NOT claim a personalized explanation when the evidence needed to support it was unavailable due to resource constraints.

15.142 Recovery from overload

After overload, delayed queues, caches, indexes, corrections, revocations, and other state SHOULD be reconciled before the system assumes normal freshness.

15.143 Autoscaling

Autoscaling MAY respond to demand but SHALL preserve trust boundaries, tenant isolation, secrets handling, and state semantics in newly created instances.

15.144 Ephemeral compute

Ephemeral workers MAY process member information when authorized, but temporary infrastructure SHALL NOT weaken privacy or provenance requirements.

15.145 Stateless components

A component MAY be operationally stateless while still participating in stateful longitudinal continuity through governed dependencies.

15.146 Stateful components

Stateful components SHOULD expose lifecycle and recovery behavior appropriate to the authority of the state they hold.

15.147 Regional scaling

Deployments MAY distribute processing across regions where applicable authorization, privacy, latency, and legal requirements are satisfied.

15.148 Regional divergence

Regional replicas SHALL NOT develop incompatible member continuity without an accountable reconciliation strategy.

15.149 Data residency

Data-residency constraints MAY affect placement and retrieval strategy but SHALL NOT be silently represented as absence of member history.

15.150 Edge processing

Edge or device-local processing MAY reduce latency or disclosure, provided correction, authorization, and synchronization responsibilities remain accountable.

15.151 Offline operation

Offline-capable components SHOULD distinguish locally provisional state from globally reconciled state where the distinction matters.

15.152 Reconnection

On reconnection, local and remote state SHOULD reconcile according to provenance, authorization, correction, lifecycle, and causal context.

15.153 Resource quotas

Quotas MAY protect system stability but SHOULD be designed so reaching a quota does not silently corrupt or discard member continuity.

15.154 Member-facing quotas

Where a product imposes storage or usage limits, the implementation SHOULD distinguish commercial limits from epistemic claims about what is worth remembering.

15.155 Commercial tiering

Subscription or product tier SHALL NOT determine whether the Guide represents known member information truthfully.

15.156 Tiered capability

Different tiers MAY expose different features or capacities, but normative claims made within an available feature SHALL remain accountable.

15.157 Memory limits

If a product imposes memory limits, selection and eviction SHOULD follow explicit product and memory-governance rules rather than arbitrary infrastructure pressure.

15.158 Eviction

Eviction from a cache SHALL NOT be conflated with forgetting from governed continuity.

15.159 Archival

Archival MAY preserve less frequently needed continuity at lower operational cost without changing its evidentiary status solely because of storage tier.

15.160 Cost observability

Operators SHOULD be able to understand major resource drivers without turning cost telemetry into ungoverned profiling of members.

15.161 Cost attribution

Cost attribution MAY occur at component, capability, workload, or tenant level where useful, but SHOULD avoid unnecessary sensitive member detail.

15.162 Optimization evidence

A performance optimization SHOULD be evaluated using both resource metrics and normative quality metrics.

15.163 Quality metrics

Relevant quality measures MAY include retrieval recall, contradiction retrieval, correction propagation, provenance retention, personalization fidelity, and explanation sufficiency.

15.164 Optimization tradeoff

When a performance optimization creates a measurable normative-quality loss, the implementation SHOULD determine whether the loss is acceptable for the applicable consequence rather than assuming faster is better.

15.165 Hidden degradation

A performance optimization SHALL NOT silently reduce member-specific understanding while continuing to market or represent the result as equivalently personalized.

15.166 Resource-induced bias

Implementations SHOULD test whether resource constraints systematically produce poorer continuity for members with longer histories, less common experiences, multilingual content, or more complex relationships.

15.167 Accessibility under load

Performance degradation SHOULD not disproportionately remove accessibility or member-control functions before optional presentation features.

15.168 Priority inversion

Low-value background processing SHOULD NOT block urgent correction, revocation, security, or high-consequence member-control actions.

15.169 Control-plane priority

Critical control-plane actions MAY receive priority over ordinary synthesis or recommendation work.

15.170 Correction priority

Material correction and revocation propagation SHOULD receive sufficient priority to prevent obsolete state from remaining active solely because background queues are busy.

15.171 Forgetting priority

Required forgetting or deletion coordination SHOULD not be indefinitely deferred for cost optimization.

15.172 Security priority

Security containment MAY temporarily supersede performance objectives when continued operation would expand compromise.

15.173 Member experience

Performance engineering SHOULD preserve a coherent member experience without disguising uncertainty, pending work, or degraded personalization.

15.174 Progressive disclosure

Interfaces MAY progressively disclose deeper continuity or explanation so initial responsiveness does not require loading every historical detail.

15.175 Progressive computation

A fast bounded result MAY be followed by deeper synthesis when the member requests it or consequence warrants it.

15.176 Progressive computation SHALL remain consistent

Later deeper processing SHOULD update or qualify earlier provisional results when new accountable context materially changes them.

15.177 Cancellation

Members or components SHOULD be able to cancel eligible long-running work without cancellation being interpreted as negative evidence about the member.

15.178 Abandoned computation

Partial internal computation that never produced an accountable state transition SHALL NOT be treated as completed member continuity.

15.179 Background synthesis

Background synthesis MAY maintain continuity efficiently when authorized, but SHOULD not generate unnecessary interventions or silently broaden purpose.

15.180 Scheduled maintenance

Maintenance operations SHOULD preserve lifecycle, authorization, correction, and provenance semantics while rebuilding indexes, migrating stores, or compacting data.

15.181 Reindexing

Reindexing SHALL NOT resurrect deleted, forgotten, revoked, or superseded information into active retrieval.

15.182 Migration at scale

Large-scale migration SHOULD validate semantic compatibility and representative member histories rather than only record counts.

15.183 Migration checkpoints

Long migrations SHOULD support checkpoints or reconciliation mechanisms appropriate to failure recovery.

15.184 Dual-running

During migration, old and new systems MAY run in parallel, but duplicate processing SHALL NOT become duplicate evidence.

15.185 Cutover

Cutover SHOULD preserve the current authoritative correction, authorization, lifecycle, and dependency state.

15.186 Rollback

Rollback of infrastructure or software SHALL NOT roll back member corrections, revocations, or other newer authoritative controls without explicit reconciliation.

15.187 Disaster recovery

Disaster recovery SHOULD restore service and accountable continuity while preserving current member-control state to the greatest extent feasible.

15.188 Recovery point

Recovery-point objectives SHOULD account for the consequence of losing recent corrections, consent changes, or member observations, not only raw transaction volume.

15.189 Recovery time

Recovery-time objectives SHOULD distinguish foundational member-control services from optional computational features.

15.190 Resource governance and security

Scaling mechanisms SHALL preserve the security and trust-boundary responsibilities of Section 14.

15.191 Resource governance and privacy

Scaling mechanisms SHALL preserve purpose limitation, minimum-necessary flow, deletion, forgetting, and member-control coordination.

15.192 Resource governance and observability

Performance optimization SHOULD remain observable enough to determine when resource choices materially shaped an outcome.

15.193 Resource governance and interoperability

Performance shortcuts SHALL NOT silently bypass semantic compatibility or version negotiation across independent implementations.

15.194 Resource governance and failure containment

Overload is a failure mode and SHOULD use the same principles of containment, narrowing, truthful limitation, reconciliation, and progressive restoration.

15.195 Resource governance and memory

Detailed memory eligibility, retention, correction, and forgetting remain governed by LGT-MEM-001; this section governs how resource strategy interacts with those responsibilities.

15.196 Resource governance and the Practice Engine

Practice candidate generation, scoring, and sequence optimization MAY be computationally optimized, but final member-facing selection SHALL remain traceable to accountable current evidence and constraints.

15.197 Resource governance and the Living Guide

The Living Guide SHALL NOT sacrifice continuity of understanding merely to fit every interaction into a fixed computational template.

15.198 Resource governance and Continuity Intelligence

Continuity Intelligence SHOULD scale by managing representation, retrieval, hierarchy, and computation - not by pretending that the member's earlier lived experience ceased to matter.

15.199 Reference implementation independence

ConjuresUp MAY demonstrate caching, queues, summaries, model routing, storage tiers, or retrieval strategies, but its specific performance stack SHALL NOT become mandatory LGT architecture.

15.200 Technology neutrality

LGT-ARCH-001 does not prescribe a database, vector store, cache, queue, cloud provider, model, orchestration system, autoscaler, storage tier, sharding scheme, or observability platform.

15.201 Conformance

A conforming implementation SHALL preserve accountable member-specific continuity, correction, authorization, privacy, provenance, and truthful degradation while operating within its declared resource and scalability profile.

15.202 Performance closing principle

A Living Guide should become more capable as its relationship with a member grows, not less faithful because the history became expensive to process. Scale is successful when the architecture can carry more continuity with greater efficiency while preserving the evidence, revisions, contradictions, and lived change that give that continuity meaning.

16. Architectural Deployment, Operations, Lifecycle Management, and Production Change Governance

16.1 Purpose

LGT SHALL permit software, models, infrastructure, schemas, policies, and implementations to evolve without silently changing the meaning, authority, provenance, correction state, authorization, or lived continuity of the member.

16.2 Production change is a continuity event

A production change that can materially alter normative behavior SHOULD be treated as an accountable system event rather than an invisible implementation detail.

16.3 Member continuity outlives releases

A member's accountable journey SHALL NOT reset merely because an application, model, component, storage engine, or deployment version changes.

16.4 Software version is not member truth

A release identifier MAY explain system behavior but SHALL NOT become evidence about the member.

16.5 Known-good baseline

Implementations SHOULD identify a known-good production baseline from which material changes are evaluated and to which affected software behavior can be compared.

16.6 Promotion

A release SHOULD be promoted only after the validation appropriate to the responsibilities and consequence it changes.

16.7 Promotion evidence

Promotion evidence SHOULD identify the version, scope, changed responsibilities, validation performed, known limitations, and rollback or containment path where material.

16.8 Historical QA is not current QA

A previously validated release SHALL NOT prove that a changed release remains conforming.

16.9 Isolated change

Material changes SHOULD be scoped narrowly enough that failures can be attributed, reviewed, and reversed without unnecessarily disturbing unrelated capabilities.

16.10 Change bundling

Large bundles of unrelated normative changes SHOULD be avoided when they make accountability, regression detection, or rollback materially harder.

16.11 Environment separation

Development, test, staging, and production environments SHOULD be separated sufficiently to prevent experimental state, credentials, synthetic data, or unvalidated behavior from silently entering production.

16.12 Production data in non-production

Use of production member information outside production SHOULD be minimized, authorized, and protected according to applicable privacy and security requirements.

16.13 Synthetic test data

Synthetic data MAY support testing but SHALL NOT be treated as proof that long-history, contradictory, corrected, multilingual, or otherwise complex real continuity behaves correctly.

16.14 Representative validation

Pre-production validation SHOULD include representative continuity depth and lifecycle conditions for the changed responsibility.

16.15 Configuration as change

Configuration capable of materially changing normative behavior SHALL be governed as production change even when no application code is deployed.

16.16 Policy as change

A policy, prompt, routing rule, threshold, model instruction, or capability profile that materially changes normative behavior SHOULD be versioned or otherwise accountable.

16.17 Model as change

Changing a model, provider, fine-tune, adapter, tool policy, or model-routing rule SHOULD trigger validation proportional to the normative responsibilities affected.

16.18 Infrastructure as change

Infrastructure changes SHOULD be evaluated for effects on availability, latency, ordering, isolation, privacy, storage semantics, correction propagation, and other relevant normative behavior.

16.19 Schema as change

Schema changes SHALL preserve or explicitly migrate the semantic meaning and lifecycle of governed state.

16.20 Dependency as change

A dependency upgrade MAY be operationally routine but SHOULD trigger regression review when it can affect normative behavior, security, rendering, interoperability, or state semantics.

16.21 Feature flags

Feature flags MAY support controlled rollout, but a flag that changes material normative behavior SHOULD be versioned, observable, scoped, and reversible.

16.22 Feature-flag authority

A feature flag SHALL NOT silently override constitutional, authorization, privacy, evidence, correction, or member-agency requirements.

16.23 Flag retirement

Temporary feature flags SHOULD be retired or incorporated into explicit configuration once their rollout purpose has ended.

16.24 Dark launches

A capability MAY be deployed without member-facing activation for validation, provided background processing remains authorized and purpose-limited.

16.25 Shadow processing

Shadow processing MAY compare new and old behavior, but shadow outputs SHALL NOT influence active member continuity unless independently promoted and authorized.

16.26 Dual running

Old and new components MAY operate in parallel during transition, but duplicate processing SHALL NOT become duplicate evidence or duplicate member intervention.

16.27 Canary rollout

Canary releases MAY limit exposure to new behavior, but canary membership SHALL NOT be inferred to describe a member trait or preference.

16.28 Progressive rollout

Progressive rollout SHOULD preserve the ability to identify which version materially shaped a member-facing outcome.

16.29 Rollout segmentation

Operational rollout segments SHOULD remain operational metadata rather than member evidence unless the member independently supplied equivalent information.

16.30 Compatibility window

Interoperating versions MAY coexist during a declared compatibility window when semantic and security compatibility are sufficient.

16.31 Compatibility is multidimensional

Version compatibility SHOULD consider syntax, semantics, authorization, provenance, lifecycle, security, and consequence rather than only transport success.

16.32 Backward compatibility

A newer component SHOULD preserve the meaning of supported older state or explicitly transform it with accountable migration.

16.33 Forward compatibility

An older component encountering unsupported newer semantics SHOULD narrow, reject, or preserve unknown state rather than silently reinterpret it.

16.34 Unknown fields

Unknown fields SHOULD NOT be discarded when doing so could prevent later recovery of member-controlled or semantically material information.

16.35 Version negotiation

Independent components SHOULD negotiate or otherwise establish compatible behavior where incompatible versions could materially change meaning.

16.36 Version pinning

Critical workflows MAY pin a component, model, schema, or policy version when reproducibility or transition safety requires it.

16.37 Pinning limitation

Version pinning SHALL NOT become an excuse to preserve known incorrect or insecure behavior indefinitely.

16.38 Deprecation

Deprecation SHOULD identify the affected capability, replacement or migration path, compatibility period where applicable, and consequence of continued use.

16.39 Removal

A deprecated capability SHOULD NOT be removed in a way that silently destroys accountable member continuity that remains governed or portable.

16.40 End of support

End-of-support status SHOULD be operationally explicit and SHALL NOT by itself invalidate historical member state produced while the component was conforming.

16.41 Release ownership

Material production changes SHOULD have an accountable owner or responsibility capable of coordinating validation, deployment, monitoring, containment, and rollback.

16.42 Operational ownership

Every production component carrying a material LGT responsibility SHOULD have an identifiable operational responsibility even when implementation is automated.

16.43 Shared ownership

Shared responsibility across teams or organizations SHOULD define boundaries sufficiently to avoid gaps where each party assumes another owns correction, security, or recovery.

16.44 Runbooks

Material operational responsibilities SHOULD have procedures sufficient for predictable response to known failure, rollback, migration, and recovery conditions.

16.45 Runbooks are not authority

Operational documentation SHALL NOT override the normative specification merely because a procedure is convenient.

16.46 Change record

A material production change SHOULD preserve sufficient record to identify what changed, when, where, under whose operational responsibility, and with what declared scope.

16.47 Change rationale

A change record SHOULD identify the operational or product purpose of a material change without requiring disclosure of private chain-of-thought.

16.48 Change provenance

The deployed artifact, configuration, model, policy, or migration responsible for material behavior SHOULD be identifiable for later review.

16.49 Release artifact integrity

Promoted artifacts SHOULD be protected against unauthorized modification between validation and deployment.

16.50 Reproducible identity

An implementation SHOULD be able to identify the material release actually running even when exact binary reproducibility is not technically available.

16.51 Immutable identifiers

Release identifiers SHOULD NOT be silently reused for materially different production artifacts.

16.52 Hotfixes

Emergency hotfixes MAY use accelerated procedures, but SHOULD preserve minimum validation, change identity, observability, and post-deployment review appropriate to consequence.

16.53 Emergency change

Urgency SHALL NOT convert an unvalidated change into evidence that the change is safe or conforming.

16.54 Post-emergency review

Emergency changes SHOULD receive retrospective validation and documentation once the immediate risk is contained.

16.55 Rollback readiness

Material releases SHOULD define whether and how software behavior can be rolled back before deployment when rollback is technically meaningful.

16.56 Rollback is not time travel

Rolling back software SHALL NOT automatically roll back newer authoritative member corrections, consent changes, revocations, security controls, or genuine lived events.

16.57 State-aware rollback

A rollback SHOULD reconcile state written by the newer release before assuming the older release can interpret it safely.

16.58 Irreversible migration

If a migration cannot be safely reversed, the release SHOULD identify that fact before promotion and provide an alternative recovery or forward-fix strategy.

16.59 Forward fix

A forward fix MAY be preferable to rollback when rollback would corrupt newer authoritative state or violate compatibility.

16.60 Rollback trigger

Rollback or containment criteria SHOULD be defined around material regression, integrity, security, authorization, privacy, member-control, or availability failures rather than cosmetic metrics alone.

16.61 Partial rollback

A component or feature MAY be rolled back independently when isolation preserves unaffected responsibilities.

16.62 Rollback observability

The system SHOULD preserve which members, states, or outcomes were affected by a faulty version when that information is necessary for reconciliation.

16.63 Regression

A regression is any change that weakens an expected normative, functional, security, privacy, interoperability, accessibility, or member-experience property relative to the promoted baseline.

16.64 Visual regression

Reference implementations MAY treat visual regressions as release failures where presentation affects usability, accessibility, trust, or intended product quality.

16.65 Functional regression

A release SHALL NOT be promoted as equivalent when a previously supported material function no longer behaves according to its declared contract.

16.66 Normative regression

A release that materially weakens evidence, correction, authorization, privacy, provenance, member agency, or another normative responsibility SHALL be treated as a conformance regression.

16.67 Regression detection

Automated and human validation SHOULD be combined where either alone is insufficient to detect the relevant failure class.

16.68 Human QA

Human review MAY remain an authoritative promotion gate for presentation, coherence, interaction, or other properties not reliably established by automation.

16.69 Automated QA

Automated tests SHOULD protect stable contracts, invariants, migrations, security boundaries, and regression-prone behavior where practical.

16.70 QA evidence

QA records SHOULD identify the version and scope actually tested rather than being copied forward from an earlier release.

16.71 QA failure

A failed promotion candidate SHOULD NOT silently become the new baseline.

16.72 Baseline promotion

Only a release that passes the applicable promotion criteria SHOULD become the next known-good baseline.

16.73 Baseline preservation

The immediately previous known-good baseline SHOULD remain recoverable for an appropriate period when rollback or comparison remains operationally relevant.

16.74 Baseline lineage

Release lineage SHOULD preserve which approved baseline each increment was built from.

16.75 Branching

Parallel development branches MAY exist, but production lineage SHOULD remain clear enough to establish which changes were actually promoted.

16.76 Merge governance

Merging independently validated changes SHOULD trigger integration validation when their interaction can create new behavior.

16.77 Environment drift

Material differences among development, staging, and production environments SHOULD be known where they can affect validation reliability.

16.78 Drift detection

Implementations SHOULD detect or periodically review configuration and dependency drift in material production components.

16.79 Unmanaged change

Material production changes outside the governed release path SHOULD be detected, reconciled, and reviewed.

16.80 Manual production edit

A manual edit MAY be necessary during incident response, but SHOULD become an accountable change rather than remaining undocumented state.

16.81 Database migration

Database migrations SHOULD preserve member identity, provenance, lifecycle, correction, authorization, and dependency semantics.

16.82 Migration validation

A migration SHOULD be validated using semantic invariants in addition to row counts, file counts, or successful execution status.

16.83 Migration completeness

Successful completion of a migration process SHALL NOT prove that every member state was semantically migrated correctly.

16.84 Migration sampling

Representative sampling MAY supplement automated migration checks, particularly for long-history or unusual continuity states.

16.85 Migration reconciliation

Failures or partial migrations SHOULD remain identifiable and repairable rather than being silently skipped.

16.86 Migration checkpoint

Long-running migrations SHOULD support checkpoints or equivalent recovery controls where restart cost or partial state is material.

16.87 Online migration

Online migrations SHOULD account for concurrent member activity so new observations, corrections, or revocations are not lost during transition.

16.88 Write fencing

Where necessary, implementations MAY fence or route writes during cutover to prevent incompatible concurrent state changes.

16.89 Data copy

Copying state during migration SHALL NOT create a second independent evidentiary source.

16.90 Data transformation

Migration transformations SHOULD remain attributable where they materially change representation or interpretation.

16.91 Lossy migration

A migration that cannot preserve material semantics SHOULD declare the loss and narrow affected downstream use rather than conceal it.

16.92 Migration of corrected state

Corrected or superseded information SHALL NOT regain active authority merely because an older representation was restored or reindexed.

16.93 Migration of forgotten state

Forgotten or deleted information SHALL NOT be resurrected into active continuity by backup restoration, reindexing, or schema migration except where a separately governed legal or operational record remains permitted and isolated.

16.94 Migration of authorization

Current authorization and revocation state SHOULD survive migration independently of historical defaults.

16.95 Migration of provenance

Material provenance SHOULD survive migration or be explicitly marked as degraded where preservation is impossible.

16.96 Migration of confidence

Confidence SHOULD NOT be increased merely because state was normalized, reserialized, or moved into a newer schema.

16.97 Migration of timestamps

Temporal semantics SHOULD distinguish original event time from migration or rewrite time.

16.98 Reindexing

Reindexing SHOULD preserve current lifecycle and authorization state and SHALL NOT convert archival or superseded material into current evidence.

16.99 Cache rebuild

Cache rebuilds SHOULD derive from current accountable state rather than stale snapshots where the difference is material.

16.100 Search-index rebuild

Search or vector-index rebuilds SHOULD preserve tenant isolation, deletion, forgetting, correction, and provenance responsibilities.

16.101 Model migration

Moving continuity between model providers or model families SHOULD preserve the governed state independently of model-specific hidden representations where possible.

16.102 Model memory

Provider-specific conversational or hidden model memory SHALL NOT be the sole authoritative store for member continuity.

16.103 Model replacement

A new model SHOULD be evaluated for changes in interpretation, uncertainty, instruction following, tool behavior, refusal behavior, and personalization fidelity relevant to its LGT role.

16.104 Model regression

Improvement on generic benchmarks SHALL NOT prove that a model change improves LGT continuity behavior.

16.105 Model rollback

Rolling back a model SHOULD NOT require rolling back independently governed member state.

16.106 Prompt migration

Prompt or instruction changes that materially alter normative behavior SHOULD be regression-tested similarly to code changes.

16.107 Prompt provenance

The applicable prompt or instruction version SHOULD be identifiable where necessary to explain a material outcome without requiring protected prompt disclosure.

16.108 Policy migration

Changing thresholds or policy logic SHOULD preserve the distinction between changed system policy and changed member evidence.

16.109 Threshold change

A member SHALL NOT be represented as having changed merely because the system changed a scoring threshold.

16.110 Reclassification

If a new version reclassifies historical state under new rules, the system SHOULD distinguish reclassification from new member evidence.

16.111 Historical reinterpretation

New analytical capability MAY reinterpret old evidence, but the reinterpretation SHOULD preserve the historical context and version of the new interpretation.

16.112 Historical output

Previously delivered guidance MAY remain historically attributable to the version that produced it even if current guidance differs.

16.113 Retroactive correction

A software fix MAY justify recomputing affected derivatives, but SHALL NOT silently alter original member-originated events.

16.114 Retroactive recomputation

Recomputed state SHOULD identify the governing version and preserve whether the result is newly derived from historical evidence.

16.115 Member-visible change

Where an upgrade materially changes the Guide's interpretation of existing continuity, the member SHOULD receive an appropriate explanation when the change affects their experience.

16.116 No upgrade-induced destiny

A new model or algorithm SHALL NOT present a changed interpretation as though the member suddenly became a different person solely because the software changed.

16.117 Genuine change preservation

Deployment reconciliation SHOULD preserve the ability to distinguish genuine member evolution from software-induced reclassification.

16.118 Operational state

Health checks, deployment status, queue depth, cache state, and other operational signals SHALL remain operational data rather than member evidence.

16.119 Health checks

Health checks SHOULD test the responsibilities necessary to determine whether a component can safely serve its declared role.

16.120 Liveness

Liveness alone SHALL NOT prove that a component is semantically correct, authorized, current, or ready for high-consequence use.

16.121 Readiness

Readiness checks SHOULD include dependencies material to the component's declared production responsibility.

16.122 Dependency degradation

A component MAY remain live while not ready for particular consequences; the architecture SHOULD preserve that distinction.

16.123 Maintenance mode

Maintenance mode MAY narrow capability, but SHOULD preserve member control, truthful status, and critical correction or security paths where feasible.

16.124 Scheduled maintenance

Scheduled maintenance SHOULD communicate material availability limitations without fabricating member-specific reasons.

16.125 Unscheduled outage

An outage SHALL NOT be interpreted as member disengagement, refusal, or lack of interest.

16.126 Operational alert

Alerts SHOULD identify material service or integrity conditions without creating unnecessary sensitive member copies.

16.127 Alert ownership

Material alerts SHOULD route to an accountable operational responsibility capable of acting on them.

16.128 Alert closure

Closing an alert SHALL NOT prove that all affected member state has been reconciled.

16.129 Incident declaration

A material incident SHOULD have a defined scope, owner, status, and recovery responsibility.

16.130 Incident/change distinction

Emergency response MAY require change, but incident state and release state SHOULD remain distinguishable for later review.

16.131 Incident containment

Containment SHOULD narrow affected capability before allowing uncertainty to spread into unrelated continuity.

16.132 Incident recovery

Recovery SHOULD include reconciliation of affected member state, not merely restoration of service availability.

16.133 Incident review

Material incidents SHOULD be reviewed for root conditions, affected responsibilities, detection gaps, member impact, and prevention or containment improvements.

16.134 Blamelessness and accountability

Operational review SHOULD focus on system conditions and accountable improvement without erasing individual responsibility where an authorized action materially mattered.

16.135 Member notification

Member notification MAY be required when an incident materially affects their information, access, continuity, or rights according to applicable policy or law.

16.136 Notification accuracy

Incident communication SHALL NOT overstate certainty about impact before the affected scope is established.

16.137 Recovery completion

An incident **SHOULD NOT** be considered fully reconciled solely because infrastructure metrics returned to normal.

16.138 Operational documentation

Architecture diagrams, dependency maps, runbooks, ownership records, and deployment manifests **SHOULD** remain sufficiently current for the responsibilities they support.

16.139 Documentation drift

Material documentation drift **SHOULD** be corrected when it could mislead operators about production behavior or responsibility.

16.140 Dependency map

Material runtime dependencies **SHOULD** be identifiable sufficiently for incident containment, change impact analysis, and recovery.

16.141 Data-flow map

Material member-information flows **SHOULD** be identifiable sufficiently for privacy, security, migration, and change review.

16.142 Capability map

Deployments **SHOULD** be able to identify which component provides each material LGT capability or responsibility.

16.143 Orphaned responsibility

A capability **SHALL NOT** be considered operationally governed when no component or owner is accountable for its material responsibilities.

16.144 Service retirement

Retiring a service **SHOULD** migrate, archive, export, or otherwise reconcile governed state before shutdown.

16.145 Component replacement

A replacement component **SHOULD** prove compatibility with the responsibilities it inherits rather than relying on functional resemblance alone.

16.146 Ownership transfer

Operational ownership **MAY** transfer, but the transfer **SHOULD** preserve access controls, documentation, incident history, and responsibility boundaries.

16.147 Organizational change

Team or vendor reorganization **SHALL NOT** silently alter member authorization, evidence authority, or continuity semantics.

16.148 Provider exit

The architecture **SHOULD** support replacement or exit of external providers without requiring abandonment of portable member continuity where portability is within scope.

16.149 Credential transition

Credential rotation during deployment **SHOULD** preserve service continuity without creating overlapping privilege longer than necessary.

16.150 Secret rollout

New and old secrets **MAY** coexist during a bounded transition when required, with explicit retirement of obsolete credentials.

16.151 Certificate expiry

Certificate or credential expiry **SHOULD** fail as a technical trust condition rather than being misinterpreted as invalid member evidence.

16.152 Clock and scheduler changes

Changes to clocks, schedulers, time zones, or recurrence logic **SHOULD** be tested where temporal semantics affect practices, reminders, lifecycle, or ordering.

16.153 Locale changes

Locale, language, formatting, or rendering changes **SHOULD** preserve substantive meaning and accessibility.

16.154 UI change

Presentation changes **SHOULD** preserve the member's ability to understand, correct, consent, refuse, navigate, and distinguish system state where those functions are normative.

16.155 API change

API changes SHOULD preserve declared contracts or use explicit versioning and migration.

16.156 Event-contract change

Event schema changes SHOULD preserve event identity, provenance, replay semantics, and consumer compatibility.

16.157 Consumer lag

A slower-upgrading consumer SHOULD not silently misinterpret newer event semantics.

16.158 Producer lag

A newer consumer SHOULD handle supported older events without rewriting their historical meaning.

16.159 Release sequencing

Dependent component releases SHOULD be sequenced or made compatible so intermediate states remain safe.

16.160 Partial deployment

A partially deployed fleet SHOULD remain within declared compatibility and normative boundaries.

16.161 Failed deployment

A failed deployment SHOULD preserve enough state to determine which instances or members were exposed to the failed version.

16.162 Deployment retry

Retrying a deployment SHALL NOT duplicate migrations, interventions, messages, charges, or evidence events.

16.163 Idempotent migration

Migrations SHOULD be idempotent or otherwise protected against unsafe repeat execution where practical.

16.164 Backup before change

Backups or equivalent recovery points SHOULD be created before high-risk changes when restoration is an appropriate recovery mechanism.

16.165 Backup validity

A backup SHOULD be validated sufficiently to establish that it can support the intended recovery purpose.

16.166 Backup age

Backup age SHOULD be considered against the risk of losing newer corrections, revocations, consent changes, and member events.

16.167 Restore rehearsal

Higher-consequence deployments SHOULD periodically validate restoration or recovery procedures rather than assuming backups are usable.

16.168 Restore reconciliation

Restored data SHOULD reconcile with authoritative newer controls before resuming full use.

16.169 Disaster failover

Failover to another environment or region SHOULD preserve member identity, authorization, lifecycle, provenance, and correction semantics.

16.170 Failback

Returning from a failover environment SHOULD reconcile state created during the failover period.

16.171 Operational scalability

Operational procedures SHOULD scale with the number of components and members without abandoning accountable ownership or change traceability.

16.172 Automation

Deployment and operational automation MAY reduce human error but SHALL remain governed and observable where it can materially change production state.

16.173 Automated promotion

Automated promotion MAY be used when machine-verifiable gates are sufficient for the changed scope; human gates MAY remain required for other properties.

16.174 Automated rollback

Automated rollback MAY contain clear regressions but SHOULD account for state compatibility before reverting.

16.175 Self-healing

Self-healing infrastructure MAY restart or replace unhealthy components but SHALL NOT silently repair member continuity by inventing or discarding substantive state.

16.176 Auto-remediation

Automated remediation SHOULD operate within explicit authority and preserve an audit trail appropriate to consequence.

16.177 Change freeze

Implementations MAY use change freezes during high-risk periods, incidents, migrations, or major launches.

16.178 Freeze exception

Exceptions to a change freeze SHOULD be explicit and proportionate to the risk being addressed.

16.179 Release cadence

LGT does not prescribe a release cadence; cadence SHOULD reflect the implementation's ability to validate and safely operate change.

16.180 Small increments

Small isolated increments are RECOMMENDED when they materially improve attribution, QA, rollback, and regression containment.

16.181 Large release

A large release MAY be appropriate when atomic migration or compatibility requirements make separation riskier, but the broader scope SHOULD receive correspondingly broader validation.

16.182 Golden baseline principle

A known-good baseline SHOULD remain the reference point until the candidate release passes the applicable QA and conformance gates.

16.183 Failure rebuild

When a candidate fails materially, rebuilding from the last known-good baseline MAY be safer than layering corrective changes on top of uncertain state.

16.184 No regression acceptance by fatigue

Repeated difficulty fixing a regression SHALL NOT make the regression conforming merely because the team wishes to move forward.

16.185 Proportional acceptance

A known difference MAY be accepted when it is understood, bounded, non-material to the applicable requirements, and explicitly judged not to constitute a regression.

16.186 Cosmetic variance

Renderer, pagination, or presentation variance MAY be acceptable when complete content, navigation, readability, and normative meaning remain intact.

16.187 Production parity

Validation SHOULD account for material differences between the test renderer or environment and the actual production/member environment.

16.188 Change and observability

Section 13 observability SHOULD make it possible to identify which production change materially shaped an affected outcome.

16.189 Change and security

Section 14 security controls SHOULD protect the release, deployment, credentials, dependencies, and administrative paths used to change production.

16.190 Change and resource governance

Section 15 performance controls SHOULD prevent rollout or migration load from weakening foundational responsibilities.

16.191 Change and privacy

Production change SHALL preserve purpose limitation, consent, revocation, deletion, forgetting, and minimum-necessary information flow.

16.192 Change and interoperability

Version evolution SHOULD preserve declared interoperability or explicitly negotiate incompatible transitions.

16.193 Change and correction

A release SHALL preserve correction reach across migrated, cached, indexed, summarized, or recomputed state.

16.194 Change and provenance

A release SHALL NOT erase the distinction between original member evidence and state newly transformed by the release.

16.195 Change and explanation

When changed system behavior materially changes a member-facing result, the architecture SHOULD be capable of distinguishing software change from member change.

16.196 Change and constitutional alignment

No deployment mechanism, operational emergency, or product release SHALL grant authority to violate the constitutional boundaries of the Living Guide.

16.197 Reference implementation learning

ConjuresUp MAY demonstrate the value of isolated production sprints, human QA, known-good baselines, rollback, changelogs, manifests, and regression reports, but those exact artifacts are not universally mandated by LGT.

16.198 Technology neutrality

LGT-ARCH-001 does not prescribe Git, GitHub, CI/CD software, containers, orchestration platforms, cloud providers, deployment strategies, issue trackers, monitoring vendors, or release-management tools.

16.199 Conformance

A conforming implementation SHALL govern material production change so software evolution does not silently rewrite member truth, bypass member control, destroy provenance, resurrect corrected state, or weaken declared normative responsibilities.

16.200 Deployment closing principle

A Living Guide is expected to evolve for as long as the member does. Its software may be replaced many times, but the member's journey must remain more durable than any release. Good operations change the system without silently changing what happened to the person.

17. Architectural Testing, Validation, Conformance, and Reference-Architecture Verification**17.1 Purpose**

LGT SHALL support demonstrable verification that an implementation satisfies the normative responsibilities it claims, rather than treating successful execution, polished output, benchmark performance, or vendor assertion as sufficient evidence of conformance.

17.2 Conformance is evidence-based

A conformance claim SHOULD be supported by identifiable validation evidence appropriate to the claimed scope and consequence.

17.3 Implementation QA and specification conformance

Implementation QA evaluates whether a particular product behaves as intended; specification conformance evaluates whether the implementation satisfies applicable LGT normative requirements. Neither SHALL automatically substitute for the other.

17.4 Passing tests is not universal proof

A finite test suite MAY support a conformance claim but SHALL NOT be represented as proof that every possible future state, member journey, integration, or failure condition is correct.

17.5 Declared scope

A conformance claim SHOULD identify the implementation, version, profile, components, capabilities, interfaces, and normative publications within scope.

17.6 Excluded scope

Known exclusions SHOULD be explicit rather than hidden behind a general claim of LGT compliance.

17.7 Version-specific claim

Conformance evidence SHALL apply to the version actually tested and SHALL NOT automatically transfer to a materially changed release.

17.8 Profile-aware validation

Validation SHOULD be performed against the consequence and capability profile the implementation claims rather than an undefined universal workload.

17.9 Requirement traceability

Normative requirements SHOULD be traceable to one or more validation methods, implementation responsibilities, or justified non-applicability determinations.

17.10 SHALL requirements

A claimed conforming implementation SHALL satisfy every applicable SHALL requirement within its declared scope.

17.11 SHOULD requirements

Deviation from an applicable SHOULD requirement SHOULD be documented with the implementation's rationale and the alternative control or limitation where relevant.

17.12 MAY requirements

A MAY requirement creates no obligation unless the implementation claims the optional capability.

17.13 Non-applicability

A requirement MAY be marked non-applicable only when the declared architecture genuinely does not exercise the responsibility addressed by that requirement.

17.14 Non-applicability is not failure avoidance

An implementation SHALL NOT redefine its scope after testing merely to hide a failed responsibility that it actually performs.

17.15 Validation categories

A complete validation program SHOULD combine structural, functional, semantic, longitudinal, security, privacy, interoperability, operational, performance, accessibility, and member-experience testing where applicable.

17.16 Structural validation

Structural tests SHOULD verify required interfaces, schemas, identifiers, lifecycle states, version metadata, provenance fields, and other machine-verifiable contracts.

17.17 Functional validation

Functional tests SHOULD verify that components perform their declared responsibilities under expected conditions.

17.18 Semantic validation

Semantic tests SHOULD verify that technically valid information retains the intended LGT meaning across transformations, storage, retrieval, migration, and interoperability.

17.19 Negative testing

Validation SHOULD include conditions that must be rejected, narrowed, quarantined, deferred, or represented as uncertain.

17.20 Boundary testing

Tests SHOULD exercise authorization, tenant, component, trust, lifecycle, and semantic boundaries rather than only happy-path flows.

17.21 Longitudinal testing

Implementations SHOULD test member continuity across extended sequences of observations, practices, corrections, contradictions, relationships, chapters, and software changes.

17.22 Long-history fixtures

Test suites SHOULD include histories sufficiently deep to expose retrieval, summarization, migration, context-selection, and performance behavior that short synthetic sessions cannot reveal.

17.23 Genuine change scenario

Validation SHOULD include a member whose later evidence genuinely differs from earlier evidence and verify that the Guide can update without erasing historical context.

17.24 Correction scenario

Validation SHOULD verify that an explicit correction reaches affected memories, summaries, indexes, caches, recommendations, explanations, and other derivatives according to dependency.

17.25 Contradiction scenario

Validation SHOULD verify that materially contradictory evidence can remain visible and does not disappear merely because one interpretation currently has greater confidence.

17.26 Uncertainty scenario

Validation SHOULD verify that insufficient evidence remains insufficient rather than being converted into confident personalization.

17.27 Sparse-evidence scenario

A new or sparsely observed member SHOULD receive bounded behavior without false claims of longitudinal understanding.

17.28 Rich-evidence scenario

A member with extensive history SHOULD receive behavior that demonstrably uses relevant personal continuity rather than degrading to generic output.

17.29 Forgetting scenario

Where forgetting is supported, validation SHOULD verify removal from active influence and affected derivatives while respecting separately governed retained records.

17.30 Revocation scenario

Validation SHOULD verify that revoked authorization stops future use within the affected purpose and propagates appropriately.

17.31 Deletion scenario

Deletion tests SHOULD verify that deleted governed information does not reappear through caches, indexes, replicas, backups, migration, or recomputation contrary to the applicable lifecycle rules.

17.32 Identity separation scenario

Validation SHOULD verify that information belonging to different members or profiles cannot be merged through retrieval, migration, account recovery, or tenancy errors.

17.33 Identity merge scenario

Where account merge is supported, tests SHOULD verify explicit authorization and preservation of provenance rather than blind concatenation.

17.34 Relationship isolation scenario

Information about one relationship SHOULD NOT contaminate another relationship context without accountable relevance.

17.35 Domain-boundary scenario

Domain-specific state SHOULD remain appropriately bounded while still supporting authorized cross-domain synthesis.

17.36 Provenance scenario

Tests SHOULD verify that original observations, imported evidence, operator interventions, model derivatives, summaries, and migrated state remain distinguishable.

17.37 Circular-evidence scenario

Validation SHOULD verify that summaries, replicas, restatements, or model-generated derivatives do not become independent confirmation of their own sources.

17.38 Confidence scenario

Tests SHOULD verify that confidence changes follow accountable evidence and are not increased merely by repetition, computation cost, or secure transport.

17.39 Temporal scenario

Validation SHOULD exercise event time, observation time, processing time, migration time, and delayed execution where temporal distinctions affect meaning.

17.40 Ordering scenario

Concurrent or delayed events SHOULD be tested where causal order matters more than simple timestamp order.

17.41 Session-boundary scenario

Ending and resuming technical sessions SHOULD preserve authorized longitudinal continuity without treating session boundaries as member transformation.

17.42 Model-change scenario

Validation SHOULD compare relevant LGT behavior before and after a model change, including uncertainty, personalization, instruction following, tool use, and correction handling.

17.43 Prompt-change scenario

Material prompt or policy changes SHOULD be tested for normative regressions similarly to application code changes.

17.44 Threshold-change scenario

A changed scoring threshold SHOULD NOT be interpreted as new member evidence.

17.45 Migration scenario

Tests SHOULD verify semantic invariants across schema, storage, model, provider, index, or infrastructure migrations.

17.46 Rollback scenario

Validation SHOULD verify that software rollback does not roll back newer authoritative corrections, revocations, consent changes, or lived events.

17.47 Restore scenario

Backup restoration SHOULD be tested for reconciliation with newer member-control and lifecycle state.

17.48 Failover scenario

Regional or service failover SHOULD preserve identity, authorization, provenance, lifecycle, and correction semantics.

17.49 Partial-failure scenario

Validation SHOULD verify predictable narrowing when one component fails while unrelated continuity remains intact.

17.50 Recovery scenario

Recovery tests SHOULD verify reconciliation of affected state rather than only restored service availability.

17.51 Overload scenario

Load testing SHOULD verify that resource pressure narrows optional sophistication before foundational authorization, privacy, correction, or evidence integrity.

17.52 Cache-staleness scenario

Tests SHOULD verify that corrected, revoked, forgotten, or superseded state does not remain active solely because a cache is stale.

17.53 Replica-divergence scenario

Validation SHOULD exercise temporary replica divergence and verify appropriate consistency or reconciliation for the affected consequence.

17.54 Queue-delay scenario

Delayed work SHOULD be reassessed for changed authorization, member state, or purpose before material execution.

17.55 Replay scenario

Duplicate event or queue delivery SHOULD NOT create duplicate evidence, interventions, messages, charges, or irreversible actions.

17.56 Interoperability testing

Independent implementations claiming interoperability SHOULD test semantic exchange, authorization, provenance, lifecycle, version negotiation, and failure handling - not merely connection success.

17.57 Cross-version interoperability

Testing SHOULD include supported mixed-version operation during declared compatibility windows.

17.58 Unknown-version behavior

Unsupported newer semantics SHOULD be rejected, preserved, or narrowed according to contract rather than silently reinterpreted.

17.59 Round-trip testing

Where state can be exported and re-imported, round-trip tests SHOULD verify preservation of material semantics and provenance.

17.60 Translation testing

Semantic translation between implementations SHOULD be tested for declared losses, approximations, and unsupported concepts.

17.61 Translation loss

A lossy translation SHALL NOT be reported as lossless conformance.

17.62 External-provider testing

External model, storage, identity, or service dependencies SHOULD be tested at the boundaries relevant to their LGT responsibility.

17.63 Provider substitution

A substituted provider SHOULD be revalidated where the substitution can materially affect normative behavior.

17.64 Security validation

Security testing SHOULD verify trust boundaries, authentication, authorization, least privilege, tenant isolation, secrets handling, injection resistance, tool scoping, integrity, and recovery according to Section 14.

17.65 Prompt-injection testing

Model-enabled components SHOULD be tested against untrusted content attempting to redefine governing instructions, broaden authority, exfiltrate context, or invoke unauthorized tools.

17.66 Tool-abuse testing

Tests SHOULD verify that model requests cannot bypass tool authorization or use a privileged component as a confused deputy.

17.67 Data-exfiltration testing

Validation SHOULD exercise outputs, logs, errors, exports, integrations, and model/tool paths capable of disclosing protected information.

17.68 Supply-chain validation

Implementations SHOULD validate dependency and artifact controls appropriate to the claimed security profile.

17.69 Privacy validation

Privacy tests SHOULD verify purpose limitation, minimum-necessary use, member control, retention, deletion, forgetting, export, and revocation where applicable.

17.70 Purpose-boundary testing

Information authorized for one purpose SHOULD be tested against unauthorized reuse for another purpose.

17.71 Consent-boundary testing

Technical access SHALL NOT be treated as proof of consent during validation.

17.72 Member-control testing

Correction, export, deletion, refusal, revocation, and other supported member controls SHOULD be tested as first-class workflows rather than edge cases.

17.73 Observability validation

Tests SHOULD verify that material outcomes can be accounted for through provenance, dependencies, versions, authorization, uncertainty, and transformations without requiring private chain-of-thought.

17.74 Explanation validation

Member-facing explanations SHOULD be tested for traceability, truthful uncertainty, and absence of post-hoc rationale laundering.

17.75 Audit validation

Audit records SHOULD be tested for completeness appropriate to consequence while preserving privacy minimization.

17.76 Performance validation

Performance tests SHOULD measure latency, throughput, resource use, retrieval quality, correction propagation, and degradation behavior appropriate to Section 15.

17.77 Quality under load

Load tests SHOULD evaluate whether personalization, contradiction retrieval, provenance, and lifecycle correctness degrade under pressure.

17.78 Resource-induced bias testing

Implementations SHOULD test whether longer, more complex, multilingual, or less common member histories receive systematically poorer continuity because they are more expensive to process.

17.79 Accessibility validation

Member-facing implementations SHOULD test accessibility properties relevant to the interfaces through which members understand, correct, consent, refuse, and navigate.

17.80 Presentation validation

Visual and interaction testing SHOULD verify that presentation does not hide material status, controls, uncertainty, or distinctions required by the specification.

17.81 Human review

Human review SHOULD be used where automated testing cannot reliably establish semantic coherence, presentation quality, accessibility, or member-facing interpretability.

17.82 Human review limitations

Human approval SHALL NOT override a reproducible normative failure merely because the output appears acceptable.

17.83 Automated testing

Automated tests SHOULD protect repeatable invariants and regression-prone responsibilities.

17.84 Automated testing limitations

Passing automated tests SHALL NOT excuse missing validation of properties the tests do not measure.

17.85 Reference test vectors

LGT MAY publish reference test vectors for portable normative behaviors.

17.86 Reference fixtures

Reference fixtures SHOULD include both expected-success and expected-failure cases.

17.87 Canonical examples

Canonical examples MAY clarify intended semantics but SHALL NOT restrict conforming implementations to the example's technology or user interface.

17.88 Adversarial fixtures

Reference validation SHOULD include malicious, malformed, contradictory, stale, duplicated, unauthorized, and semantically incompatible inputs.

17.89 Longitudinal fixtures

Reference fixtures SHOULD include multi-stage member journeys that evolve over time rather than only isolated requests.

17.90 Conformance matrix

Implementations SHOULD maintain a matrix or equivalent mapping of applicable normative requirements to validation evidence.

17.91 Evidence artifact

A conformance evidence artifact MAY include test reports, version manifests, architecture mappings, security assessments, interoperability results, migration reports, and documented exceptions.

17.92 Evidence freshness

Conformance evidence SHOULD be refreshed when material implementation changes invalidate its assumptions.

17.93 Evidence provenance

Validation evidence SHOULD identify the environment, version, configuration, model, profile, and test inputs necessary to understand what was actually tested.

17.94 Reproducibility

Machine-verifiable tests SHOULD be reproducible to a reasonable degree where deterministic behavior is expected.

17.95 Probabilistic systems

For nondeterministic components, validation SHOULD use repeated trials, bounded acceptance criteria, invariant checks, or other methods appropriate to probabilistic behavior.

17.96 Statistical claims

Statistical conformance claims SHOULD identify sample size, acceptance threshold, measured property, and relevant uncertainty.

17.97 Benchmark separation

Generic model benchmarks MAY inform capability selection but SHALL NOT substitute for LGT-specific conformance testing.

17.98 Golden-output limitation

Exact string matching SHOULD NOT be the sole validation method for generative behavior when multiple outputs can satisfy the same normative responsibility.

17.99 Invariant testing

Generative tests SHOULD emphasize required invariants, prohibited behaviors, evidence use, uncertainty, authorization, and semantic outcomes where exact wording is not normative.

17.100 Differential testing

Old and new implementations MAY be compared to detect behavioral changes, but the older behavior SHALL NOT automatically define correctness.

17.101 Regression corpus

Implementations SHOULD maintain representative regression cases for previously discovered material failures.

17.102 Failure-derived tests

A corrected production failure SHOULD become a regression test where practical and where recurrence would be material.

17.103 Test isolation

Tests SHOULD isolate the responsibility under examination sufficiently to distinguish implementation failure from unrelated dependency failure.

17.104 Integration testing

Component interactions SHOULD be tested where individually conforming components can produce non-conforming combined behavior.

17.105 End-to-end testing

Representative end-to-end member journeys SHOULD verify that normative properties survive across the full path from input through continuity, guidance, action, correction, and later retrieval.

17.106 Contract testing

Component contracts SHOULD be tested independently where interoperability or deployment sequencing depends on them.

17.107 Property-based testing

Property-based or generative testing MAY be used to explore large state spaces and boundary combinations.

17.108 Fuzz testing

Parsers, importers, APIs, and structured interfaces MAY use fuzz testing to identify malformed-input failures.

17.109 Mutation testing

Mutation testing MAY assess whether a test suite can detect meaningful implementation faults.

17.110 Chaos testing

Resilient deployments MAY use controlled fault injection to verify containment, degradation, and recovery behavior.

17.111 Chaos boundaries

Chaos testing SHALL NOT expose real member information or production continuity to unjustified risk.

17.112 Production validation

Some properties MAY require bounded production observation, but production experimentation SHALL remain authorized, reversible where appropriate, and distinguishable from member evidence.

17.113 Synthetic monitoring

Synthetic probes MAY validate service behavior but SHALL remain operational data.

17.114 Shadow validation

Shadow systems MAY compare candidate behavior against production inputs when authorized, but candidate outputs SHALL NOT affect members before promotion.

17.115 Canary validation

Canary results SHOULD be evaluated for both operational and normative regressions before broader rollout.

17.116 Acceptance criteria

Each material test category SHOULD define what constitutes pass, fail, inconclusive, or accepted bounded variance.

17.117 Inconclusive result

An inconclusive test SHALL NOT be reported as a pass.

17.118 Bounded variance

A known variance MAY be accepted when it is understood, non-material to applicable requirements, and documented.

17.119 Pagination variance

Document pagination or renderer variance MAY be accepted when complete content, structure, navigation, readability, and normative meaning remain intact.

17.120 Failure severity

Failures SHOULD be classified according to affected normative responsibility and consequence rather than only implementation inconvenience.

17.121 Release-blocking failure

A failure affecting foundational member control, evidence integrity, authorization, privacy, correction, identity isolation, or other declared high-consequence responsibility SHOULD block promotion until resolved or explicitly removed from scope.

17.122 Non-blocking defect

A defect MAY be non-blocking when its impact is bounded, understood, does not violate an applicable normative requirement, and is tracked appropriately.

17.123 Waiver

A temporary waiver of a SHOULD-level or profile-permitted requirement SHOULD be explicit, scoped, justified, and revisited.

17.124 No waiver of truth

An implementation SHALL NOT waive a requirement in a manner that permits knowingly false claims about member evidence, consent, or conformance.

17.125 Conformance level

LGT MAY define conformance levels or profiles that vary required capabilities and assurance according to consequence.

17.126 Profile declaration

An implementation claiming a profile SHOULD identify the exact profile version.

17.127 Partial conformance

A product MAY claim conformance for a bounded component or capability if the boundary is clear and the claim does not imply whole-system conformance.

17.128 Composite conformance

A system composed of individually conforming components SHALL NOT automatically be considered conforming as a whole without validating their interactions.

17.129 Dependency conformance

Use of a conforming dependency MAY support evidence but SHALL NOT transfer conformance to the consuming system automatically.

17.130 Reference implementation

ConjuresUp MAY serve as a reference implementation and source of implementation learning but SHALL NOT be the sole oracle of LGT conformance.

17.131 Reference implementation divergence

A reference implementation MAY contain product-specific behavior not required by the specification.

17.132 Specification authority

Where a reference implementation and normative specification differ, the applicable normative publication governs the conformance claim.

17.133 Test-suite authority

An official test suite MAY operationalize normative requirements but SHALL NOT silently redefine them.

17.134 Test-suite defect

A defective conformance test SHOULD be corrected without treating implementation behavior that merely satisfies the defective test as normative truth.

17.135 Specification ambiguity

If a normative requirement cannot be tested because its meaning is materially ambiguous, the ambiguity SHOULD be resolved in the specification rather than hidden in implementation-specific test logic.

17.136 Interpretive record

Material conformance interpretations SHOULD be documented sufficiently to support consistency across implementations.

17.137 Certification

A certification program MAY attest that an implementation satisfied a defined validation process at a defined time.

17.138 Certification limitation

Certification SHALL NOT be represented as proof that an implementation can never fail, regress, be compromised, or produce an incorrect interpretation.

17.139 Certification expiry

Certification MAY expire or require reassessment after time or material change.

17.140 Self-attestation

Self-attestation MAY be permitted for appropriate profiles if scope, evidence, and responsibility are explicit.

17.141 Independent assessment

Higher-consequence profiles MAY require independent review of selected controls or conformance evidence.

17.142 Assessor independence

An independent assessor SHOULD disclose material conflicts that could compromise the credibility of the assessment.

17.143 Conformance report

A conformance report SHOULD identify scope, version, profile, applicable requirements, validation methods, results, exceptions, and known limitations.

17.144 Public claim

Public conformance language SHOULD match the actual validated scope and SHALL NOT imply capabilities or assurance that were not assessed.

17.145 Claim revocation

A conformance claim SHOULD be withdrawn or qualified when a material regression invalidates the evidence supporting it.

17.146 Claim restoration

A withdrawn claim MAY be restored after the affected scope is corrected and appropriately revalidated.

17.147 Continuous conformance

Implementations MAY use continuous testing and monitoring to maintain confidence between formal assessments.

17.148 Continuous conformance limitation

Continuous monitoring SHALL NOT eliminate the need for targeted revalidation after material architectural change.

17.149 Drift detection

Validation SHOULD detect configuration, model, dependency, schema, or policy drift capable of invalidating prior evidence.

17.150 Production evidence

Production telemetry MAY support conformance monitoring but SHALL remain subject to privacy, security, and operational-data boundaries.

17.151 Member reports

Member-reported failures SHOULD be eligible to trigger investigation and regression testing without automatically being treated as proof of system-wide nonconformance.

17.152 Disputed result

A disputed result SHOULD preserve enough evidence to determine whether the issue arose from member evidence, system interpretation, software change, incomplete retrieval, or another cause.

17.153 Correction after validation

Discovering a test or conformance error SHOULD result in corrected records rather than preserving an inaccurate pass for institutional convenience.

17.154 Conformance history

Historical conformance status MAY be preserved for accountability even after a newer version fails or supersedes it.

17.155 No conformance laundering

A non-conforming component SHALL NOT become conforming merely because it is wrapped by a conforming interface, deployed in a secure environment, or certified for an unrelated standard.

17.156 No benchmark laundering

High benchmark scores SHALL NOT be represented as LGT conformance without LGT-specific evidence.

17.157 No documentation laundering

Complete documentation SHALL NOT substitute for working normative behavior.

17.158 No security laundering

Security certification SHALL NOT prove epistemic, consent, memory, or continuity conformance.

17.159 No reference-implementation laundering

Matching ConjuresUp behavior SHALL NOT establish conformance when the matched behavior is product-specific or non-normative.

17.160 No test gaming

Implementations SHOULD NOT optimize narrowly for published test cases while violating the underlying normative responsibility in untested conditions.

17.161 Adversarial conformance review

Conformance programs SHOULD include attempts to find counterexamples to claimed compliance rather than only demonstrations designed to pass.

17.162 Evidence preservation

Material conformance evidence SHOULD be retained for a period appropriate to the claim, version lifecycle, and consequence.

17.163 Evidence privacy

Test artifacts containing member information SHALL remain subject to applicable privacy and security controls.

17.164 Test-data minimization

Real member information SHOULD NOT be used for validation when synthetic or appropriately de-identified data can establish the same property.

17.165 De-identification limitation

De-identification SHALL NOT be assumed irreversible when longitudinal combinations could re-identify a person.

17.166 Test-environment isolation

Validation environments SHOULD prevent test actions from unintentionally affecting production member continuity.

17.167 Production-like environment

Where environment differences materially affect behavior, validation SHOULD use a sufficiently production-like environment or explicitly account for the difference.

17.168 Cross-renderer verification

Publication or interface artifacts MAY require verification across relevant renderers when rendering differences can affect usability or normative content.

17.169 Publication verification

Normative LGT publications SHOULD be checked for section completeness, navigation, readable rendering, version identity, and absence of accidental blank or missing content before promotion.

17.170 Source-publication parity

Where multiple publication formats exist, they SHOULD contain substantively equivalent normative content even when pagination differs.

17.171 Source authority

The publication program SHOULD identify which source representation governs if formatting artifacts create ambiguity.

17.172 Change validation

Every material specification increment SHOULD be reviewed against the prior approved baseline for accidental deletion, renumbering error, contradiction, or scope drift.

17.173 Cross-publication validation

LGT publications SHOULD be reviewed for contradictory definitions, duplicated authority, incompatible lifecycle rules, and circular dependencies.

17.174 Normative ownership

A requirement SHOULD have a clear normative home so conformance does not depend on resolving competing duplicate rules across publications.

17.175 Reference integrity

Cross-references SHOULD identify valid publications, sections, or concepts and SHOULD be updated when normative structure changes.

17.176 Terminology validation

Defined terms SHOULD be used consistently enough that implementations can determine when two requirements refer to the same architectural concept.

17.177 Constitution alignment

Reference Architecture conformance SHALL remain subordinate to the LGT Constitution where the Constitution establishes foundational boundaries.

17.178 Evidence-model alignment

Testing SHALL preserve the distinction between observation, evidence, interpretation, confidence, memory, and derived guidance established by the LGT framework.

17.179 Memory-specification alignment

Detailed memory tests SHOULD defer to LGT-MEM-001 where that publication owns the normative memory behavior.

17.180 Interoperability-specification alignment

Detailed exchange tests SHOULD defer to the applicable interoperability publication where it owns protocol semantics.

17.181 Conformance and security

Testing SHALL preserve the distinction between technical trust and epistemic authority established by Section 14.

17.182 Conformance and performance

Performance validation SHALL preserve the correctness-first and truthful-degradation principles established by Section 15.

17.183 Conformance and operations

Release validation SHALL preserve the known-good-baseline and state-aware rollback principles established by Section 16.

17.184 Member-centered test design

Test design SHOULD ask whether the implementation preserves the member's lived continuity, agency, corrections, uncertainty, and genuine change - not merely whether components returned expected status codes.

17.185 Member simulation

Synthetic member journeys MAY be used to test longitudinal behavior but SHALL remain synthetic operational artifacts rather than evidence about real people.

17.186 Diverse journeys

Validation SHOULD include journeys with different lengths, rhythms, contradictions, preferences, outcomes, and patterns so conformance does not assume one canonical human trajectory.

17.187 No destiny fixture

A test suite SHALL NOT require the Guide to force a predetermined developmental outcome on every member in order to pass.

17.188 No therapeutic authority fixture

A conformance test SHALL NOT reward behavior that turns the Guide into an unqualified therapist, oracle, or authority over the member.

17.189 Observation-first fixture

Validation SHOULD include scenarios where the correct behavior is to continue observing rather than conclude prematurely.

17.190 Evidence-update fixture

A Guide SHOULD be tested on its ability to revise a prior interpretation when new evidence weakens or overturns it.

17.191 Member-disagreement fixture

Validation SHOULD include a member disagreeing with the Guide and verify that disagreement can become relevant evidence or correction without coercive defense of the prior model.

17.192 Setback fixture

A setback SHOULD NOT automatically erase prior growth or be interpreted as destiny; tests SHOULD preserve longitudinal nuance.

17.193 Success fixture

A successful practice or outcome SHOULD inform continuity without becoming universal proof that the same intervention will always work.

17.194 Recommendation traceability fixture

A recommendation SHOULD be testable for traceability to accumulated member evidence, constraints, and current context at the level required by its consequence.

17.195 Practice-outcome fixture

Where the Practice Engine is present, tests SHOULD verify that completed experiences and outcomes can inform later Living Guide understanding without reducing the member to click history.

17.196 Continuity Intelligence fixture

A longitudinal conformance scenario SHOULD demonstrate that meaning emerges across connected lived experiences rather than from isolated transactions alone.

17.197 Reference-architecture verification

The Reference Architecture itself SHOULD be reviewed as an integrated system before v1.0.0 to verify that all normative responsibilities have a coherent place, boundary, and conformance path.

17.198 Holistic review

Pre-v1.0 review SHOULD evaluate gaps, contradictions, duplicated authority, undefined terms, orphaned responsibilities, missing failure behavior, and incompatible cross-section assumptions.

17.199 No section-count target

Additional sections SHOULD be added only when a genuine architectural responsibility remains uncovered, not to reach an arbitrary publication size or version count.

17.200 Release-candidate gate

The Reference Architecture SHOULD enter release-candidate status only when holistic review finds no known material architectural gap that should be resolved before v1.0.0.

17.201 Technology neutrality

LGT-ARCH-001 does not prescribe a testing framework, CI system, programming language, benchmark harness, certification body, QA platform, model evaluator, or assessment vendor.

17.202 Conformance

A conforming implementation SHALL be able to produce credible evidence that its claimed LGT responsibilities were validated against the applicable normative requirements and declared profile.

17.203 Verification closing principle

A Living Guide should earn trust through behavior that can withstand examination. Conformance is not a badge, benchmark, or promise that failure is impossible; it is an accountable body of evidence showing that the implementation was tested against the responsibilities it claims, including the difficult cases where continuity, uncertainty, correction, and human change matter most.

18. Architectural Profiles, Extension Points, Implementation Boundaries, and Capability Declaration

18.1 Purpose

LGT SHALL permit independent implementations to specialize, extend, and deploy the architecture in materially different environments without silently redefining its constitutional core, normative semantics, or member-centered responsibilities.

18.2 Architecture versus product

LGT defines architectural responsibilities and boundaries; an LGT implementation MAY provide product-specific experiences, interfaces, business models, content, terminology, and workflows beyond those requirements.

18.3 Reference implementation independence

ConjuresUp is the first reference implementation of LGT but SHALL NOT define the only valid implementation shape.

18.4 Constitutional invariants

No profile, extension, product feature, deployment model, or capability declaration SHALL override the foundational constraints established by the LGT Constitution.

18.5 Normative invariants

An extension SHALL NOT redefine an applicable SHALL requirement into optional behavior merely because an implementation prefers a different design.

18.6 Specialization

An implementation MAY specialize how a normative responsibility is fulfilled when the specialized mechanism preserves the responsibility's intended semantics.

18.7 Implementation freedom

LGT SHALL remain sufficiently technology-neutral to permit different programming languages, storage systems, models, user interfaces, deployment platforms, organizational structures, and business models.

18.8 Freedom is bounded by responsibility

Technology neutrality does not permit an implementation to discard accountability, evidence integrity, member agency, correction, privacy, security, provenance, or other applicable normative responsibilities.

18.9 Architectural profile

An architectural profile is a declared subset and specialization of LGT capabilities, consequences, interfaces, and assurance requirements for a defined implementation context.

18.10 Profile purpose

Profiles SHOULD make conformance more precise by identifying which optional capabilities are present and which consequence-specific requirements apply.

18.11 Profile version

A profile SHOULD have an identifiable version so conformance claims can be interpreted against the exact requirements in force.

18.12 Profile inheritance

A specialized profile MAY inherit another profile and add requirements, but SHALL NOT silently weaken inherited SHALL requirements.

18.13 Profile restriction

A profile MAY prohibit optional capabilities that would be inappropriate for its context.

18.14 Profile expansion

A profile MAY require capabilities that are optional in the base architecture when its consequence or use case warrants them.

18.15 Profile naming

Profile names SHOULD describe the implementation context or assurance boundary without implying capabilities that are not actually required.

18.16 Profile declaration

An implementation claiming an LGT profile SHOULD declare the profile identifier, version, supported capabilities, exclusions, and known limitations.

18.17 Base profile

LGT MAY define a base profile representing the minimum coherent set of capabilities required for a general Living Guide implementation.

18.18 Higher-consequence profile

LGT MAY define higher-consequence profiles with stronger requirements for validation, security, human review, explanation, recovery, or operational assurance.

18.19 Domain profile

A domain-specific profile MAY add constraints appropriate to a domain while preserving the core distinction between observation, evidence, interpretation, confidence, and member authority.

18.20 Deployment profile

A deployment profile MAY define requirements appropriate to local, cloud, enterprise, embedded, offline, federated, or other environments.

18.21 Interaction profile

An interaction profile MAY define requirements for conversational, dashboard, wearable, ambient, voice, or other member interfaces.

18.22 Memory profile

A profile MAY restrict or require memory capabilities, but detailed memory semantics remain governed by the applicable memory specification.

18.23 Interoperability profile

A profile MAY require particular exchange capabilities, semantic versions, or portability guarantees.

18.24 Privacy profile

A profile MAY impose stronger privacy or locality constraints than the base architecture.

18.25 Security profile

A profile MAY impose stronger authentication, isolation, cryptographic, review, or recovery requirements according to consequence.

18.26 Profile composition

Multiple compatible profiles MAY be composed when their combined requirements do not conflict.

18.27 Profile conflict

If two profiles impose incompatible requirements, the implementation SHALL NOT claim both without an explicit resolution defined by the governing specifications.

18.28 Profile precedence

Profile precedence SHOULD be explicit where layered profiles can impose different constraints on the same responsibility.

18.29 Profile conformance

Conformance to a profile SHALL require conformance to all applicable base requirements plus the profile's additional requirements.

18.30 Partial profile claim

An implementation SHALL NOT claim full profile conformance when it implements only selected profile capabilities.

18.31 Capability

A capability is a declared function or responsibility that an implementation can perform within defined constraints.

18.32 Capability declaration

Implementations SHOULD expose or document a capability declaration sufficient for members, operators, integrators, or conformance tooling to determine what the implementation actually supports.

18.33 Capability identity

A portable capability SHOULD have a stable identifier or unambiguous semantic name.

18.34 Capability version

A capability MAY be independently versioned where its contract evolves separately from the overall implementation.

18.35 Capability status

A capability declaration SHOULD distinguish available, unavailable, experimental, deprecated, restricted, or otherwise materially limited states where relevant.

18.36 Capability conditions

A capability MAY depend on member authorization, product tier, jurisdiction, device support, provider availability, profile, or other declared conditions.

18.37 Capability condition is not evidence

A capability condition SHALL NOT be interpreted as evidence about the member's identity, character, motivation, or development.

18.38 Capability scope

A declaration SHOULD identify the domain, consequence, data classes, interfaces, or member controls relevant to a capability where ambiguity would matter.

18.39 Capability assurance

A declaration MAY identify the validation or assurance level associated with a capability.

18.40 Capability dependency

A capability SHOULD identify material dependencies when their absence changes whether the capability can safely operate.

18.41 Capability degradation

A capability declaration SHOULD permit degraded or temporarily unavailable states without falsely representing full functionality.

18.42 Capability discovery

Independent components MAY discover capabilities dynamically where interoperability benefits from runtime negotiation.

18.43 Capability negotiation

Components SHOULD negotiate compatible capabilities rather than assuming that another LGT implementation supports every optional feature.

18.44 Capability mismatch

A capability mismatch SHOULD result in narrowing, translation, preservation, or explicit rejection rather than silent semantic loss.

18.45 Capability absence

Absence of a capability SHALL NOT imply that the implementation is non-conforming if the capability is optional and outside its declared profile.

18.46 Capability overclaim

An implementation SHALL NOT advertise a capability whose applicable normative responsibilities it does not satisfy.

18.47 Capability revocation

A capability MAY be withdrawn or disabled when safety, security, provider, legal, or operational conditions require it.

18.48 Capability revocation is not member change

Withdrawal of a system capability SHALL NOT be represented as a change in the member.

18.49 Capability portability

Portable capability declarations SHOULD use semantics independent of product-specific UI labels.

18.50 Implementation boundary

An implementation boundary defines which components, services, models, stores, operators, providers, and interfaces participate in a declared LGT responsibility.

18.51 Boundary declaration

Material implementation boundaries SHOULD be identifiable for conformance, security, privacy, interoperability, and operational ownership.

18.52 Boundary crossing

Information crossing an implementation boundary SHOULD retain the provenance, authorization, lifecycle, semantic, and security context required by its downstream use.

18.53 Boundary does not erase responsibility

Outsourcing a component or crossing a vendor boundary SHALL NOT erase the implementation's responsibility for the LGT behavior it claims.

18.54 External provider

An external provider MAY perform a bounded LGT function without becoming authoritative over the member's entire continuity.

18.55 Provider contract

Material external providers SHOULD have a defined technical and semantic contract appropriate to the responsibility delegated to them.

18.56 Provider limitation

A provider's technical success response SHALL NOT prove that its output is epistemically correct or authorized for every downstream purpose.

18.57 Provider substitution

An implementation SHOULD be able to replace a provider without changing member truth solely because the underlying vendor changed.

18.58 Provider-specific metadata

Provider-specific metadata MAY be retained for operations or provenance but SHOULD not become portable member evidence unless semantically justified.

18.59 Model boundary

A model SHALL be treated as a bounded reasoning or generation component rather than the sole authoritative holder of member continuity.

18.60 Model plurality

Different models MAY perform different LGT capabilities within one implementation.

18.61 Model specialization

A specialized model MAY be used for a bounded domain or task when its limitations and authority remain appropriately constrained.

18.62 Model output boundary

Model output SHOULD enter governed continuity according to its evidentiary role rather than being treated as member-originated truth.

18.63 Tool boundary

Tools MAY perform actions or retrieve information, but their access and outputs SHALL remain bounded by authorization, purpose, provenance, and consequence.

18.64 Human boundary

Human operators, reviewers, practitioners, or administrators MAY participate in an LGT implementation without automatically becoming epistemic authorities over the member.

18.65 Human-originated state

Human interventions SHOULD be distinguishable from member-originated evidence and model-generated derivatives.

18.66 Organizational boundary

Multiple organizations MAY jointly provide an LGT system, but responsibility boundaries SHOULD be explicit enough to prevent gaps in member control, security, correction, or recovery.

18.67 Device boundary

Device-local components MAY participate in observation or continuity while preserving the applicable authorization and synchronization semantics.

18.68 Offline boundary

Offline components SHOULD distinguish locally available continuity from globally reconciled continuity where the distinction matters.

18.69 Edge boundary

Edge processing MAY reduce latency or disclosure but SHALL NOT weaken correction, lifecycle, or provenance responsibilities.

18.70 Tenant boundary

Multi-tenant implementations SHALL preserve isolation and SHALL NOT treat shared infrastructure as shared member continuity.

18.71 Account boundary

An account identifier MAY support identity resolution but SHALL NOT by itself define every person, profile, relationship, or role represented within the account.

18.72 Profile boundary

Multiple member profiles SHOULD remain distinct unless an authorized merge or relationship explicitly connects them.

18.73 Domain boundary

Domains MAY maintain specialized semantics and components while preserving accountable cross-domain synthesis.

18.74 Relationship boundary

Relationship-specific continuity SHOULD remain appropriately scoped to the relationship in which it was observed.

18.75 Extension point

An extension point is an explicitly permitted location where an implementation or external specification may add behavior without redefining the core contract.

18.76 Extension-point declaration

Normative extension points SHOULD identify what may be extended, what invariants must remain, and how unsupported extensions are handled.

18.77 Extension namespace

Portable extensions SHOULD use identifiers or namespaces that minimize collision with core LGT semantics and other extensions.

18.78 Extension version

An extension SHOULD identify its version when semantics may evolve.

18.79 Extension ownership

An extension SHOULD identify the specification, organization, or implementation responsible for defining its semantics.

18.80 Extension documentation

An extension SHOULD document its purpose, data semantics, capability dependencies, member impact, lifecycle, and interoperability behavior where material.

18.81 Extension conformance

An extension MAY define additional conformance requirements without weakening applicable base requirements.

18.82 Extension isolation

A faulty or unsupported extension SHOULD be containable without corrupting unrelated core continuity.

18.83 Extension failure

Failure of an optional extension SHOULD degrade the extension before it degrades foundational LGT responsibilities.

18.84 Unknown extension

An implementation encountering an unknown extension SHOULD preserve, ignore, reject, or quarantine it according to contract rather than guessing its meaning.

18.85 Extension preservation

Unknown extension data SHOULD be preserved across round trips when practical and when preservation does not violate privacy, security, or lifecycle requirements.

18.86 Extension translation

Translation of an extension SHOULD declare semantic loss when the destination cannot represent its full meaning.

18.87 Extension collision

Two extensions SHALL NOT silently assign incompatible meanings to the same portable identifier.

18.88 Extension promotion

A widely adopted extension MAY later become part of the core specification through normal governance and versioning.

18.89 Extension promotion is not retroactive

Promotion of an extension into the core SHALL NOT silently rewrite the historical semantics of older extension versions.

18.90 Private extension

An implementation MAY use private extensions internally when they do not masquerade as portable LGT semantics.

18.91 Experimental extension

Experimental extensions SHOULD be clearly identified and SHOULD NOT be presented as stable portable contracts.

18.92 Product-specific extension

Product features MAY extend LGT while remaining explicitly product-specific.

18.93 Product vocabulary

A product MAY use member-friendly vocabulary different from specification terminology if the underlying normative mapping remains clear.

18.94 Product archetypes

A product MAY use archetypes, themes, chapters, badges, milestones, or similar constructs, but SHALL NOT allow them to become deterministic identities that override evidence or genuine change.

18.95 Product practices

A product MAY provide proprietary practices, rituals, exercises, or experiences while preserving the Practice Engine's role as a source of lived evidence rather than authority over the member.

18.96 Product content

Content libraries MAY support an LGT product but SHALL NOT by themselves constitute Continuity Intelligence.

18.97 Product recommendations

Recommendation systems MAY vary substantially among implementations, but member-facing recommendations SHOULD remain traceable to accountable evidence where required by consequence.

18.98 Product business model

Free, subscription, enterprise, nonprofit, research, local-device, or other business models MAY implement LGT.

18.99 Commercial boundary

Commercial entitlements MAY determine feature availability but SHALL NOT redefine known member truth or fabricate evidence.

18.100 Advertising boundary

An implementation that includes advertising or sponsorship SHOULD preserve a clear boundary between commercial influence and evidence-based member guidance.

18.101 Sponsored content

Sponsored or commercially influenced content SHALL NOT be represented as a personalized conclusion derived from the member's continuity unless it independently satisfies the same evidence and disclosure requirements.

18.102 Analytics boundary

Product analytics MAY measure feature use and operations but SHALL NOT automatically become member evidence.

18.103 Research boundary

Research uses MAY analyze appropriately governed data, but research findings SHALL NOT automatically become individualized member truth.

18.104 Population-learning boundary

Population-level learning MAY inform priors or system design where authorized, but member-specific evidence SHOULD supersede generic population assumptions when sufficient personal evidence exists.

18.105 Import boundary

Imported information SHOULD declare source, authority, confidence, lifecycle, and semantic compatibility appropriate to its use.

18.106 Export boundary

Exported continuity SHOULD preserve enough semantics and provenance for the recipient to avoid treating derivatives as original evidence.

18.107 Legacy boundary

Legacy systems MAY participate through adapters when semantic loss and unsupported responsibilities are explicit.

18.108 Adapter

An adapter MAY translate syntax or semantics but SHALL NOT silently manufacture information required by the destination contract.

18.109 Adapter provenance

Material transformations performed by an adapter SHOULD remain attributable.

18.110 Gateway

A gateway MAY enforce authentication, routing, rate limits, or policy but SHALL NOT become epistemic authority merely because all traffic passes through it.

18.111 Aggregator

An aggregator MAY combine multiple sources while preserving source identity and avoiding false independence.

18.112 Orchestrator

An orchestrator MAY coordinate components but SHALL NOT silently broaden their individual authority.

18.113 Agent boundary

Agentic components MAY plan or invoke tools within bounded authority, consequence, and member-control constraints.

18.114 Agent delegation

Delegation from one agent to another SHOULD preserve purpose, authorization, provenance, and applicable limitations.

18.115 Agent recursion

Recursive delegation SHALL NOT increase authority merely because a task passed through additional agents.

18.116 Agent memory

An agent's local working memory SHALL NOT automatically become durable member memory.

18.117 Capability composition

Complex capabilities MAY be composed from simpler capabilities when the combined behavior preserves all applicable responsibilities.

18.118 Composition is not authority accumulation

Combining multiple components SHALL NOT create greater epistemic or operational authority than the authorized composition actually permits.

18.119 Emergent behavior

Implementations SHOULD test composed systems for emergent behavior that is not visible from individual component contracts.

18.120 Hidden dependency

A material hidden dependency SHOULD be surfaced when its failure or policy can change a claimed capability.

18.121 Optional component

An optional component SHOULD be removable without breaking unrelated core responsibilities.

18.122 Required component

A profile MAY require a component category or responsibility without requiring a specific vendor or technology.

18.123 Replaceability

Core architecture SHOULD favor replaceable components where practical so member continuity does not become captive to one implementation dependency.

18.124 Portability

Portability SHOULD preserve semantic continuity, not merely export raw bytes.

18.125 Portability limitation

A destination implementation MAY lack capabilities needed to reproduce every source behavior; such limitations SHOULD be declared.

18.126 Minimum portable core

LGT MAY define a minimum portable continuity representation sufficient to preserve identity, provenance, lifecycle, corrections, and essential member-controlled history across conforming implementations.

18.127 Rich portability

Implementations MAY exchange richer optional state when compatible capabilities and extensions are available.

18.128 Semantic downgrade

A downgrade to a less expressive profile SHOULD preserve original state where possible and clearly identify unavailable interpretation or functionality.

18.129 Semantic upgrade

An upgrade to a richer profile MAY derive new interpretations from existing evidence but SHALL distinguish newly derived state from newly observed member evidence.

18.130 Capability fallback

When a preferred capability is unavailable, fallback behavior SHOULD remain within the declared profile and preserve truthful limitation.

18.131 Generic fallback

Generic guidance MAY replace unavailable personalization when necessary, but SHALL NOT be presented as though it used member-specific continuity that was unavailable.

18.132 No-op fallback

A safe no-op or defer response MAY be preferable to a semantically incorrect fallback.

18.133 Extension fallback

Unsupported optional extensions MAY be ignored only when doing so cannot materially distort the core meaning of the exchange.

18.134 Interface contract

A public LGT interface SHOULD define input, output, errors, authorization, versioning, and semantic expectations appropriate to its consequence.

18.135 Internal interface

Internal interfaces MAY be implementation-specific but SHOULD preserve normative semantics across component boundaries.

18.136 API stability

Portable APIs SHOULD declare stability and deprecation expectations appropriate to independent adoption.

18.137 Event stability

Portable events SHOULD maintain stable identity and lifecycle semantics across compatible versions.

18.138 Schema extension

Schemas SHOULD provide controlled extension mechanisms where ecosystem specialization is expected.

18.139 Reserved namespace

Core specifications MAY reserve identifiers to prevent collision with future normative semantics.

18.140 Unknown-field preservation

Forward-compatible implementations SHOULD preserve unknown portable fields where practical and safe.

18.141 Strict validation

Strict rejection MAY be appropriate when unknown data could alter authorization, security, lifecycle, or high-consequence meaning.

18.142 Lenient validation

Lenient preservation MAY be appropriate for optional descriptive extensions that cannot alter protected semantics.

18.143 Semantic versioning

Specifications, profiles, capabilities, and extensions SHOULD use versioning that communicates compatibility expectations.

18.144 Major change

A change that intentionally breaks portable semantic compatibility SHOULD require a version transition that makes the break explicit.

18.145 Minor change

A compatible addition MAY be represented as a minor evolution when older implementations can safely narrow or preserve the new semantics.

18.146 Patch change

Clarifications or corrections that do not intentionally change compatible semantics MAY be represented as patch-level evolution.

18.147 Version semantics are declared

LGT does not require one numeric versioning syntax, but compatibility meaning SHOULD be declared and consistent.

18.148 Capability manifest

A machine-readable capability manifest is RECOMMENDED for interoperable implementations.

18.149 Manifest content

A capability manifest SHOULD identify implementation/version, profiles, supported capabilities, relevant extensions, interface versions, and material limitations.

18.150 Manifest privacy

A capability manifest SHOULD describe system capability without exposing unnecessary member information or secrets.

18.151 Manifest authenticity

Where capability declarations affect trust or automated interoperability, their authenticity and integrity SHOULD be verifiable.

18.152 Manifest freshness

Runtime-discoverable manifests SHOULD distinguish current capability from historical or cached declarations when staleness matters.

18.153 Conformance manifest

An implementation MAY separately expose conformance claims and evidence references rather than mixing them with runtime capability discovery.

18.154 Deployment manifest

Operational deployments MAY identify which concrete components satisfy declared capabilities.

18.155 Member-facing capability disclosure

Member interfaces SHOULD disclose limitations when a missing capability materially changes what the Guide can understand, remember, explain, or do.

18.156 No implementation mystique

An implementation SHALL NOT imply that proprietary architecture grants supernatural certainty, destiny knowledge, or authority over the member.

18.157 No hidden oracle profile

No profile SHALL define the Living Guide as an infallible oracle.

18.158 No hidden therapist profile

No profile SHALL redefine the Living Guide as a therapist or clinical authority merely through product positioning.

18.159 No deterministic identity extension

An extension SHALL NOT convert probabilistic patterns into permanent member identity without accountable evidence and revisability.

18.160 No memory bypass extension

An extension SHALL NOT create durable member memory outside applicable memory governance merely to avoid lifecycle or member-control requirements.

18.161 No consent bypass extension

An extension SHALL NOT treat technical availability of information as permission for unrelated use.

18.162 No provenance bypass extension

An extension SHALL NOT erase source distinctions merely to simplify implementation.

18.163 No correction bypass extension

An extension SHALL NOT create derivatives that are intentionally unreachable by applicable correction or revocation.

18.164 No security bypass extension

An extension SHALL NOT weaken trust boundaries or tool authorization established by the core architecture.

18.165 No conformance bypass extension

An implementation SHALL NOT label non-conforming behavior as an extension in order to avoid an applicable core requirement.

18.166 Extension review

Extensions affecting evidence, memory, authorization, privacy, security, member agency, or high-consequence actions SHOULD receive review proportional to their impact.

18.167 Extension testing

Extensions SHOULD define validation sufficient to demonstrate preservation of core invariants.

18.168 Extension interoperability

Portable extensions SHOULD define how unsupported peers behave and how semantic loss is represented.

18.169 Extension lifecycle

Extensions SHOULD define introduction, versioning, deprecation, migration, and retirement behavior where durable state is involved.

18.170 Extension state ownership

Durable extension state SHOULD have an identifiable normative or product owner responsible for lifecycle semantics.

18.171 Extension migration

Extension migrations SHOULD preserve the same correction, provenance, authorization, and member-control principles applicable to core state.

18.172 Extension removal

Removing an extension SHOULD reconcile durable state rather than orphaning it without lifecycle meaning.

18.173 Experimental capability

Experimental capabilities SHOULD be clearly distinguishable from stable capabilities in conformance and interoperability claims.

18.174 Beta status

A beta label SHALL NOT excuse violation of foundational constitutional requirements when real member continuity is involved.

18.175 Research implementation

A research implementation MAY claim bounded experimental conformance when its limitations and non-production status are explicit.

18.176 Educational implementation

An educational implementation MAY omit production capabilities when it does not imply conformance beyond its declared scope.

18.177 Local-only implementation

A local-only implementation MAY satisfy LGT without cloud services when it fulfills the applicable responsibilities of its profile.

18.178 Cloud implementation

A cloud implementation MAY satisfy LGT when tenant isolation, privacy, security, lifecycle, and continuity responsibilities are preserved.

18.179 Hybrid implementation

A hybrid implementation SHOULD declare which state and capabilities reside locally versus remotely.

18.180 Federated implementation

A federated implementation SHOULD preserve source authority and member identity across participating nodes.

18.181 Decentralized implementation

A decentralized architecture MAY implement LGT if it can still provide accountable provenance, authorization, correction, lifecycle, and conformance evidence.

18.182 Embedded implementation

Embedded or wearable implementations MAY narrow capabilities according to device constraints without misrepresenting those constraints as member limitations.

18.183 Ambient implementation

Ambient observation capabilities SHOULD require especially clear purpose, authorization, privacy, and member-control boundaries.

18.184 Multimodal implementation

Text, voice, image, sensor, behavioral, or other modalities MAY contribute observations when their provenance and uncertainty are preserved.

18.185 Sensor extension

Sensor-derived information SHALL NOT be treated as direct proof of a psychological, spiritual, or personal interpretation merely because the sensor measurement is technically accurate.

18.186 Biomedical extension

Future biomedical integrations, if implemented, SHOULD maintain a strict boundary between measured signals and non-clinical Living Guide interpretation unless separately governed clinical authority exists.

18.187 Third-party content extension

Third-party content MAY enrich experiences but SHOULD remain distinguishable from conclusions derived from the member's own continuity.

18.188 Community extension

Community or social capabilities SHOULD preserve boundaries between another person's statements and evidence about the member.

18.189 Practitioner extension

A practitioner-facing extension MAY support human collaboration while preserving member agency and distinguishing practitioner judgment from Living Guide inference.

18.190 Enterprise extension

Enterprise deployments MAY add organizational controls but SHALL NOT allow organizational interests to silently override the member's applicable rights and evidence boundaries.

18.191 White-label implementation

A white-label product MAY implement LGT under another brand if its conformance claim accurately identifies the underlying specification and profile.

18.192 Fork

An implementation MAY fork an open specification or extension, but SHALL NOT continue to claim compatibility with unchanged LGT semantics after introducing incompatible normative changes.

18.193 Compatible derivative

A derivative implementation MAY add stricter requirements while remaining LGT-compatible if it preserves all applicable base semantics.

18.194 Incompatible derivative

An incompatible derivative SHOULD identify itself distinctly rather than using LGT terminology in a way that implies conformance.

18.195 Trademark and conformance

Brand or trademark permission, if separately governed, SHALL NOT substitute for technical or normative conformance.

18.196 Open ecosystem

LGT SHOULD support an ecosystem in which multiple independent implementations can interoperate, compete, specialize, and innovate without fragmenting the meaning of member continuity.

18.197 Innovation boundary

Innovation is encouraged at extension points, product layers, interfaces, models, experiences, and implementation mechanisms while constitutional and normative invariants remain stable.

18.198 Architectural humility

The Reference Architecture SHOULD define only what must be shared for accountable Living Guide behavior and leave implementation-specific choices outside the normative core where possible.

18.199 Profile governance

New official profiles SHOULD be added only when a recurring implementation context requires a coherent set of additional or restricted requirements.

18.200 Extension governance

New core extension points SHOULD be added only when independent specialization cannot be safely expressed through existing boundaries.

18.201 Capability-governance review

Before v1.0.0, the architecture SHOULD verify that every major optional subsystem can be declared, omitted, degraded, or extended without ambiguity about conformance.

18.202 Ecosystem conformance

Independent implementations SHOULD be able to determine compatibility from declared profiles, capabilities, versions, extensions, and limitations rather than relying on brand recognition or private agreements.

18.203 Reference Architecture boundary

LGT-ARCH-001 SHALL define architectural contracts and boundaries without absorbing detailed requirements that belong to specialized normative publications.

18.204 Specialized specification boundary

Specialized LGT publications MAY define detailed memory, interoperability, evidence, governance, or other semantics while remaining consistent with the Reference Architecture.

18.205 Cross-publication ownership

When a specialized publication owns a detailed responsibility, LGT-ARCH-001 SHOULD reference the boundary rather than duplicate competing normative text.

18.206 Future specification family

The architecture SHOULD permit additional normative publications to evolve without requiring the constitutional core to be rewritten for every new implementation domain.

18.207 No premature standardization

LGT SHOULD NOT standardize product-specific mechanisms merely because the reference implementation currently uses them.

18.208 No accidental monopoly

No vendor, model provider, cloud platform, content library, application, or reference implementation SHALL be architecturally required unless the normative function cannot be expressed independently of that technology.

18.209 Conformance

A conforming implementation SHALL accurately declare the profiles, capabilities, boundaries, extensions, and limitations relevant to its LGT claim and SHALL preserve applicable constitutional and normative invariants across them.

18.210 Extension closing principle

Living Guide Technology should be stable enough to mean the same thing across implementations and open enough to be implemented in ways its original authors did not predict. The architecture succeeds when innovation can occur around the member without allowing innovation to redefine the member, erase their authority, or fracture the continuity that belongs to them.

19. Architectural Baseline and Integrated Conformance**19.1 Minimum architectural distinction**

A conforming implementation SHALL be able to account for which architectural responsibility performed a material observation, evidence, memory, continuity, reflection, or companion function even when those responsibilities share infrastructure.

19.2 Accountable forward flow

A conforming implementation SHALL preserve an accountable path from eligible input through evidence and continuity to any material downstream use.

19.3 Accountable reverse flow

A conforming implementation SHALL support correction or reconciliation paths sufficient to prevent invalid upstream information from remaining silently authoritative downstream.

19.4 Authorization continuity

Authorization boundaries SHALL persist across transformations and downstream use. Derivation SHALL NOT create new authorization.

19.5 Provenance continuity

Material derived states SHOULD retain sufficient provenance to identify their accountable support and relevant system interventions.

19.6 Non-determinism

The architecture SHALL preserve uncertainty, member agency, chance, context change, and the possibility that prior continuity becomes less applicable.

19.7 No implementation lock-in

Conformance SHALL be evaluated by architectural behavior and responsibilities rather than similarity to ConjuresUp's technology stack.

19.8 Integrated closing baseline

Living Guide Technology is not a single model, database, memory store, recommendation engine, or interface. It is an accountable architecture in which distinct responsibilities cooperate to preserve continuity without surrendering evidence, correction, authorization, member change, or human agency.